

IF2224 TEORI BAHASA FORMAL DAN OTOMATA
LAPORAN TUGAS BESAR
MILESTONE 3



Disusun oleh:

Kelompok 01 – Kelas 01

Muhammad Jordan Ferimeison	(13524047)
Arina Azka	(13524049)
Hakam Avicena Mustain	(13524075)
Yavie Azka Putra Araly	(13524077)
Angelina Andra Alanna	(13524079)

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

2026

BAB I

LANDASAN TEORI

1.1. Semantic Analysis

Semantic analysis adalah tahap dalam front-end compiler yang memeriksa makna program setelah program sudah melewati parser dan dinyatakan memiliki sintaks yang benar. Semantic Analysis memiliki peran untuk memastikan program bermakna secara semantik sesuai aturan bahasa. Pada fase ini compiler menginterpretasikan hubungan antar entitas program (identifikasi, fungsi, tipe) yang tidak dapat ditangkap hanya oleh pemeriksaan sintaksis.

Semantic analysis melakukan validasi deklarasi dan penggunaan identifikasi, pengecekan tipe (*type checking*), inferensi tipe, validasi aturan semantik khusus bahasa seperti return pada fungsi, jumlah argumen pemanggilan fungsi. Hasil dari fase ini dapat berupa pesan kesalahan semantik atau anotasi pada struktur internal seperti AST (Abstract Syntax Error) yang menandakan tipe dan referensi simbol.

Semantic analysis berperan sebagai jembatan antara fase analisis (lexical analysis dan syntax analysis) dan fase sintesis (intermediate code generation dan optimasi). Tanpa verifikasi semantik yang benar, kode antara yang dihasilkan mungkin salah atau tidak dapat dioptimalkan secara aman.

1.2. Abstract Syntax Tree (AST)

Abstract Syntax Tree (AST) adalah representasi pohon abstrak dari struktur sintaksis program yang menghilangkan detail sintaksis yang tidak relevan (seperti tanda kurung atau token terminal yang redundan) dan hanya menyimpan konstruksi struktur program yang bermakna seperti ekspresi, pernyataan, deklarasi. AST lebih ringkas dan lebih mudah diproses untuk analisis semantik dan transformasi daripada parse tree penuh.

Perbedaan AST dengan parse tree (parse/derivation tree) terletak pada tingkat abstraksi. Parse tree adalah hasil parsing sesuai aturan grammar secara lengkap termasuk semua simbol terminal dan non-terminal, sedangkan AST menyederhanakan struktur dengan menghapus node yang tidak perlu dan menggabungkan beberapa tingkat produksi menjadi node semantik tunggal. Pembangunan AST dilakukan bersamaan dengan atau

segera setelah proses parsing selesai. Ada dua cara umum yang digunakan. Pertama, parser langsung membentuk node AST setiap kali sebuah aturan produksi grammar berhasil dikenal. Pendekatan ini biasa dilakukan dengan menyisipkan aksi semantik di dalam parser. Kedua, parse tree yang sudah terbentuk diubah menjadi AST melalui fase terpisah dengan setiap node parse tree dipetakan ke node AST yang sesuai atau dihilangkan jika tidak relevan untuk analisis selanjutnya.

AST adalah struktur utama yang digunakan dalam fase semantic analysis, optimasi, dan code generation karena menyediakan representasi yang jelas dan terstruktur untuk operasi transversal, anotasi, dan transformasi. AST memungkinkan implementasi analisis seperti type checking, data-flow analysis, dan optimasi lokal/global.

1.3. Decorated AST

Decorated AST adalah AST yang telah ditambahkan anotasi (dekorasi) pada node-node pohon untuk menyimpan informasi semantik yang diperoleh selama semantic analysis, seperti tipe ekspresi, alamat atau referensi ke entri symbol table, dan informasi lexical level atau offset. Anotasi ini membuat AST menjadi sumber tunggal kebenaran untuk informasi yang dibutuhkan tahap selanjutnya.

Decorated AST adalah AST yang telah ditambahkan anotasi (dekorasi) pada node-node pohon untuk menyimpan informasi semantik yang diperoleh selama semantic analysis, seperti tipe ekspresi, alamat atau referensi ke entri symbol table, dan informasi lexical level atau offset. Anotasi ini membuat AST menjadi sumber tunggal kebenaran untuk informasi yang dibutuhkan tahap selanjutnya (mis. kode antara atau pengecekan tipe lanjut).

Decorated AST memungkinkan fase pembuatan kode antara dan back-end untuk membaca informasi semantik secara langsung dari AST tanpa mengulang analisis. Ini mempercepat generasi kode dan mengurangi kemungkinan inkonsistensi. Selain itu, annotated AST mempermudah optimasi berbasis tipe dan analisis liveness/aliasing.

Proses dekorasi dilakukan melalui transversal DFS pada AST menggunakan fungsi visit untuk setiap jenis node. Setiap fungsi visit bertanggung jawab untuk mendapatkan informasi semantik node tersebut, seperti mencari tipe data, melakukan

lookup ke symbol table, dan memverifikasi kompatibilitas tipe, kemudian menyimpan hasilnya sebagai anotasi pada node yang bersangkutan.

<i>Tabel 1.3.1 Perbedaan AST dan Decorated AST</i>
Sebelum Dekorasi (AST):
AssignNode -- VarNode("a") -- NumberNode(5)
Sesudah Dekorasi (Decorated AST):
AssignNode [type: void] -- VarNode("a") [type: Integer, tab_index: 34, lev: 0] -- NumberNode(5) [type: Integer]

1.4. Perbedaan Parse Tree dan Abstract Syntax Tree

Parse tree atau derivation tree adalah struktur pohon yang merepresentasikan proses penurunan (derivasi) suatu string dari grammar secara lengkap. Setiap node internal pada parse tree merepresentasikan simbol non-terminal, sedangkan node daun merepresentasikan simbol terminal. Parse tree menyertakan semua simbol yang muncul dalam produksi grammar, termasuk simbol-simbol yang hanya berfungsi sebagai penanda sintaksis seperti tanda kurung, titik koma, dan kata kunci struktural.

Abstract Syntax Tree (AST) merupakan penyederhanaan dari parse tree. AST hanya mempertahankan node-node yang memiliki makna semantik dan membuang node yang tidak relevan untuk analisis lebih lanjut. Beberapa transformasi yang terjadi saat parse tree diubah menjadi AST adalah sebagai berikut.

- Node perantara yang hanya meneruskan satu anak (unit production) dihilangkan dan digantikan langsung oleh anaknya.
- Token terminal yang bersifat struktural seperti tanda kurung, titik koma, begin, end, dan kata kunci lainnya tidak dijadikan node karena tidak membawa informasi semantik.
- Beberapa tingkat produksi grammar yang hanya mengatur presedensi operator digabungkan menjadi satu node BinOp dengan atribut operator.

Sebagai ilustrasi, ekspresi $a := (b + c)$ pada parse tree akan menghasilkan node untuk assignment-statement, variable, expression, simple-expression, term, factor, tanda kurung kiri, expression, tanda kurung kanan, dan seterusnya secara hierarkis sesuai produksi grammar. Pada AST, struktur yang sama direpresentasikan hanya sebagai AssignNode dengan dua anak: VarNode("a") dan BinOpNode(plus) yang memiliki anak VarNode("b") dan VarNode("c").

Proses transformasi parse tree menjadi AST dapat dilakukan secara langsung selama parsing berlangsung melalui semantic actions, atau melalui fase traversal terpisah setelah parse tree selesai dibangun.

1.5. Attributed Grammar & SDT

Attributed grammar adalah context-free grammar yang diperluas dengan atribut pada setiap simbol dan aturan semantik yang menentukan cara menghitung atribut tersebut. Atribut digunakan untuk menyebarkan informasi semantik selama proses parsing, sehingga compiler dapat melakukan penerjemahan secara terarah mengikuti struktur sintaks program.

Atribut dibedakan menjadi dua jenis: synthesized attributes yang dihitung dari bawah ke atas (dari anak ke induk), dan inherited attributes yang diteruskan dari atas ke bawah (dari induk ke anak). L-attributed grammar adalah kelas attributed grammar yang atributnya dapat dihitung dalam satu traversal kiri ke kanan, sehingga cocok digunakan bersama parser top-down seperti Recursive Descent. Syntax-Directed Translation Scheme (SDT) merupakan penerapan attributed grammar yang menyisipkan aksi semantik langsung ke dalam produksi grammar untuk membangun AST atau menghasilkan output lain selama parsing berlangsung.

Attributed grammar dan SDT menjadi dasar formal implementasi semantic analysis. Dengan menyisipkan semantic actions pada setiap produksi grammar, compiler dapat membangun AST, mengisi symbol table, dan melakukan pengecekan tipe secara langsung tanpa memerlukan fase tambahan yang terpisah.

1.6. Syntax-Directed Translation & Semantic Action

Syntax-Directed Translation (SDT) adalah teknik dalam kompilasi yang menggabungkan pemrosesan semantik langsung ke dalam proses parsing dengan cara menyisipkan semantic actions pada produksi grammar. Semantic action adalah potongan kode yang dieksekusi pada titik tertentu selama parsing berlangsung, baik di tengah maupun di akhir suatu produksi.

Terdapat dua jenis penempatan semantic action dalam SDT. Pertama, postfix SDT menempatkan semantic action hanya di akhir setiap produksi sehingga action dieksekusi setelah seluruh sisi kanan produksi selesai diproses. Pendekatan ini cocok untuk parser bottom-up. Kedua, general SDT memperbolehkan semantic action disisipkan di posisi mana saja dalam produksi, sehingga cocok untuk parser top-down seperti Recursive Descent Parser.

Semantic action umumnya digunakan untuk melakukan hal-hal berikut.

- Membangun node AST dan menghubungkannya ke node induk.
- Mengisi symbol table saat menemukan deklarasi identifier.
- Menghitung dan meneruskan nilai atribut (synthesized maupun inherited).
- Melakukan pengecekan semantik sederhana seperti validasi tipe pada titik produksi tertentu.

Sebagai contoh, untuk produksi grammar berikut:

assignment-statement $\rightarrow$ variable := expression

Semantic action yang disisipkan dapat berupa:

assignment-statement $\rightarrow$ variable := expression { node = new AssignNode(variable.node, expression.node); checkAssignCompatible(variable.type, expression.type); }

Dalam implementasi Arion Compiler, semantic actions tidak disisipkan secara eksplisit ke dalam produksi grammar melainkan diimplementasikan melalui Visitor Pattern pada AST. Setiap fungsi visit berperan sebagai semantic action yang dieksekusi saat node yang bersangkutan dikunjungi selama traversal DFS. Pendekatan ini

memisahkan logika semantik dari logika parsing sehingga lebih mudah dikelola dan dikembangkan.

Perbedaan antara Attributed Grammar dan SDT terletak pada tingkat formalitasnya. Attributed grammar mendefinisikan aturan komputasi atribut secara deklaratif tanpa menentukan urutan eksekusi, sedangkan SDT bersifat prosedural dengan urutan eksekusi yang ditentukan oleh posisi semantic action dalam produksi grammar.

1.7. Symbol Table

Symbol table adalah struktur data yang menyimpan informasi tentang setiap identifier dalam program, seperti nama, jenis objek, tipe data, lexical level, dan alamat memori. Fungsi utamanya adalah menjadi sumber referensi tunggal yang digunakan selama semantic analysis untuk memverifikasi deklarasi dan penggunaan identifier, serta selama code generation untuk menentukan alokasi memori.

Implementasi symbol table bervariasi tergantung kebutuhan bahasa. Untuk bahasa dengan nested scope seperti Pascal, symbol table umumnya diimplementasikan sebagai stack of tables, satu tabel per blok, di mana compiler melakukan push saat masuk blok baru dan pop saat keluar. Operasi dasarnya meliputi insert saat menemukan deklarasi, lookup saat menemukan penggunaan identifier, dan update saat informasi identifier perlu diperbarui. Pada implementasi yang digunakan di tugas ini, symbol table terdiri dari tiga tabel terpisah: tab untuk identifier, btab untuk blok prosedural dan fungsi, serta atab untuk array.

Sebagai ilustrasi, untuk program sederhana:

<i>Tabel 1.7.1 Contoh Program Sederhana</i>	
<pre>program Hello; var a, b: integer; begin a := 5; end.</pre>	

Sebelum semantic analysis dimulai, symbol table diinisialisasi dengan predefined identifiers yang merupakan bagian dari bahasa, mencakup tipe dasar, konstanta bawaan, dan prosedur bawaan. Identifier yang dideklarasikan oleh user akan ditambahkan

setelahnya. Sebagai ilustrasi, setelah semantic analysis memproses program di atas, symbol table akan terisi sebagai berikut.

<i>Tabel 1.7.2 TAB (Identifier Table)</i>								
idx	name	obj	type	ref	nrm	lev	adr	link
1	Integer	type	Integer	0	1	0	0	0
2	Real	type	Real	0	1	0	0	1
3	Char	type	Char	0	1	0	0	2
4	Boolean	type	Boolean	0	1	0	0	3
5	String	type	String	0	1	0	0	4
6	True	constant	Boolean	0	1	0	1	5
7	False	constant	Boolean	0	1	0	0	6
8	writeln	procedure	Void	0	1	0	0	7
9	readln	procedure	Void	0	1	0	0	8
33	Hello	program	Void	0	1	0	0	9
34	a	variable	Integer	0	1	0	0	33
35	b	variable	Integer	0	1	0	1	34

<i>Tabel 1.7.3 BTAB (Block Table)</i>				
idx	last	lpar	psze	vsze
0	35	0	0	2

<i>Tabel 1.7.4 ATAB (Array Table)</i>

idx	xtyp	etyp	eref	low	high	elsz	size
—	—	—	—	—	—	—	—

(kosong karena tidak ada deklarasi array)

Indeks 1–9 merupakan predefined identifiers yang sudah terdefinisi sebelum program user dibaca. Identifier user dimulai dari indeks 33. Field link membentuk linked list antar identifier dalam satu blok, dengan nilai 0 menandakan akhir dari linked list. btab[0] merepresentasikan blok global dengan last=35 menunjuk ke identifier terakhir yang dideklarasikan yaitu b, dan vsze=2 menunjukkan total dua variabel lokal (a dan b).

1.8. Scope Resolution

Scope adalah wilayah dalam program di mana suatu identifier valid dan dapat diakses. Lexical scoping atau static scoping berarti cakupan suatu identifier ditentukan berdasarkan struktur teks program, yaitu posisi deklarasi di dalam blok kode, bukan berdasarkan urutan eksekusi saat runtime. Dengan lexical scoping, compiler sudah dapat menentukan identifier mana yang dirujuk pada waktu kompilasi.

Resolusi scope dilakukan dengan mencari deklarasi identifier dari scope terdalam ke terluar. Jika ditemukan identifier dengan nama yang sama di beberapa scope, deklarasi pada scope terdalam yang dipakai, ini disebut shadowing. Mekanisme ini diimplementasikan menggunakan struktur display yang memetakan setiap lexical level ke blok aktif di symbol table, sehingga compiler dapat menelusuri identifier secara efisien dari level lokal hingga global.

1.9. Type Checking

Type checking adalah proses verifikasi bahwa setiap operasi dalam program diterapkan pada operand dengan tipe yang sesuai. Tujuannya adalah mendeteksi ketidaksesuaian tipe sebelum program dijalankan, sehingga mencegah bug yang sulit dilacak saat runtime. Type checking yang dilakukan pada waktu kompilasi disebut static type checking.

Type compatibility mengacu pada apakah dua tipe dapat digunakan bersama dalam suatu ekspresi, sedangkan assignment compatibility lebih spesifik. Contoh aturan umum: Integer assignment-compatible dengan Real karena nilai integer dapat

dipromosikan ke real tanpa kehilangan informasi, namun String tidak kompatibel dengan Integer tanpa konversi eksplisit. Pengecekan tipe dilakukan dengan membaca informasi tipe yang sudah dianotasikan pada node Decorated AST dan entri di symbol table.

1.10. Visitor Pattern

Visitor pattern adalah pola desain yang memisahkan operasi dari struktur data yang dioperasikan. Dalam konteks compiler, pola ini digunakan untuk melakukan traversal AST, setiap jenis node memiliki fungsi visit tersendiri yang menangani logika semantik untuk node tersebut, seperti pengecekan tipe, pengisian symbol table, atau anotasi informasi.

Cara kerjanya, yaitu setiap node AST diproses oleh fungsi visit yang sesuai dengan jenisnya. Fungsi visit tersebut memanggil visit pada child-nya terlebih dahulu, kemudian mengakses atau memperbarui symbol table, menghitung tipe semantik, dan menambahkan anotasi pada node. Dengan pendekatan ini, logika untuk setiap jenis node terorganisir di satu tempat dan mudah dikembangkan tanpa perlu mengubah struktur node itu sendiri. Visitor pattern juga mendukung berbagai jenis traversal seperti pre-order dan post-order sesuai kebutuhan analisis.

1.11. Type System

Type system adalah komponen dalam semantic analyzer yang mendefinisikan himpunan tipe data yang valid dalam suatu bahasa pemrograman beserta aturan interaksi antar tipe tersebut. Dalam bahasa Arion, type system diimplementasikan melalui dua komponen utama: tipe data yang direpresentasikan sebagai enum `DataType`, dan fungsi-fungsi yang menentukan kompatibilitas antar tipe.

Tipe data yang didukung bahasa Arion adalah sebagai berikut.

<i>Tabel 1.11.1 DataType dalam bahasa Arion</i>	
DataType	Keterangan
INTEGER	Bilangan bulat, dihasilkan dari token <code>intcon</code>
REAL	Bilangan riil, dihasilkan dari token <code>realcon</code>

CHAR	Karakter tunggal, dihasilkan dari token charcon
BOOLEAN	Nilai logika, dihasilkan dari True atau False
STRING	Sekuens karakter, dihasilkan dari token string
SUBRANGE	Rentang nilai, didefinisikan dari constant..constant
ENUMERATED	Tipe enumerasi, didefinisikan dari (ident, ident, ...)
ARRAY	Tipe terstruktur dengan index dan elemen
RECORD	Tipe terstruktur dengan field bernama
VOID	Tipe prosedur atau statement, tidak memiliki nilai
UNKNOWN	Tipe tidak diketahui atau hasil operasi yang tidak valid, digunakan untuk error recovery

1.12. Type Compatibility Rule

Type Compatibility Rule mendefinisikan kapan dua tipe dapat digunakan bersama dalam suatu operasi atau assignment. Terdapat dua jenis kompatibilitas yang diimplementasikan.

1. Kompatibilitas Umum (isCompatible)

Dua tipe T1 dan T2 dinyatakan compatible jika memenuhi salah satu kondisi berikut.

<i>Tabel 1.12.1 Compatibility Rule dalam bahasa Arion</i>	
Kondisi	Contoh
T1 dan T2 adalah tipe yang sama	Integer dengan Integer
Salah satu atau keduanya adalah SUBRANGE dari tipe yang sama (bukan REAL)	SUBRANGE dengan INTEGER, SUBRANGE dengan SUBRANGE

<i>Tabel 1.12.1 Compatibility Rule dalam bahasa Arion</i>	
Keduanya STRING	String dengan String
Keduanya RECORD dengan ref yang sama (named type)	point1 dan point2 dari tipe record yang sama
Keduanya ARRAY dengan ref yang sama	Array dengan definisi tipe yang identik

Tipe UNKNOWN tidak pernah compatible dengan tipe apapun. SUBRANGE tidak compatible dengan REAL. Dua anonymous record, meskipun memiliki struktur yang sama, tidak compatible karena ref-nya berbeda.

2. Kompatibilitas Assignment (isAssignCompatible)

Tipe T2 dinyatakan assignment-compatible dengan tipe T1 (artinya T2 dapat di-assign ke T1) jika memenuhi salah satu kondisi berikut.

<i>Tabel 1.12.2 Compability Assignment dalam bahasa Arion</i>	
Kondisi	Contoh
T1 adalah REAL dan T2 adalah INTEGER (widening)	var r: Real; r := 5;
T1 dan T2 adalah tipe yang sama	var i: Integer; i := 10;
T1 dan T2 compatible dan bertipe INTEGER, BOOLEAN, CHAR, atau SUBRANGE	var s: 1..10; s := 5;
T1 dan T2 keduanya STRING dan compatible	String ke String
T1 dan T2 keduanya ARRAY atau RECORD dengan ref yang sama	Named type yang identik

Narrowing tidak diperbolehkan: REAL tidak dapat di-assign ke INTEGER tanpa konversi eksplisit.

3. Inferensi Tipe Operasi Biner (inferBinOpType)

<i>Tabel 1.12.3 Tabel Inferensi Tipe Operasi Biner</i>

Operator	Operand Kiri	Operand Kanan	Tipe Hasil
plus, minus, times	INTEGER	INTEGER	INTEGER
plus, minus, times	INTEGER atau REAL	INTEGER atau REAL (salah satu REAL)	REAL
rdiv	INTEGER atau REAL	INTEGER atau REAL	REAL
idiv, imod	INTEGER	INTEGER	INTEGER
idiv, imod	Selain INTEGER	—	UNKNOWN
andsy, orsy	BOOLEAN	BOOLEAN	BOOLEAN
andsy, orsy	Selain BOOLEAN	—	UNKNOWN
eql, neq, lss, leq, gtr, geq	Tipe relasional yang compatible	Tipe relasional yang compatible	BOOLEAN
Operator apapun	UNKNOWN	—	UNKNOWN

Tipe yang valid untuk operasi relasional adalah INTEGER, REAL, CHAR, STRING, BOOLEAN, dan SUBRANGE. ARRAY dan RECORD tidak dapat dibandingkan secara langsung.

4. Inferensi Tipe Operasi Unary (inferUnaryOpType)

<i>Tabel 1.12.4 Tabel Inferensi Tipe Operasi Unary</i>		
Operator	Operand	Tipe Hasil
notsy	BOOLEAN	BOOLEAN
notsy	Selain BOOLEAN	UNKNOWN
plus, minus	INTEGER	INTEGER
plus, minus	REAL	REAL
plus, minus	Selain INTEGER/REAL	UNKNOWN

5. Aturan Tambahan

- Index type array tidak boleh bertipe REAL. Tipe yang valid sebagai index adalah INTEGER, BOOLEAN, CHAR, SUBRANGE, dan ENUMERATED.
- Subrange tidak boleh memiliki tipe REAL.
- Lower bound subrange tidak boleh lebih besar dari upper bound.
- Kondisi pada if, while, dan repeat-until harus bertipe BOOLEAN.

1.13. Error Handling dalam Semantic Analysis

Error handler pada Arion Compiler diimplementasikan sebagai modul terpisah yang dapat dipanggil dari mana saja dalam semantic analyzer. Terdapat tiga level kesalahan yang didukung.

<i>Tabel 1.13.1 Tabel Level pada error handling</i>	
Level	Keterangan
WARNING	Peringatan yang tidak menghentikan analisis
ERROR	Kesalahan semantik yang dicatat dan analisis dilanjutkan untuk menemukan error lain
FATAL	Kesalahan yang menghentikan seluruh proses analisis

Fungsi-fungsi error yang disediakan beserta kondisi pemanggilannya adalah sebagai berikut.

<i>Tabel 1.13.2 Tabel fungsi dan kondisi pemanggilan pada error handling</i>	
Fungsi	Kondisi Pemanggilan
undeclaredError	Identifier digunakan sebelum dideklarasikan
redeclarationError	Identifier dideklarasikan ulang di scope yang sama
typeMismatchError	Tipe tidak sesuai dalam suatu konteks

	operasi
assignIncompatibleError	Tipe nilai tidak assignment-compatible dengan tipe variabel tujuan
invalidOperandError	Operator biner diterapkan pada tipe yang tidak valid
invalidUnaryOperandError	Operator unary diterapkan pada tipe yang tidak valid
nonBooleanConditionError	Kondisi if/while/repeat bukan bertipe BOOLEAN
invalidSubrangeError	Lower bound subrange lebih besar dari upper bound
invalidIndexTypeError	Index type array adalah REAL atau tipe yang tidak valid
wrongArgCountError	Jumlah argumen pemanggilan prosedur/fungsi tidak sesuai
wrongObjectKindError	Identifier digunakan tidak sesuai dengan jenis objeknya
realSubrangeError	Subrange didefinisikan dengan tipe REAL

Error handler menyimpan jumlah error dan warning secara kumulatif melalui `getErrorCount()` dan `getWarningCount()`. Di akhir analisis, `printErrorSummary()` mencetak ringkasan status kompilasi.

1.14. Predefined Identifier

Sebelum program user dianalisis, symbol table diinisialisasi dengan predefined identifiers yang merupakan bagian bawaan dari bahasa Arion. Identifier ini terdaftar pada indeks 1–9 di tabel tab, sebelum identifier user yang dimulai dari indeks 33.

Tabel 1.14.1 Tabel predefined identifier				
Indeks	Nama	Obj	Type	Keterangan
1	Integer	TYPE	INTEGER	Tipe data bilangan bulat

2	Real	TYPE	REAL	Tipe data bilangan riil
3	Char	TYPE	CHAR	Tipe data karakter tunggal
4	Boolean	TYPE	BOOLEAN	Tipe data logika
5	String	TYPE	STRING	Tipe data string
6	True	CONSTANT	BOOLEAN	Konstanta Boolean bernilai true (adr=1)
7	False	CONSTANT	BOOLEAN	Konstanta Boolean bernilai false (adr=0)
8	writeln	PROCEDURE	VOID	Prosedur bawaan untuk output ke terminal
9	readln	PROCEDURE	VOID	Prosedur bawaan untuk input dari terminal

Identifier user dimulai dari indeks 33 sesuai spesifikasi. Indeks 10–32 dikosongkan sebagai reserved space untuk kemungkinan penambahan predefined identifier di masa mendatang.

BAB II

PERANCANGAN DAN IMPLEMENTASI

2.1. Gambaran Umum

Semantic Analyzer pada Arion Compiler merupakan tahap ketiga dalam pipeline kompilasi, yaitu semantic analysis. Tahap ini menerima Abstract Syntax Tree (AST) yang dibangun dari parse tree hasil Milestone 2, kemudian menelusurinya secara rekursif untuk memastikan makna program secara logis sudah benar. Semantic Analyzer diimplementasikan sebagai kelas `SemanticAnalyzer` yang menerima root AST dan objek `Symbol Table`. Traversal dilakukan secara DFS menggunakan Visitor Pattern, dimana setiap jenis node memiliki fungsi visit tersendiri. Hasil akhirnya adalah Decorated AST yang sudah dianotasi dan Symbol Tabel yang terisi lengkap. Pipeline kompilasi secara keseluruhan adalah sebagai berikut.

Tahap	Input	Output
Lexical Analysis (M1)	Source code (.txt)	Token stream
Syntax Analysis (M2)	Token stream	Parse tree
Semantic Analysis (M3)	Parse tree / AST	Decorated AST + Symbol Table

2.2. Struktur File

File	Peran
<code>semanticAnalyzer.h</code>	Deklarasi kelas <code>SemanticAnalyzer</code> dan semua visit functions
<code>semanticAnalyzer.cpp</code>	Implementasi seluruh visit functions bagian deklarasi
<code>symbolTable.hpp</code>	Definisi struct <code>TabEntry</code> , <code>BTabEntry</code> , <code>ATabEntry</code> , dan kelas <code>SymbolTable</code>
<code>symbolTable.cpp</code>	Implementasi operasi symbol table: <code>enterTab</code> , <code>lookup</code> , <code>pushScope</code> , <code>popScope</code> , dsb.
<code>ASTNode.h</code>	Definisi semua class node AST
<code>ASTNode.cpp</code>	Implementasi fungsi <code>print()</code> untuk setiap node

TypeSystem.h/.cpp	Definisi aturan kompatibilitas tipe dan inferensi tipe operasi biner
ErrorHandler.h/.cpp	Implementasi seluruh fungsi pelaporan error semantik
main.cpp	Entry point: membaca input, menjalankan lexer, parser, dan semantic analyzer

2.3. Alur Kerja Program

Semantic Analyzer bekerja dengan menelusuri AST secara top-down menggunakan pola Visitor. Setiap jenis node memiliki fungsi visit yang bersesuaian. Alur kerja secara keseluruhan adalah sebagai berikut.

1. Program dimulai dari main.cpp yang membangun symbol table dan ast root (dari ParseTree) dengan memanggil builder(symTable) dan builder.build(root).
2. Program kemudian memanggil analyzer.analyze(astRoot).
3. analyze() melakukan cast root ke ProgramNode, lalu memanggil visitProgram().
4. visitProgram() mendaftarkan nama program ke symbol table, kemudian memproses seluruh deklarasi secara berurutan melalui visitStatement().
5. visitStatement() adalah dispatcher umum yang menentukan jenis node dan memanggil visit yang sesuai.
6. Setelah seluruh deklarasi selesai diproses, visitProgram() memanggil visitCompoundStmt() untuk memproses body program

Urutan pemrosesan deklarasi mengikuti urutan kemunculan di AST, yaitu const → type → var → procedure → function. "Urutan ini penting karena type alias harus didaftarkan sebelum var agar resolusi tipe saat deklarasi variabel dapat dilakukan, dan prosedur/fungsi harus diproses setelah var agar variabel global sudah tersedia saat body prosedur dianalisis

2.4. Manajemen Scope

Scope adalah cakupan berlakunya suatu identifier. Variabel yang dideklarasikan di dalam prosedur hanya berlaku di dalam prosedur tersebut dan tidak dapat diakses dari luar. Manajemen scope diimplementasikan menggunakan tiga komponen utama.

2.4.1. Lexical Level

Setiap identifier memiliki atribut lev yang menyatakan kedalaman scope tempat identifier tersebut dideklarasikan. Level 0 adalah scope global (program utama). Level 1 adalah scope prosedur atau fungsi di level pertama. Level 2 adalah scope prosedur atau fungsi yang bersarang di dalam prosedur level 1, dan seterusnya.

2.4.2. Stack Scope dengan display[]

Symbol table menggunakan array display[] untuk melacak blok aktif pada setiap level. display[lev] menyimpan indeks btab dari blok yang sedang aktif pada level tersebut. Saat pushScope() dipanggil, currentLevel naik satu dan blok baru dibuat di btab. Saat popScope() dipanggil, currentLevel turun kembali dan currentBlock dikembalikan ke blok induknya.

Contoh keadaan display[] saat menganalisis program dengan prosedur bersarang:

Kondisi	currentLevel	display[0]	display[1]	display[2]
Scope global	0	btab[0]	-	-
Masuk procedure outer	1	btab[0]	btab[1]	-
Masuk procedure inner (nested)	2	btab[0]	btab[1]	btab[2]
Keluar dari inner	1	btab[0]	btab[1]	-
Keluar dari outer	0	btab[0]	-	-

2.4.3. Deteksi Redclaration

Untuk mendeteksi redeclaration, digunakan lookupCurrentScope() yang hanya menelusuri linked list identifier di blok aktif saat ini. Berbeda dengan lookup() yang menelusuri semua level dari dalam ke luar, lookupCurrentScope() berhenti di level saat ini sehingga variabel dengan nama sama di scope berbeda tidak dianggap konflik.

2.5. Struktur Data

Semantic analyzer menggunakan tiga tabel yang saling berkaitan untuk menyimpan informasi identifier. Tiga tabel terpisah (tab, btab, atab) dipilih karena setiap kategori entitas memiliki atribut yang berbeda. Karena identifier umum, blok scope, dan array sulit untuk direpresentasikan dalam satu tabel tanpa banyak field kosong. Visitor pattern dipilih karena memisahkan logika semantik dari struktur node sehingga lebih mudah dikembangkan per jenis node tanpa mengubah definisi AST.

2.5.1. Tab (Identifier Table)

Tab adalah tabel utama yang menyimpan semua identifier yang dideklarasikan dalam program. Setiap entri memiliki atribut berikut.

Atribut	Tipe	Keterangan
name	string	Nama identifier
link	int	Indeks entri sebelumnya dalam scope yang sama (linked list)
obj	AllowedObj	Jenis objek: VARIABLE, CONSTANT, TYPE, PROCEDURE, FUNCTION, PROGRAM
type	DataType	Tipe data: INTEGER, REAL, CHAR, BOOLEAN, STRING, ARRAY, RECORD, VOID, dsb.
ref	int	Referensi ke atab (array) atau btab (blok prosedur/fungsi/record)
nrm	int	1 = parameter by-value, 0 = parameter by-reference
lev	int	Lexical level tempat identifier dideklarasikan
adr	int	Untuk CONSTANT: nilai konstanta; untuk VARIABLE: offset di stack frame

Indeks 0 dikosongkan sebagai sentinel (tanda akhir linked list). Indeks 1–9 diisi predefined identifiers (Integer, Real, Char, Boolean, String, True, False, writeln, readln). Identifier user mulai dari indeks 33.

```

struct TabEntry{
    string name;
    int link;
    AllowedObj obj;
    DataType type;
    int ref;
    int nrm;
    int lev;
    int adr;
};

```

2.5.2. Btab (Block Table)

Btab menyimpan informasi setiap blok (program utama, prosedur, fungsi, atau record). Setiap entri memiliki atribut berikut.

Atribut	Tipe	Keterangan
last	int	Indeks tab dari identifier terakhir yang terdaftar di blok ini
lpar	int	Indeks tab dari parameter terakhir prosedur/fungsi
psze	int	Total ukuran parameter dalam unit memori
vsze	int	Total ukuran variabel lokal dalam unit memori

```

struct BTabEntry{
    int last;
    int lpar;
    int psze;
    int vsze;
};

```

2.5.3. Atab (Array Table)

Atab menyimpan informasi tipe array. Setiap entri memiliki atribut berikut.

Atribut	Tipe	Keterangan
xtyp	DataType	Tipe index array (tidak boleh REAL)
etyp	DataType	Tipe elemen array
eref	int	Referensi ke atab/btab jika elemen bertipe array/record
low	int	Batas bawah index
high	int	Batas atas index
elsz	int	Ukuran satu elemen dalam unit memori
size	int	Total ukuran array = $(\text{high} - \text{low} + 1) * \text{elsz}$

```
struct ATabEntry{
    DataType xtyp;
    DataType etyp;
    int eref;
    int low;
    int high;
    int elsz;
    int size;
};
```

2.6. Perancangan Type System

Tulis penjelasanmu kawan

2.6.1. Aturan compatible dan assignment-compatible

Tulis penjelasanmu kawan

2.6.2. Aturan inferBinOpType dan inferUnaryOpType

Tulis penjelasanmu kawan

2.7. Perancangan AST Node

2.7.1. Class hierarchy ASTNode

Semua node AST merupakan turunan dari struct ASTNode sebagai base class.

ASTNode memiliki tiga atribut semantik yang diisi oleh SemanticAnalyzer:

- dtype : tipe data hasil ekspresi node
- tabIndex : indeks ke symbol table (-1 jika tidak relevan)
- lexLevel : lexical level scope tempat node berada

Setiap node mengimplementasikan dua virtual function: nodeName() untuk nama node saat print, dan print() untuk mencetak node beserta subtree-nya dengan format tree.

2.7.2. Daftar 24 jenis ASTNode

No.	Nama Node	Deskripsi	Atribut Tambahan
1.	ProgramNode	Root program	name, declarations, body
2.	VarDeclNode	Deklarasi variabel	varName, varType, typeRef
3.	ConstDeclNode	Deklarasi konstanta	constName, constType, value
4.	TypeDeclNode	Deklarasi tipe	typeName, baseType, typeRef
5.	ProcDeclNode	Deklarasi prosedur	procName, params, body
6.	FuncDeclNode	Deklarasi fungsi	funcName, returnType, params, body
7.	CompoundStmtNode	Blok begin..end	statements
8.	AssignNode	Assignment	target, value
9.	BinOpNode	Operasi biner	op, left, right
10.	UnaryOpNode	Operasi unary	op, operand
11.	VarNode	Penggunaan variabel	varName
12.	NumberNode	Literal integer/real	rawValue
13.	CharNode	Literal char	rawValue
14.	StringNode	Literal string	rawValue
15.	ProcCallNode	Pemanggilan prosedur/fungsi	procName, args

16.	IfNode	Statement if	condition, thenBranch, elseBranch
17.	WhileNode	Statement while	condition, body
18.	RepeatNode	Statement repeat	statements, condition
19.	ForNode	Statement for	varName, fromExpr, toExpr, isDownto, body
20.	CaseNode	Statement case	selector, branches
21.	ArrayAccessNode	Akses elemen array	array, indices
22.	FieldAccessNode	Akses field record	record, fieldName

2.8. Perancangan AST Builder

Tulis penjelasanmu kawan

2.8.1. Mapping ParseNode ke ASTNode

ASTBuilder menerima root ParseNode dari Parser dan membangun ASTNode tree melalui fungsi build(). Setiap build*() function bertanggung jawab atas satu jenis ParseNode dan mengembalikan ASTNode yang sesuai.

ParseNode	ASTNode	Keterangan
<program>	ProgramNode	Root program
<program-header>	-	Dibuang, nama diambil langsung
<declaration-part>	-	Dispatcher, tidak jadi node
<const-declaration>	ConstDeclNode	Satu node per konstanta
<type-declaration>	TypeDeclNode	Satu node per tipe
<var-declaration>	VarDeclNode	Satu node per variabel
<subprogram-declaration>	-	Dispatcher ke proc/func
<procedure-declaration>	ProcDeclNode	
<function-declaration>	FuncDeclNode	

block	CompoundStmtNode	
<formal-parameter-list>	VarDeclNode	Satu node per parameter
<parameter-group>	VarDeclNode	
<compound-statement>	CompoundStmtNode	
<statement-list>	-	Dibuang, statements dikumpulkan langsung
<statement>	-	Transparent dispatcher
<assignment-statement>	AssignNode	
<if-statement>	IfNode	
<while-statement>	WhileNode	
<repeat-statement>	RepeatNode	
<for-statement>	ForNode	
<case-statement>	CaseNode	
<case-block>	-	Branches dikumpulkan ke CaseNode
<procedure/function-call >	ProcCallNode	
<expression>	BinOpNode / passthrough	BinOp jika ada relational operator
<simple-expression>	BinOpNode / passthrough	BinOp jika ada additive operator
<term>	BinOpNode / passthrough	BinOp jika ada multiplicative operator
<factor>	berbagai node	Tergantung isi factor
<variable>	VarNode / ArrayAccessNode / FieldAccessNode	Tergantung component-variable
<component-variable>	ArrayAccessNode / FieldAccessNode	

<index-list>	-	Index dikumpulkan ke ArrayAccessNode
<relational-operator>	-	Operator diambil langsung
<additive-operator>	-	Operator diambil langsung
<multiplicative-operator >	-	Operator diambil langsung
<parameter-list>	-	Args dikumpulkan ke ProcCallNode
<identifier-list>	-	Ident dikumpulkan langsung
<type>	-	Resolver, tidak jadi node
<array-type>	-	Resolver ke atab
<range>	-	Resolver ke atab
<enumerated>	-	Resolver ke tab
<record-type>	-	Resolver ke btab
<field-list>	-	Fields dikumpulkan ke btab
<field-part>	-	
<constant>	-	Nilai diambil langsung

2.8.2. Informasi Sintaksis yang Dibuang dan Informasi Sintaksis yang Disimpan

Berikut adalah daftar informasi sintaksis yang dibuang karena yang tidak diperlukan untuk analisis semantik.

- Keyword terminal: programsy, varsy, constsy, typesy, proceduresy, functionsy, beginsy, endsy, recordsy, arraysy
- Punctuation: semicolon, colon, comma, period, lparent, rparent, lbrack, rbrack
- Operator terminal: becomes, eql, ofsy, dosy, thensy, tosy, downtosy, untilsy, elsesy

- Node wrapper: <statement>, <statement-list>, <declaration-part>, <program-header>

Berikut adalah daftar informasi sintaksis yang dipertahankan karena yang diperlukan untuk analisis semantik.

- Nama identifier: nama program, variabel, konstanta, prosedur, fungsi
- Nilai literal: intcon, realcon, charcon, string
- Operator: plus, minus, times, rdiv, idiv, imod, andsy, orsy, notsy, eql, neq, lss, leq, gtr, geq
- Struktur hierarki: hubungan parent-child antar node
- Arah perulangan: isDownto pada ForNode
- Cabang kondisional: thenBranch dan elseBranch pada IfNode

2.9. Implementasi

2.9.1. Implementasi Symbol Table

Symbol table diimplementasikan sebagai kelas SymbolTable yang terdiri dari tiga tabel utama: tab (vector of TabEntry) untuk menyimpan seluruh identifier, btab (vector of BTabEntry) untuk menyimpan informasi blok scope, dan atab (vector of ATabEntry) untuk menyimpan informasi tipe array. State scope aktif dilacak melalui currentLevel (lexical level saat ini), currentBlock (indeks btab blok aktif), dan display (array yang memetakan setiap level ke indeks btab blok aktif pada level tersebut).

Saat inisialisasi melalui init(), seluruh tabel dikosongkan dan diisi ulang. Slot indeks 0 pada tab dikosongkan sebagai sentinel karena link = 0 digunakan sebagai tanda akhir linked list. Indeks 1–32 diisi 32 reserved words, di mana indeks 22–26 (INTEGER, REAL, BOOLEAN, CHAR, STRING) didaftarkan sebagai AllowedObj::TYPE dengan DataType yang sesuai, sementara sisanya didaftarkan sebagai AllowedObj::KEYWORD. Indeks 33–36 diisi predefined identifier true, false, writeln, dan readln. Slot 37 ke atas disiapkan untuk identifier user, sesuai konstanta PredefinedIdx::USER_START.

enterTab() mendaftarkan identifier baru ke tab dengan mengambil btab[currentBlock].last sebagai nilai link (membentuk linked list per blok), lalu

memperbarui `btab[currentBlock].last` ke indeks entri baru. Fungsi ini mengembalikan indeks entri yang baru ditambahkan.

`lookup()` menelusuri identifier dari scope terdalam ke terluar menggunakan `display[lvl]` untuk mendapatkan indeks `btab` pada setiap level, lalu mengikuti linked list link di setiap blok hingga menemukan nama yang cocok atau mencapai sentinel 0.

`lookupCurrentScope()` hanya menelusuri blok aktif saat ini (`btab[currentBlock].last`) dan berhenti saat menemukan identifier dengan `lev < currentLevel`, sehingga hanya mendeteksi redeclaration dalam scope yang sama tanpa menganggap shadowing dari scope luar sebagai konflik.

`pushScope()` menaikkan `currentLevel`, membuat blok baru di `btab` via `enterBtab()`, memperbarui `currentBlock` dan `display[currentLevel]` ke blok baru, lalu mengembalikan indeks `btab` blok tersebut. `popScope()` menurunkan `currentLevel` kembali dan mengembalikan `currentBlock` ke `display[currentLevel]` (blok parent).

`enterAtab()` mendaftarkan entri array baru ke `atab` dengan menghitung `size = (high - low + 1) * elsz` secara otomatis. `enterBtab()` mendaftarkan entri blok baru ke `btab` dan mengembalikan indeksnya.

`printTables()` mencetak isi ketiga tabel ke terminal dalam format kolom menggunakan `setw()`, dengan konversi `DataType` dan `AllowedObj` ke string melalui `DataTypeToString()` dan `AllowedObjToString()`. Slot kosong pada tab (nama kosong) dilewati saat pencetakan.

```
#include <iomanip>
#include <iostream>
#include <stdexcept>
#include "symbolTable.hpp"
using namespace std;

SymbolTable::SymbolTable() {
    init();
}
```

```

void SymbolTable::init(){
    //clear dulu semua
    tab.clear();
    atab.clear();
    btab.clear();
    display.clear();
    currentLevel = 0;
    currentBlock = 0;

    enterBtab(0,0,0,0);
    // slot 0 i tab dikosongin, krn link = 0 dipakai sebagai
tanda sudah habis saat nelusurin linked list.
        tab.push_back({"", 0, AllowedObj::VARIABLE,
DataType::UNKNOWN, 0, 0, 0, 0});

    // tambahin 32 reserved words (indeks 1 - 32)
    const vector<string> reservedWords = {
        "AND", "ARRAY", "BEGIN", "CASE", "CONST", "DIV",
"DOWNTO", "DO",
        "ELSE", "END", "FOR", "FUNCTION", "IF", "MOD", "NOT",
"OF",
        "OR", "PROCEDURE", "PROGRAM", "RECORD", "REPEAT",
        "INTEGER", "REAL", "BOOLEAN", "CHAR", "STRING",
        "THEN", "TO", "TYPE", "UNTIL", "VAR", "WHILE"
    };

    for (int i = 0; i < (int)reservedWords.size(); i++) {
        string w = reservedWords[i];
        if (i >= 21 && i <= 25) {
            DataType dt = DataType::UNKNOWN;
            if (w == "INTEGER") dt = DataType::INTEGER;
            else if (w == "REAL") dt = DataType::REAL;
            else if (w == "BOOLEAN") dt = DataType::BOOLEAN;
            else if (w == "CHAR") dt = DataType::CHAR;
            else if (w == "STRING") dt = DataType::STRING;
            addPredefined(w, AllowedObj::TYPE, dt, 0, 1, 0,
0);
        } else {
            addPredefined(w, AllowedObj::KEYWORD,
DataType::UNKNOWN, 0, 1, 0, 0);

```

```

    }

}

// constant
    addPredefined("true",    AllowedObj::CONSTANT,
DataType::BOOLEAN, 0, 1, 0, 1); // idx 33
    addPredefined("false",   AllowedObj::CONSTANT,
DataType::BOOLEAN, 0, 1, 0, 0); // idx 34

// procedure
    addPredefined("writeln",  AllowedObj::PROCEDURE,
DataType::VOID, 0, 1, 0, 0); // idx 35
    addPredefined("readln",   AllowedObj::PROCEDURE,
DataType::VOID, 0, 1, 0, 0); // idx 36

// slot kosong dulu sampe USER_START
    for (int i = (int)tab.size(); i <
PredefinedIdx::USER_START; i++) {
        tab.push_back({"", 0, AllowedObj::VARIABLE,
DataType::UNKNOWN, 0, 0, 0, 0});
    }

// setup display dan global block
display.push_back(0);
}

void SymbolTable::addPredefined(const string& name, AllowedObj
obj, DataType type, int ref, int nrm, int lev, int adr){
    int link = btab[currentBlock].last;
    tab.push_back({name, link, obj, type, ref, nrm, lev,
adr});
    btab[currentBlock].last = (int)tab.size()-1;
}

// operasi tab
// masukan ident baru ke tab, return indeksnya
int SymbolTable::enterTab(const string& name, AllowedObj obj,
DataType type, int ref, int nrm, int lev, int adr){
    int link = btab[currentBlock].last;

```

```

    TabEntry entry;
    entry.name = name;
    entry.link = link;
    entry.obj  = obj;
    entry.type = type;
    entry.ref  = ref;
    entry.nrm  = nrm;
    entry.lev  = lev;
    entry.adr  = adr;

    tab.push_back(entry);
    int idx = (int)tab.size() - 1;

    btab[currentBlock].last = idx;

    return idx;
}

int SymbolTable::lookup(const string& name) const {
    for (int lvl = currentLevel; lvl >= 0; lvl--) {
        int blockIdx = display[lvl];
        int i = btab[blockIdx].last;
        // Telusuri linked list identifier di blok ini
        while (i > 0) {
            if (tab[i].name == name) return i;
            i = tab[i].link;
        }
    }
    return -1;
}

int SymbolTable::lookupCurrentScope(const string &name) const
{
    int i = btab[currentBlock].last;
    while (i > 0)
    {

        if (tab[i].lev < currentLevel && currentLevel > 0)
            break;
        if (tab[i].name == name)

```

```

        return i;
        i = tab[i].link;
    }
    return -1;
}

// Operasi btab
// Buat entri blok baru di btab, return indeksinya
int SymbolTable::enterBtab(int last, int lpar, int psze, int
vsze) {
    BTabEntry entry;
    entry.last = last;
    entry.lpar = lpar;
    entry.psize = psze;
    entry.vsize = vsze;
    btab.push_back(entry);
    return (int)btab.size() - 1;
}

// Operasi atab
// Buat entri array baru di atab, return indeksinya
int SymbolTable::enterAtab(DataType xtyp, DataType etyp, int
eref,
                                int low, int high, int elsz) {
    ATabEntry entry;
    entry.xtyp = xtyp;
    entry.etype = etyp;
    entry.eref = eref;
    entry.low = low;
    entry.high = high;
    entry.elsz = elsz;
    entry.size = (high - low + 1) * elsz;
    atab.push_back(entry);
    return (int)atab.size() - 1;
}

int SymbolTable::pushScope() {
    currentLevel++;

    // Buat blok baru di btab

```



```

    int newBlock = enterBtab(0, 0, 0, 0);
    currentBlock = newBlock;

    // Update display
    if (currentLevel < (int)display.size()) {
        display[currentLevel] = newBlock;
    } else {
        display.push_back(newBlock);
    }

    return newBlock;
}

void SymbolTable::popScope() {
    if (currentLevel <= 0) {
        throw runtime_error("SymbolTable::popScope() dipanggil
saat sudah di level global");
    }

    // Kembali ke blok parent
    currentLevel--;
    currentBlock = display[currentLevel];
}

// Utilitas
string SymbolTable::DataTypeToString(DataType t) {
    switch (t) {
        case DataType::INTEGER:    return "Integer";
        case DataType::REAL:       return "Real";
        case DataType::CHAR:       return "Char";
        case DataType::BOOLEAN:    return "Boolean";
        case DataType::STRING:     return "String";
        case DataType::SUBRANGE:   return "Subrange";
        case DataType::ENUMERATED: return "Enumerated";
        case DataType::ARRAY:      return "Array";
        case DataType::RECORD:     return "Record";
        case DataType::VOID:       return "Void";
        case DataType::UNKNOWN:    return "Unknown";
        default:                   return "?";
    }
}

```

```

}

string SymbolTable::AllowedObjToString(AllowedObj o) {
    switch (o) {
        case AllowedObj::CONSTANT:    return "constant";
        case AllowedObj::VARIABLE:    return "variable";
        case AllowedObj::TYPE:        return "type";
        case AllowedObj::PROCEDURE:    return "procedure";
        case AllowedObj::FUNCTION:    return "function";
        case AllowedObj::PROGRAM:     return "program";
        case AllowedObj::KEYWORD:     return "keyword";
        default:                      return "?";
    }
}

// Print isi tabel ke terminal
void SymbolTable::printTables() const {
    // tab
    cout << "\n TAB (Identifier Table) " << endl;
    cout << left
        << setw(5) << "idx"
        << setw(16) << "name"
        << setw(12) << "obj"
        << setw(12) << "type"
        << setw(5) << "ref"
        << setw(5) << "nrm"
        << setw(5) << "lev"
        << setw(5) << "adr"
        << setw(5) << "link"
        << endl;
    cout << string(70, '-') << endl;

    for (int i = 1; i < (int)tab.size(); i++) {
        const TabEntry& e = tab[i];
        if (e.name.empty()) continue; // skip slot kosong
        cout << left
            << setw(5) << i
            << setw(16) << e.name
            << setw(12) << AllowedObjToString(e.obj)
            << setw(12) << DataTypeToString(e.type)

```

```

        << setw(5) << e.ref
        << setw(5) << e.nrm
        << setw(5) << e.lev
        << setw(5) << e.adr
        << setw(5) << e.link
        << endl;
    }

    // btab
    cout << "\n BTAB (Block Table)" << endl;
    cout << left
        << setw(5) << "idx"
        << setw(8) << "last"
        << setw(8) << "lpar"
        << setw(8) << "psze"
        << setw(8) << "vsze"
        << endl;
    cout << string(40, '-') << endl;

    for (int i = 0; i < (int)btab.size(); i++) {
        const BTabEntry& e = btab[i];
        cout << left
            << setw(5) << i
            << setw(8) << e.last
            << setw(8) << e.lpar
            << setw(8) << e.psize
            << setw(8) << e.vsize
            << endl;
    }

    // atab
    cout << "\n ATAB (Array Table)" << endl;
    if (atab.empty()) {
        cout << "(kosong)" << endl;
    } else {
        cout << left
            << setw(5) << "idx"
            << setw(12) << "xtyp"
            << setw(12) << "etyp"
            << setw(6) << "eref"

```

```

        << setw(6) << "low"
        << setw(6) << "high"
        << setw(6) << "elsz"
        << setw(6) << "size"
        << endl;
    cout << string(60, '-') << endl;

    for (int i = 0; i < (int)atab.size(); i++) {
        const ATabEntry& e = atab[i];
        cout << left
            << setw(5) << i
            << setw(12) << DataTypeToString(e.xtyp)
            << setw(12) << DataTypeToString(e.ety)
            << setw(6) << e.eref
            << setw(6) << e.low
            << setw(6) << e.high
            << setw(6) << e.elsz
            << setw(6) << e.size
            << endl;
    }
}

```

2.9.2. Implementasi AST Node

Setiap node mengimplementasikan `print()` yang mencetak node beserta subtree-nya dalam format tree. Format output menggunakan karakter `|—` dan `└—` untuk menunjukkan hierarki. Setiap baris node dilengkapi anotasi semantik dari fungsi `annotation()` yang menghasilkan string berformat:

```

→ type:integer
→ tab_index:38, type:integer, lev:1

```

`annotation()` menghasilkan string kosong untuk `tabIndex == -1`, dan menyertakan `tab_index`, `type`, dan `lev` jika `tabIndex >= 0`. Tipe data diubah ke lowercase menggunakan `DataTypeToString()` dari `SymbolTable`.

`printHeader()` digunakan oleh semua node untuk mencetak baris header dengan prefix dan connector yang sesuai. `printChildren()` digunakan oleh node yang memiliki list children seperti `CompoundStmtNode` dan `ProcCallNode`.

Sementara itu, `toSource()` menghasilkan representasi string sumber dari node untuk digunakan sebagai label pada `AssignNode::nodeName()`. Implementasi per node:

- `BinOpNode::toSource()` — menggabungkan `left->toSource()` + operator + `right->toSource()`. Operator dikonversi dari nama token ke simbol: plus → +, minus → -, star → *, dll
- `UnaryOpNode::toSource()` — menggabungkan operator + `operand->toSource()`
- `ArrayAccessNode::toSource()` — menghasilkan "`arr[i]`" dengan iterasi indices
- `FieldAccessNode::toSource()` — menghasilkan "`record.field`" dengan menggabungkan `record->toSource()` + "." + `fieldName`
- `AssignNode::nodeName()` memanfaatkan `toSource()` dari target dan value untuk menghasilkan label deskriptif seperti "`Assign(a := 5)`" dan "`Assign(arr[i] := 10)`"

```
#include "ASTNode.h"
#include <sstream>

string ASTNode::annotation() const
{
    string tStr = SymbolTable::DataTypeToString(dtype);
    for (char &c : tStr) c = tolower(c); // Lowercase type

    string s = "\t\t→ type:" + tStr;
    if (tabIndex >= 0)
        s = "\t\t→ tab_index:" + to_string(tabIndex) + ",
type:" + tStr + ", lev:" + to_string(lexLevel);
    return s;
}

void ASTNode::printHeader(ostream &out,
                        const string &prefix,
                        bool isLast,
                        bool isRoot,
                        const string &extra) const
```

```

{
    if (isRoot)
    {
        out << nodeName() << annotation();
        if (!extra.empty())
            out << " " << extra;
        out << "\n";
    }
    else
    {
        out << prefix << (isLast ? "└─ " : "├─ ");
        out << nodeName() << annotation();
        if (!extra.empty())
            out << " " << extra;
        out << "\n";
    }
}

void ASTNode::printChildren(ostream &out,
                           const string &prefix,
                           bool isLast,
                           bool isRoot,
                           const vector<ASTNodePtr>
&children) const
{
    string childPrefix = prefix + (isRoot ? "" : (isLast ? "
" : "├─ "));
    for (size_t i = 0; i < children.size(); i++)
    {
        if (children[i])
            children[i]->print(out, childPrefix, i ==
children.size() - 1, false);
    }
}

void ProgramNode::print(ostream &out, const string &prefix,
bool isLast, bool isRoot) const
{
    printHeader(out, prefix, isLast, isRoot);
}

```

```

        string childPrefix = prefix + (isRoot ? "" : (isLast ? "
" : "|  "));

        if (!declarations.empty())
        {
            bool hasBody = (body != nullptr);
            out << childPrefix << (hasBody ? "└─ " : "└─ ") <<
"Declarations\n";
            string declPrefix = childPrefix + (hasBody ? "|  " :
"  ");
            for (size_t i = 0; i < declarations.size(); i++)
            {
                bool last = (i == declarations.size() - 1);
                if (declarations[i])
                    declarations[i]->print(out, declPrefix, last,
false);
            }
        }
        if (body)
            body->print(out, childPrefix, true, false);
    }

    void VarDeclNode::print(ostream &out, const string &prefix,
bool isLast, bool isRoot) const
    {
        printHeader(out, prefix, isLast, isRoot);
    }

    void ConstDeclNode::print(ostream &out, const string &prefix,
bool isLast, bool isRoot) const
    {
        printHeader(out, prefix, isLast, isRoot);
    }

    void TypeDeclNode::print(ostream &out, const string &prefix,
bool isLast, bool isRoot) const
    {
        printHeader(out, prefix, isLast, isRoot);
    }

```

```

void ProcDeclNode::print(ostream &out, const string &prefix,
bool isLast, bool isRoot) const
{
    printHeader(out, prefix, isLast, isRoot);
    string childPrefix = prefix + (isRoot ? "" : (isLast ? "
" : "|  "));
    for (size_t i = 0; i < params.size(); i++)
    {
        if (params[i])
            params[i]->print(out, childPrefix, (i ==
params.size() - 1) && !body, false);
    }
    if (body)
        body->print(out, childPrefix, true, false);
}

void FuncDeclNode::print(ostream &out, const string &prefix,
bool isLast, bool isRoot) const
{
    printHeader(out, prefix, isLast, isRoot);
    string childPrefix = prefix + (isRoot ? "" : (isLast ? "
" : "|  "));
    for (size_t i = 0; i < params.size(); i++)
    {
        if (params[i])
            params[i]->print(out, childPrefix, (i ==
params.size() - 1) && !body, false);
    }
    if (body)
        body->print(out, childPrefix, true, false);
}

void CompoundStmtNode::print(ostream &out, const string
&prefix, bool isLast, bool isRoot) const
{
    printHeader(out, prefix, isLast, isRoot);
    printChildren(out, prefix, isLast, isRoot, statements);
}

```



```

void AssignNode::print(ostream &out, const string &prefix,
bool isLast, bool isRoot) const
{
    printHeader(out, prefix, isLast, isRoot);
    string childPrefix = prefix + (isRoot ? "" : (isLast ? "
" : "|  "));
    if (target)
        target->print(out, childPrefix, !value, false);
    if (value)
        value->print(out, childPrefix, true, false);
}

void BinOpNode::print(ostream &out, const string &prefix,
bool isLast, bool isRoot) const
{
    printHeader(out, prefix, isLast, isRoot);
    string childPrefix = prefix + (isRoot ? "" : (isLast ? "
" : "|  "));
    if (left)
        left->print(out, childPrefix, !right, false);
    if (right)
        right->print(out, childPrefix, true, false);
}

void UnaryOpNode::print(ostream &out, const string &prefix,
bool isLast, bool isRoot) const
{
    printHeader(out, prefix, isLast, isRoot);
    string childPrefix = prefix + (isRoot ? "" : (isLast ? "
" : "|  "));
    if (operand)
        operand->print(out, childPrefix, true, false);
}

void VarNode::print(ostream &out, const string &prefix, bool
isLast, bool isRoot) const
{
    printHeader(out, prefix, isLast, isRoot);

```

```

}

void NumberNode::print(ostream &out, const string &prefix,
bool isLast, bool isRoot) const
{
    printHeader(out, prefix, isLast, isRoot);
}

void CharNode::print(ostream &out, const string &prefix, bool
isLast, bool isRoot) const
{
    printHeader(out, prefix, isLast, isRoot);
}

void StringNode::print(ostream &out, const string &prefix,
bool isLast, bool isRoot) const
{
    printHeader(out, prefix, isLast, isRoot);
}

void ProcCallNode::print(ostream &out, const string &prefix,
bool isLast, bool isRoot) const
{
    printHeader(out, prefix, isLast, isRoot);
    printChildren(out, prefix, isLast, isRoot, args);
}

void IfNode::print(ostream &out, const string &prefix, bool
isLast, bool isRoot) const
{
    printHeader(out, prefix, isLast, isRoot);
    string childPrefix = prefix + (isRoot ? "" : (isLast ? "
" : "|  "));
    bool hasElse = (elseBranch != nullptr);
    if (condition)
        condition->print(out, childPrefix, !thenBranch &&
!hasElse, false);
    if (thenBranch)
        thenBranch->print(out, childPrefix, !hasElse, false);
}

```

```

        if (elseBranch)
            elseBranch->print(out, childPrefix, true, false);
    }

    //
    =====
    // WhileNode
    //
    =====

void WhileNode::print(ostream &out, const string &prefix,
    bool isLast, bool isRoot) const
{
    printHeader(out, prefix, isLast, isRoot);
    string childPrefix = prefix + (isRoot ? "" : (isLast ? "
" : "|  "));
    if (condition)
        condition->print(out, childPrefix, !body, false);
    if (body)
        body->print(out, childPrefix, true, false);
}

void RepeatNode::print(ostream &out, const string &prefix,
    bool isLast, bool isRoot) const
{
    printHeader(out, prefix, isLast, isRoot);
    string childPrefix = prefix + (isRoot ? "" : (isLast ? "
" : "|  "));
    for (size_t i = 0; i < statements.size(); i++)
    {
        if (statements[i])
            statements[i]->print(out, childPrefix, (i ==
statements.size() - 1) && !condition, false);
    }
    if (condition)
        condition->print(out, childPrefix, true, false);
}

void ForNode::print(ostream &out, const string &prefix, bool
isLast, bool isRoot) const

```

```

{
    printHeader(out, prefix, isLast, isRoot);
    string childPrefix = prefix + (isRoot ? "" : (isLast ? "
" : "|  "));
    if (fromExpr)
        fromExpr->print(out, childPrefix, !toExpr && !body,
false);
    if (toExpr)
        toExpr->print(out, childPrefix, !body, false);
    if (body)
        body->print(out, childPrefix, true, false);
}

void CaseNode::print(ostream &out, const string &prefix, bool
isLast, bool isRoot) const
{
    printHeader(out, prefix, isLast, isRoot);
    string childPrefix = prefix + (isRoot ? "" : (isLast ? "
" : "|  "));
    if (selector)
        selector->print(out, childPrefix, branches.empty(),
false);
    for (size_t i = 0; i < branches.size(); i++)
    {
        string label = "CaseBranch(";
        for (size_t j = 0; j < branches[i].first.size(); j++)
        {
            if (j)
                label += ", ";
            label += branches[i].first[j];
        }
        label += ")";
        bool last = (i == branches.size() - 1);
        out << childPrefix << (last ? "└─ " : "├─ ") <<
label << "\n";
        string branchPrefix = childPrefix + (last ? "
" :
"|  ");
        if (branches[i].second)
            branches[i].second->print(out, branchPrefix,

```

```

true, false);
    }
}

void ArrayAccessNode::print(ostream &out, const string
&prefix, bool isLast, bool isRoot) const
{
    printHeader(out, prefix, isLast, isRoot);
    string childPrefix = prefix + (isRoot ? "" : (isLast ? "
" : "|  "));
    if (array)
        array->print(out, childPrefix, indices.empty(),
false);
    for (size_t i = 0; i < indices.size(); i++)
    {
        if (indices[i])
            indices[i]->print(out, childPrefix, i ==
indices.size() - 1, false);
    }
}

void FieldAccessNode::print(ostream &out, const string
&prefix, bool isLast, bool isRoot) const
{
    printHeader(out, prefix, isLast, isRoot);
    string childPrefix = prefix + (isRoot ? "" : (isLast ? "
" : "|  "));
    if (record)
        record->print(out, childPrefix, true, false);
}

string BinOpNode::toSource() const
{
    string lStr = left ? left->toSource() : "";
    string rStr = right ? right->toSource() : "";
    string opStr = op;
    if (op == "plus") opStr = " + ";
    else if (op == "minus") opStr = " - ";
    else if (op == "star") opStr = " * ";

```

```

        else if (op == "slash") opStr = " / ";
        else if (op == "equal") opStr = " = ";
        else if (op == "less") opStr = " < ";
        else if (op == "greater") opStr = " > ";
        else if (op == "lessequal") opStr = " <= ";
        else if (op == "greaterequal") opStr = " >= ";
        else if (op == "notequal") opStr = " <> ";
        return lStr + opStr + rStr;
    }

string UnaryOpNode::toSource() const
{
    string opStr = op;
    if (op == "plus") opStr = " +";
    else if (op == "minus") opStr = " -";
    return opStr + (operand ? operand->toSource() : "");
}

string ArrayAccessNode::toSource() const
{
    string s = array ? array->toSource() : "";
    s += "[";
    for (size_t i = 0; i < indices.size(); i++) {
        if (indices[i]) s += indices[i]->toSource();
        if (i < indices.size() - 1) s += ",";
    }
    s += "]";
    return s;
}

string FieldAccessNode::toSource() const
{
    return (record ? record->toSource() : "") + "." +
fieldName;
}

#include "ASTNode.h"
#include <sstream>

string ASTNode::annotation() const

```



```

&children) const
{
    string childPrefix = prefix + (isRoot ? "" : (isLast ? "
" : "|  "));
    for (size_t i = 0; i < children.size(); i++)
    {
        if (children[i])
            children[i]->print(out, childPrefix, i ==
children.size() - 1, false);
    }
}

void ProgramNode::print(ostream &out, const string &prefix,
bool isLast, bool isRoot) const
{
    printHeader(out, prefix, isLast, isRoot);
    string childPrefix = prefix + (isRoot ? "" : (isLast ? "
" : "|  "));

    if (!declarations.empty())
    {
        bool hasBody = (body != nullptr);
        out << childPrefix << (hasBody ? "└─ " : "└─ ") <<
"Declarations\n";
        string declPrefix = childPrefix + (hasBody ? "|  " :
"  ");
        for (size_t i = 0; i < declarations.size(); i++)
        {
            bool last = (i == declarations.size() - 1);
            if (declarations[i])
                declarations[i]->print(out, declPrefix, last,
false);
        }
    }

    if (body)
        body->print(out, childPrefix, true, false);
}

void VarDeclNode::print(ostream &out, const string &prefix,

```



```

bool isLast, bool isRoot) const
{
    printHeader(out, prefix, isLast, isRoot);
}

void ConstDeclNode::print(ostream &out, const string &prefix,
bool isLast, bool isRoot) const
{
    printHeader(out, prefix, isLast, isRoot);
}

void TypeDeclNode::print(ostream &out, const string &prefix,
bool isLast, bool isRoot) const
{
    printHeader(out, prefix, isLast, isRoot);
}

void ProcDeclNode::print(ostream &out, const string &prefix,
bool isLast, bool isRoot) const
{
    printHeader(out, prefix, isLast, isRoot);
    string childPrefix = prefix + (isRoot ? "" : (isLast ? "
" : "|    "));
    for (size_t i = 0; i < params.size(); i++)
    {
        if (params[i])
            params[i]->print(out, childPrefix, (i ==
params.size() - 1) && !body, false);
    }
    if (body)
        body->print(out, childPrefix, true, false);
}

void FuncDeclNode::print(ostream &out, const string &prefix,
bool isLast, bool isRoot) const
{
    printHeader(out, prefix, isLast, isRoot);
    string childPrefix = prefix + (isRoot ? "" : (isLast ? "
" : "|    "));

```

```

        for (size_t i = 0; i < params.size(); i++)
        {
            if (params[i])
                params[i]->print(out, childPrefix, (i ==
params.size() - 1) && !body, false);
        }
        if (body)
            body->print(out, childPrefix, true, false);
    }

void CompoundStmtNode::print(ostream &out, const string
&prefix, bool isLast, bool isRoot) const
{
    printHeader(out, prefix, isLast, isRoot);
    printChildren(out, prefix, isLast, isRoot, statements);
}

void AssignNode::print(ostream &out, const string &prefix,
bool isLast, bool isRoot) const
{
    printHeader(out, prefix, isLast, isRoot);
    string childPrefix = prefix + (isRoot ? "" : (isLast ? "
" : "|  "));
    if (target)
        target->print(out, childPrefix, !value, false);
    if (value)
        value->print(out, childPrefix, true, false);
}

void BinOpNode::print(ostream &out, const string &prefix,
bool isLast, bool isRoot) const
{
    printHeader(out, prefix, isLast, isRoot);
    string childPrefix = prefix + (isRoot ? "" : (isLast ? "
" : "|  "));
    if (left)
        left->print(out, childPrefix, !right, false);
    if (right)
        right->print(out, childPrefix, true, false);
}

```

```

}

void UnaryOpNode::print(ostream &out, const string &prefix,
bool isLast, bool isRoot) const
{
    printHeader(out, prefix, isLast, isRoot);
    string childPrefix = prefix + (isRoot ? "" : (isLast ? "
" : "|  "));
    if (operand)
        operand->print(out, childPrefix, true, false);
}

void VarNode::print(ostream &out, const string &prefix, bool
isLast, bool isRoot) const
{
    printHeader(out, prefix, isLast, isRoot);
}

void NumberNode::print(ostream &out, const string &prefix,
bool isLast, bool isRoot) const
{
    printHeader(out, prefix, isLast, isRoot);
}

void CharNode::print(ostream &out, const string &prefix, bool
isLast, bool isRoot) const
{
    printHeader(out, prefix, isLast, isRoot);
}

void StringNode::print(ostream &out, const string &prefix,
bool isLast, bool isRoot) const
{
    printHeader(out, prefix, isLast, isRoot);
}

void ProcCallNode::print(ostream &out, const string &prefix,
bool isLast, bool isRoot) const
{

```

```

        printHeader(out, prefix, isLast, isRoot);
        printChildren(out, prefix, isLast, isRoot, args);
    }

void IfNode::print(ostream &out, const string &prefix, bool
isLast, bool isRoot) const
{
    printHeader(out, prefix, isLast, isRoot);
    string childPrefix = prefix + (isRoot ? "" : (isLast ? "
" : "|  "));
    bool hasElse = (elseBranch != nullptr);
    if (condition)
        condition->print(out, childPrefix, !thenBranch &&
!hasElse, false);
    if (thenBranch)
        thenBranch->print(out, childPrefix, !hasElse, false);
    if (elseBranch)
        elseBranch->print(out, childPrefix, true, false);
}

//
=====
// WhileNode
//
=====
void WhileNode::print(ostream &out, const string &prefix,
bool isLast, bool isRoot) const
{
    printHeader(out, prefix, isLast, isRoot);
    string childPrefix = prefix + (isRoot ? "" : (isLast ? "
" : "|  "));
    if (condition)
        condition->print(out, childPrefix, !body, false);
    if (body)
        body->print(out, childPrefix, true, false);
}

void RepeatNode::print(ostream &out, const string &prefix,
bool isLast, bool isRoot) const

```

```

{
    printHeader(out, prefix, isLast, isRoot);
    string childPrefix = prefix + (isRoot ? "" : (isLast ? "
" : "|  "));
    for (size_t i = 0; i < statements.size(); i++)
    {
        if (statements[i])
            statements[i]->print(out, childPrefix, (i ==
statements.size() - 1) && !condition, false);
    }
    if (condition)
        condition->print(out, childPrefix, true, false);
}

void ForNode::print(ostream &out, const string &prefix, bool
isLast, bool isRoot) const
{
    printHeader(out, prefix, isLast, isRoot);
    string childPrefix = prefix + (isRoot ? "" : (isLast ? "
" : "|  "));
    if (fromExpr)
        fromExpr->print(out, childPrefix, !toExpr && !body,
false);
    if (toExpr)
        toExpr->print(out, childPrefix, !body, false);
    if (body)
        body->print(out, childPrefix, true, false);
}

void CaseNode::print(ostream &out, const string &prefix, bool
isLast, bool isRoot) const
{
    printHeader(out, prefix, isLast, isRoot);
    string childPrefix = prefix + (isRoot ? "" : (isLast ? "
" : "|  "));
    if (selector)
        selector->print(out, childPrefix, branches.empty(),
false);
    for (size_t i = 0; i < branches.size(); i++)

```

```

{
    string label = "CaseBranch(";
    for (size_t j = 0; j < branches[i].first.size(); j++)
    {
        if (j)
            label += ",";
        label += branches[i].first[j];
    }
    label += ")";
    bool last = (i == branches.size() - 1);
    out << childPrefix << (last ? "└─ " : "├─ ") <<
label << "\n";
    string branchPrefix = childPrefix + (last ? "    " :
"│    ");
    if (branches[i].second)
        branches[i].second->print(out, branchPrefix,
true, false);
}
}

void ArrayAccessNode::print(ostream &out, const string
&prefix, bool isLast, bool isRoot) const
{
    printHeader(out, prefix, isLast, isRoot);
    string childPrefix = prefix + (isRoot ? "" : (isLast ? "
" : "│    "));
    if (array)
        array->print(out, childPrefix, indices.empty(),
false);
    for (size_t i = 0; i < indices.size(); i++)
    {
        if (indices[i])
            indices[i]->print(out, childPrefix, i ==
indices.size() - 1, false);
    }
}

void FieldAccessNode::print(ostream &out, const string
&prefix, bool isLast, bool isRoot) const

```

```

{
    printHeader(out, prefix, isLast, isRoot);
    string childPrefix = prefix + (isRoot ? "" : (isLast ? "
" : "|  "));
    if (record)
        record->print(out, childPrefix, true, false);
}

string BinOpNode::toSource() const
{
    string lStr = left ? left->toSource() : "";
    string rStr = right ? right->toSource() : "";
    string opStr = op;
    if (op == "plus") opStr = " + ";
    else if (op == "minus") opStr = " - ";
    else if (op == "star") opStr = " * ";
    else if (op == "slash") opStr = " / ";
    else if (op == "equal") opStr = " = ";
    else if (op == "less") opStr = " < ";
    else if (op == "greater") opStr = " > ";
    else if (op == "lessequal") opStr = " <=";
    else if (op == "greaterequal") opStr = " >=";
    else if (op == "notequal") opStr = " <> ";
    return lStr + opStr + rStr;
}

string UnaryOpNode::toSource() const
{
    string opStr = op;
    if (op == "plus") opStr = " +";
    else if (op == "minus") opStr = " -";
    return opStr + (operand ? operand->toSource() : "");
}

string ArrayAccessNode::toSource() const
{
    string s = array ? array->toSource() : "";
    s += "[";
    for (size_t i = 0; i < indices.size(); i++) {

```

```

        if (indices[i]) s += indices[i]->toSource();
        if (i < indices.size() - 1) s += ",";
    }
    s += "];";
    return s;
}

string FieldAccessNode::toSource() const
{
    return (record ? record->toSource() : "") + "." +
    fieldName;
}

```



2.9.3. Implementasi Parse Tree to AST Node

ASTBuilder mengimplementasikan pendekatan Recursive Descent Transformation di mana kelas ini bertindak sebagai penerjemah (translator) yang menelusuri pohon ParseNode (Concrete Syntax Tree) secara depth-first. Berbeda dengan pola Visitor, ASTBuilder mengevaluasi tag string dari setiap node (seperti "<statement>" atau "<expression>") dan mendelegasikan pemrosesan ke fungsi build... yang spesifik. Fungsi-fungsi ini secara rekursif mengekstrak token yang relevan, memanipulasi Symbol Table untuk tipe bentukan seperti array, dan mengonstruksi ASTNode yang memiliki strong typing.

```

#pragma once
#include "lexer.h"
#include "ASTNode.h"
#include "symbolTable.hpp"
#include <memory>
#include <string>
#include <vector>

using namespace std;

class ASTBuilder
{

```



```

public:
    ASTBuilder(SymbolTable &st);
    ASTNodePtr build(shared_ptr<ParseNode> root);

private:
    // Symbol Table
    SymbolTable &st;

    // Program/declarations
    ASTNodePtr buildProgram(shared_ptr<ParseNode> n);
    void buildDeclarationPart(shared_ptr<ParseNode> n, vector<ASTNodePtr>
&out);
    void buildConstDeclaration(shared_ptr<ParseNode> n,
vector<ASTNodePtr> &out);
    void buildTypeDeclaration(shared_ptr<ParseNode> n, vector<ASTNodePtr>
&out);
    void buildVarDeclaration(shared_ptr<ParseNode> n, vector<ASTNodePtr>
&out);
    void buildSubprogramDeclaration(shared_ptr<ParseNode> n,
vector<ASTNodePtr> &out);
    ASTNodePtr buildProcedureDeclaration(shared_ptr<ParseNode> n);
    ASTNodePtr buildFunctionDeclaration(shared_ptr<ParseNode> n);
    ASTNodePtr buildBlock(shared_ptr<ParseNode> n, vector<ASTNodePtr>
&declsOut);

    // Parameters
    void buildFormalParameterList(shared_ptr<ParseNode> n, vector<ASTNodePtr>
&out);
    void buildParameterGroup(shared_ptr<ParseNode> n, vector<ASTNodePtr> &out);

    // Statements
    ASTNodePtr buildCompoundStatement(shared_ptr<ParseNode> n);
    ASTNodePtr buildStatement(shared_ptr<ParseNode> n);
    ASTNodePtr buildAssignmentStatement(shared_ptr<ParseNode> n);
    ASTNodePtr buildIfStatement(shared_ptr<ParseNode> n);
    ASTNodePtr buildWhileStatement(shared_ptr<ParseNode> n);
    ASTNodePtr buildRepeatStatement(shared_ptr<ParseNode> n);
    ASTNodePtr buildForStatement(shared_ptr<ParseNode> n);
    ASTNodePtr buildCaseStatement(shared_ptr<ParseNode> n);
    ASTNodePtr buildProcFuncCall(shared_ptr<ParseNode> n);

    // Expressions
    ASTNodePtr buildExpression(shared_ptr<ParseNode> n);
    ASTNodePtr buildSimpleExpression(shared_ptr<ParseNode> n);
    ASTNodePtr buildTerm(shared_ptr<ParseNode> n);
    ASTNodePtr buildFactor(shared_ptr<ParseNode> n);
    ASTNodePtr buildVariable(shared_ptr<ParseNode> n);

    // Type helpers

    // Resolve a <type> ParseNode to (DataType, atab/btab ref)

```

```

struct TypeInfo { DataType type; int ref; };
TypeInfo resolveType(shared_ptr<ParseNode> typeNode);
TypeInfo resolveArrayType(shared_ptr<ParseNode> arrayNode);
TypeInfo resolveRangeAsAtab(shared_ptr<ParseNode> rangeNode);

// Parse a <constant> ParseNode to its string value + DataType
struct ConstInfo { DataType type; string value; };
ConstInfo resolveConstant(shared_ptr<ParseNode> constNode);

// Utility

// Find first child whose type == tag
static shared_ptr<ParseNode> child(shared_ptr<ParseNode> n, const string
&tag);
// Find all children with type == tag
static vector<shared_ptr<ParseNode>> children(shared_ptr<ParseNode> n,
const string &tag);
// Collect all ident leaves from an <identifier-list>
static vector<string> collectIdents(shared_ptr<ParseNode> identListNode);
};

```

2.9.4. Implementasi Type Sistem

TypeSystem menyediakan fungsi-fungsi untuk validasi dan inferensi tipe: `isCompitable()` memeriksa dua tipe dapat digunakan bersama dalam satu operasi, dengan aturan khusus RECORD dan ARRAY yang mensyaratkan `ref` yang sama agar dinyatakan compatible. `isAssignCompatible()` memeriksa nilai bertipe T2 dapat di-assign ke variabel bertipe T1 dengan satu-satunya pengecualian pelebaran yang diizinkan adalah INTEGER ke REAL. `inferBinOpType()` menentukan tipe hasil operasi biner berdasarkan operator dan tipe operand, operator relasional selalu menghasilkan Boolean dan operator logika and/or hanya valid untuk operand boolean. `inferUnaryOpType()` menentukan tipe hasil operasi unary sedangkan plus/minus unary hanya valid untuk numerik. Seluruh fungsi mengembalikan Unknown jika operand tidak valid yang digunakan sebagai mekanisme error recovery agar analisis dapat dilanjutkan tanpa double error.

```
bool isCompatible(DataType t1, DataType t2, int t1ref, int t2ref)
{
    if (t1 == DataType::UNKNOWN || t2 == DataType::UNKNOWN)
        return true;

    if (t1 == t2)
    {
        if (t1 == DataType::RECORD)
        {
            return (t1ref != 0 && t1ref == t2ref);
        }
        if (t1 == DataType::ARRAY)
        {
            return (t1ref != 0 && t1ref == t2ref);
        }
        return true;
    }

    if (t1 == DataType::SUBRANGE || t2 == DataType::SUBRANGE)
    {
        DataType other = (t1 == DataType::SUBRANGE) ? t2 : t1;
        if (other == DataType::REAL)
            return false;
        return (other == DataType::INTEGER ||
                other == DataType::CHAR ||
                other == DataType::BOOLEAN ||
                other == DataType::SUBRANGE);
    }
}
```

```

bool isAssignCompatible(DataType t1, DataType t2, int t1ref, int t2ref)
{
    if (t1 == DataType::UNKNOWN || t2 == DataType::UNKNOWN)
        return true;
    if (t1 == DataType::REAL && t2 == DataType::INTEGER)
        return true;

    if (t1 == t2)
    {
        if (t1 == DataType::ARRAY)
            return (t1ref != 0 && t1ref == t2ref);
        if (t1 == DataType::RECORD)
            return (t1ref != 0 && t1ref == t2ref);
        return true;
    }

    if (isCompatible(t1, t2, t1ref, t2ref))
    {
        if (t1 == DataType::INTEGER || t1 == DataType::BOOLEAN ||
            t1 == DataType::CHAR || t1 == DataType::SUBRANGE)
        {
            return true;
        }
    }

    if (t1 == DataType::STRING && t2 == DataType::STRING)
    {
        return isCompatible(t1, t2, t1ref, t2ref);
    }

    return false;
}

```

```
DataType inferBinOpType(const std::string &op, DataType leftType, DataType rightType)
{
    if (leftType == DataType::UNKNOWN || rightType == DataType::UNKNOWN)
    {
        return DataType::UNKNOWN;
    }

    if (op == "rdiv")
    {
        if ((leftType == DataType::INTEGER || leftType == DataType::REAL) &&
            (rightType == DataType::INTEGER || rightType == DataType::REAL))
        {
            return DataType::REAL;
        }
        return DataType::UNKNOWN;
    }

    if (op == "idiv" || op == "imod")
    {
        if (leftType == DataType::INTEGER && rightType == DataType::INTEGER)
        {
            return DataType::INTEGER;
        }
        return DataType::UNKNOWN;
    }

    if (op == "plus" || op == "minus" || op == "times")
    {
        if (leftType == DataType::INTEGER && rightType == DataType::INTEGER)
        {
            return DataType::INTEGER;
        }
    }
}
```

```

DataType inferUnaryOpType(const std::string &op, DataType operandType)
{
    if (operandType == DataType::UNKNOWN)
        return DataType::UNKNOWN;

    if (op == "notsy")
    {
        if (operandType == DataType::BOOLEAN)
            return DataType::BOOLEAN;
        return DataType::UNKNOWN;
    }

    if (op == "plus" || op == "minus")
    {
        if (operandType == DataType::INTEGER)
            return DataType::INTEGER;
        if (operandType == DataType::REAL)
            return DataType::REAL;
        return DataType::UNKNOWN;
    }

    return DataType::UNKNOWN;
}

```

2.9.5. Implementais Error Handler

ErrorHandler menyediakan fungsi pelaporan error yang dapat dipanggil dari mana saja di semantic analyzer, seperti undeclaredError() untuk identifier yang dideklarasikan ulang di scope yang sama, redaclarationError() untuk identifier yang dideklarasikan ulang di scope yang sama, assignIncompatibleError() untuk ketidakcocokan tipe pada assignment, invalidOperandError() dan invalidUnaryOperandError() untuk operand yang tidak valid pada operasi biner dan unary, nonBooleanConditionError() untuk kondisi if/while/repeat yang bukan Boolean, invalidIndexTypeError() dan realSubrangeError() untuk pelanggaran aturan index array, serta

`wrongArgCountError()` dan `wrongObjectKindError()` untuk kesalahan pemanggilan prosedur dan fungsi. Error handler mendukung tiga level: WARNING untuk peringatan, ERROR untuk kesalahan yang tidak menghentikan analisis, dan FATAL untuk kesalahan yang menghentikan seluruh proses. Jumlah error dan warning disimpan secara kumulatif melalui `getErrorCount()` dan `getWarningCount()`, sehingga semantic analyzer dapat melanjutkan traversal AST setelah menemukan error dan melaporkan sebanyak mungkin kesalahan dalam satu kali kompilasi.

```
void undeclaredError(const std::string &identName,
                    const SourceLocation &loc)
{
    emit(ErrorLevel::ERROR,
         "Undeclared identifier: '" + identName + "'",
         loc);
}

void redeclarationError(const std::string &identName,
                      const SourceLocation &loc)
{
    emit(ErrorLevel::ERROR,
         "Redeclaration of '" + identName + "' in the same scope",
         loc);
}
```

```

void invalidOperandError(const std::string &op,
                        DataType leftType, DataType rightType,
                        const SourceLocation &loc)
{
    std::ostringstream oss;
    oss << "Invalid operand types for '" << op << "': '"
        << dataTypeToString(leftType) << "' and '"
        << dataTypeToString(rightType) << "'";
    emit(ErrorLevel::ERROR, oss.str(), loc);
}

void invalidUnaryOperandError(const std::string &op,
                             DataType operandType,
                             const SourceLocation &loc)
{
    std::ostringstream oss;
    oss << "Operator '" << op << "' cannot be applied to '"
        << dataTypeToString(operandType) << "'";
    emit(ErrorLevel::ERROR, oss.str(), loc);
}

void nonBooleanConditionError(const std::string &stmtType,
                             DataType got,
                             const SourceLocation &loc)
{
    std::ostringstream oss;
    oss << "Condition of '" << stmtType
        << "' must be Boolean, got '" << dataTypeToString(got) << "'";
    emit(ErrorLevel::ERROR, oss.str(), loc);
}

```



```

void invalidSubrangeError(int low, int high,
                          const SourceLocation &loc)
{
    std::ostringstream oss;
    oss << "Invalid subrange: lower bound " << low
        << " > upper bound " << high;
    emit(ErrorLevel::ERROR, oss.str(), loc);
}

void invalidIndexTypeError(DataType indexType,
                           const SourceLocation &loc)
{
    std::ostringstream oss;
    oss << "Invalid array index type: '"
        << dataTypeToString(indexType)
        << "' is not allowed as index type (Real is forbidden)";
    emit(ErrorLevel::ERROR, oss.str(), loc);
}

void wrongArgCountError(const std::string &procName,
                        int expected, int got,
                        const SourceLocation &loc)
{
    std::ostringstream oss;
    oss << "Wrong number of arguments for '" << procName
        << "': expected " << expected << ", got " << got;
    emit(ErrorLevel::ERROR, oss.str(), loc);
}

```

```

void wrongObjectKindError(const std::string &identName,
                          AllowedObj expected, AllowedObj got,
                          const SourceLocation &loc)
{
    std::ostringstream oss;
    oss << "'" << identName << "' is a "
        << SymbolTable::AllowedObjToString(got)
        << ", expected a "
        << SymbolTable::AllowedObjToString(expected);
    emit(ErrorLevel::ERROR, oss.str(), loc);
}

void realSubrangeError(const SourceLocation &loc)
{
    emit(ErrorLevel::ERROR,
        "Subrange type cannot be Real",
        loc);
}

int getErrorCount() { return s_errorCount; }
int getWarningCount() { return s_warningCount; }
bool hasFatalError() { return s_fatalFlag; }

void resetErrorHandler()
{
    s_errorCount = 0;
    s_warningCount = 0;
    s_fatalFlag = false;
}

```

2.9.6. Implementasi Visit Function

Semantic Analyzer mengimplementasikan Visitor Pattern di mana setiap jenis node AST memiliki fungsi `visit` yang bersesuaian. Fungsi-fungsi ini dipanggil secara rekursif selama transversal DFS dan bertanggung jawab untuk mendaftarkan identifier, melakukan pengecekan tipe, dan mengatasi node.

2.9.6.1. analyze() dan visitProgram()

analyze() adalah entry point yang dipanggil dari main.cpp. Fungsi ini melakukan cast root AST ke ProgramNode. Jika cast gagal, error langsung dilaporkan dan analisis dihentikan.

1. visitProgram() adalah fungsi yang pertama kali dipanggil saat analisis dimulai. Urutannya adalah sebagai berikut.
2. Daftarkan nama program ke tab dengan obj=PROGRAM, type=VOID, lev=0.
3. Anotasi node dengan tabIndex, lexLevel, dan dtype.
4. Loop seluruh declarations dan panggil visitStatement() untuk masing-masing.
5. Setelah semua deklarasi selesai, panggil visitCompoundStmt() untuk body.

Deklarasi diproses sebelum body karena body akan memakai identifier yang baru saja didaftarkan. Jika body diproses lebih dulu, semua lookup identifier di body akan gagal.

```
void SemanticAnalyzer::analyze(ASTNodePtr root)
{
    if (!root)
    {
        semanticError("AST is empty, there is nothing to be analyzed.");
        return;
    }

    ProgramNode *prog = dynamic_cast<ProgramNode *>(root.get());
    if (!prog)
    {
        semanticError("AST root is not a ProgramNode.");
        return;
    }

    visitProgram(prog);
}
```

```

void SemanticAnalyzer::visitProgram(ProgramNode *node)
{
    int idx = st.enterTab(
        node->name,
        AllowedObj::PROGRAM,
        DataType::VOID,
        0,
        1,
        0,
        0
    );

    node->tabIndex = idx;
    node->lexLevel = st.currentLevel; // = 0
    node->dtype = DataType::VOID;

    for (auto &decl : node->declarations)
    {
        if (decl) visitStatement(decl.get());
    }

    int bodyBlock = st.pushScope();
    st.tab[idx].ref = bodyBlock;

    if (node->body)
        visitCompoundStmt(dynamic_cast<CompoundStmtNode *>(node->body.get()));

    st.popScope();
}

```

2.9.6.2. visitVarDecl()

visitVarDecl() memproses satu deklarasi variabel. Langkah-langkahnya adalah sebagai berikut.

1. Panggil lookupCurrentScope() untuk mendeteksi redeclaration. Jika ditemukan, panggil redeclarationError() dan return.
2. Ambil varType dan typeRef dari node (sudah diisi Orang 2).
3. Panggil enterTab() dengan obj=VARIABLE, type=varType, lev=currentLevel, adr=0.
4. Anotasi node dengan tabIndex, lexLevel, dan dtype.

```

void SemanticAnalyzer::visitVarDecl(VarDeclNode *node)
{
    if (st.lookupCurrentScope(node->varName) != -1)
    {
        redeclarationError(node->varName);
        node->dtype = DataType::UNKNOWN;
        return;
    }

    int typeRef = node->typeRef;
    DataType finalType = node->varType;

    int offset = st.btab[st.currentBlock].vsze;

    int sz = elementSize(finalType, typeRef);
    st.btab[st.currentBlock].vsze += sz;

    int idx = st.enterTab(
        node->varName,
        AllowedObj::VARIABLE,
        finalType,
        typeRef,
        1,
        st.currentLevel,
        offset
    );

    node->tabIndex = idx;
    node->lexLevel = st.currentLevel;
    node->dtype = finalType;
}

```

2.9.6.3. visitConstDecl()

visitConstDecl() memproses satu deklarasi konstanta. Bedanya dari visitVarDecl() adalah obj=CONSTANT dan field adr diisi dengan nilai konstanta (bukan offset), karena konstanta tidak punya alamat di stack. Untuk tipe INTEGER dan BOOLEAN, nilai konstanta dikonversi ke integer menggunakan stoi(). Untuk tipe lain (REAL, CHAR, STRING), adr diisi 0 dan nilai sebenarnya tersimpan di node.

```

void SemanticAnalyzer::visitConstDecl(ConstDeclNode *node)
{
    if (st.lookupCurrentScope(node->constName) != -1)
    {
        redeclarationError(node->constName);
        node->dtype = DataType::UNKNOWN;
        return;
    }

    DataType t = node->constType;

    int adrValue = 0;
    if (t == DataType::INTEGER || t == DataType::BOOLEAN)
    {
        try { adrValue = std::stoi(node->value); }
        catch (...) { adrValue = 0; }
    }

    int idx = st.enterTab(
        node->constName,
        AllowedObj::CONSTANT,
        t,
        0,
        1,
        st.currentLevel,
        adrValue
    );

    node->tabIndex = idx;
    node->lexLevel = st.currentLevel;
    node->dtype = t;
}

```

2.9.6.4. visitTypeDecl()

visitTypeDecl() memproses deklarasi tipe baru seperti alias atau array. Setelah cek redeclaration, fungsi ini melakukan validasi tambahan untuk tipe ARRAY: index type tidak boleh bertipe REAL. Validasi ini dilakukan dengan memanggil isValidIndexType() milik Orang 3 pada atab[ref].xtyp. Jika tidak valid, invalidIndexTypeError() dipanggil namun proses tetap dilanjutkan agar analisis bisa menemukan error lain.

```

void SemanticAnalyzer::visitTypeDecl(TypeDeclNode *node)
{
    if (st.lookupCurrentScope(node->typeName) != -1)
    {
        redeclarationError(node->typeName);
        node->dtype = DataType::UNKNOWN;
        return;
    }

    int ref = node->typeRef;

    // Validasi index type untuk ARRAY
    if (node->baseType == DataType::ARRAY &&
        ref >= 0 && ref < (int)st.atab.size())
    {
        if (!isValidIndexType(st.atab[ref].xtyp))
            invalidIndexTypeError(st.atab[ref].xtyp);
    }

    int idx = st.enterTab(
        node->typeName,
        AllowedObj::TYPE,
        node->baseType,
        ref,
        1,
        st.currentLevel,
        0
    );

    node->tabIndex = idx;
    node->lexLevel = st.currentLevel;
    node->dtype = node->baseType;
}

```

2.9.6.5. visitProcDecl()

visitProcDecl() adalah fungsi yang paling kompleks di bagian deklarasi karena harus mengelola scope baru. Urutannya adalah sebagai berikut.

1. Cek redeclaration di scope saat ini.
2. Daftarkan nama prosedur di scope luar (sebelum pushScope) dengan obj=PROCEDURE, type=VOID. Ini penting agar

prosedur bisa dipanggil secara rekursif atau dari scope yang sama.

3. Panggil `pushScope()` untuk membuat scope baru. Simpan indeks btab baru ke `st.tab[procIdx].ref`.
4. Proses setiap parameter: cek redeclaration, tentukan nrm (by-value atau by-reference), lalu panggil `enterTab()` di level baru.
5. Setelah semua parameter diproses, simpan indeks parameter terakhir ke `btab[newBlock].lpar`.
6. Anotasi node prosedur. `lexLevel` diisi dengan `currentLevel - 1` karena prosedur dideklarasikan di scope luar.
7. Panggil `visitCompoundStmt()` untuk body prosedur.
8. Panggil `popScope()` untuk kembali ke scope sebelumnya.

2.9.6.6. visitFuncDecl()

`visitFuncDecl()` hampir identik dengan `visitProcDecl()`. Perbedaanannya adalah `obj=FUNCTION` dan `type` diisi dengan `returnType` (bukan `VOID`). Informasi return type ini penting saat memvalidasi ekspresi return di dalam body fungsi.


```

void SemanticAnalyzer::visitFuncDecl(FuncDeclNode *node)
{
    if (st.lookupCurrentScope(node->funcName) != -1)
    {
        redeclarationError(node->funcName);
        return;
    }

    int declLevel = st.currentLevel;

    int funcIdx = st.enterTab(
        node->funcName,
        AllowedObj::FUNCTION,
        node->returnType,
        0,          // ref diisi setelah pushScope
        1,
        declLevel,
        0
    );

    int newBlock = st.pushScope();
    st.tab[funcIdx].ref = newBlock;

    for (auto &param : node->params)
    {
        if (!param) continue;

        VarDeclNode *p = dynamic_cast<VarDeclNode *>(param.get());
        if (!p) continue;

        if (st.lookupCurrentScope(p->varName) != -1)
        {
            redeclarationError(p->varName);
            continue;
        }

        int nrm = (p->typeRef < 0) ? 0 : 1;
        int tref = (p->typeRef < 0) ? 0 : p->typeRef;
    }
}

```

```

    int offset = st.btab[newBlock].psze;
    st.btab[newBlock].psze += elementSize(p->varType, tref);

    int pIdx = st.enterTab(
        p->varName,
        AllowedObj::VARIABLE,
        p->varType,
        tref,
        nrm,
        st.currentLevel,
        offset
    );

    p->tabIndex = pIdx;
    p->lexLevel = st.currentLevel;
    p->dtype    = p->varType;
}

st.btab[newBlock].lpar = st.btab[newBlock].last;

node->tabIndex = funcIdx;
node->lexLevel = declLevel;
node->dtype    = node->returnType;

if (node->body)
    visitCompoundStmt(dynamic_cast<CompoundStmtNode *>(node->body.get()));

st.popScope();
}

```

2.9.6.7. visitStatement()

visitStatement() adalah fungsi dispatcher yang menerima sembarang ASTNode dan menentukan visit function mana yang harus dipanggil. Penentuan dilakukan menggunakan dynamic_cast secara berurutan. Jika cast berhasil, fungsi visit yang sesuai dipanggil dan fungsi langsung return. Jika tidak ada cast yang cocok, node dianggap milik statement atau ekspresi.

```

void SemanticAnalyzer::visitStatement(ASTNode *node)
{
    if (!node) return;

    if (auto *n = dynamic_cast<VarDeclNode *>(node)) { visitVarDecl(n); return; }
    if (auto *n = dynamic_cast<ConstDeclNode *>(node)) { visitConstDecl(n); return; }
    if (auto *n = dynamic_cast<TypeDeclNode *>(node)) { visitTypeDecl(n); return; }
    if (auto *n = dynamic_cast<ProcDeclNode *>(node)) { visitProcDecl(n); return; }
    if (auto *n = dynamic_cast<FuncDeclNode *>(node)) { visitFuncDecl(n); return; }

    // --- Statement & Ekspresi (Orang 5) ---
    if (auto *n = dynamic_cast<AssignNode *>(node)) { visitAssign(n); return; }
    if (auto *n = dynamic_cast<IfNode *>(node)) { visitIf(n); return; }
    if (auto *n = dynamic_cast<WhileNode *>(node)) { visitWhile(n); return; }
    if (auto *n = dynamic_cast<ForNode *>(node)) { visitFor(n); return; }
    if (auto *n = dynamic_cast<RepeatNode *>(node)) { visitRepeat(n); return; }
    if (auto *n = dynamic_cast<CaseNode *>(node)) { visitCase(n); return; }
    if (auto *n = dynamic_cast<ProcCallNode *>(node)) { visitProcCall(n); return; }
    if (auto *n = dynamic_cast<CompoundStmtNode *>(node)) { visitCompoundStmt(n); return; }

    // --- Expressions ---
    if (auto *n = dynamic_cast<VarNode *>(node)) { visitVar(n); return; }
    if (auto *n = dynamic_cast<BinOpNode *>(node)) { visitBinOp(n); return; }

    if (auto *n = dynamic_cast<ArrayAccessNode *>(node)) { visitArrayAccess(n); return; }
    if (auto *n = dynamic_cast<NumberNode *>(node)) { /* dtype sudah di-set saat build */ return; }
    if (auto *n = dynamic_cast<StringNode *>(node)) { /* dtype sudah di-set saat build */ return; }
    if (auto *n = dynamic_cast<CharNode *>(node)) { /* dtype sudah di-set saat build */ return; }
    if (auto *n = dynamic_cast<UnaryOpNode *>(node)) { visitUnaryOp(n); return; }
}

```

2.9.6.8. resolveTypeName()

resolveTypeName() adalah helper yang mengubah nama tipe dalam bentuk string menjadi DataType dan ref yang sesuai. Fungsi ini berguna ketika mengisi tipe sebagai nama string dan perlu di-resolve ke symbol table. Prosesnya adalah lookup nama di symbol table, validasi bahwa obj-nya adalah TYPE, lalu kembalikan type dan ref.

```

DataType SemanticAnalyzer::resolveTypeName(const std::string &typeName, int &outRef)
{
    outRef = 0;

    int idx = st.lookup(typeName);
    if (idx == -1)
    {
        undeclaredError(typeName);
        return DataType::UNKNOWN;
    }

    if (st.tab[idx].obj != AllowedObj::TYPE)
    {
        wrongObjectKindError(typeName, AllowedObj::TYPE, st.tab[idx].obj);
        return DataType::UNKNOWN;
    }

    outRef = st.tab[idx].ref;
    return st.tab[idx].type;
}

```

2.9.6.9. visitAssign()

visitAssign() adalah fungsi untuk mengevaluasi operasi pemberian nilai. Fungsi ini melakukan evaluasi target dan value, kemudian mengecek apakah operasi pemberian/perubahan nilai dapat terjadi. Terakhir, fungsi memvalidasi apakah tipe data target dan value kompatibel menggunakan aturan isAssignCompatible().

```
void SemanticAnalyzer::visitAssign(AssignNode *node)
{
    if (node->target)
        visitStatement(node->target.get());
    if (node->value)
        visitStatement(node->value.get());

    if (node->target && node->value)
    {
        std::string varName = "";

        if (auto *v = dynamic_cast<VarNode *>(node->target.get()))
        {
            varName = v->varName;
            if (v->tabIndex != -1)
            {
                AllowedObj targetObj = st.tab[v->tabIndex].obj;

                // Cek: assignment ke constant tidak diizinkan
                if (targetObj == AllowedObj::CONSTANT)
                {
                    semanticError("Cannot assign a value to constant '" + varName + "'.");
                    node->dtype = DataType::VOID;
                    return;
                }

                if (targetObj == AllowedObj::FUNCTION)
                {
                    int funcLev = st.tab[v->tabIndex].lev;
                    if (funcLev != st.currentLevel - 1)
                    {
                        semanticError("Cannot assign to function '" + varName + "' outside its own body.");
                        node->dtype = DataType::VOID;
                        return;
                    }
                }
            }
        }
    }
}
```

```

        if (!isAssignCompatible(node->target->dtype,
node->value->dtype))
        {
            if (node->value->dtype != DataType::VOID)
            {
                int targetRef = node->target->typeRef;
                int valueRef = node->value->typeRef;

                if (!isAssignCompatible(node->target->dtype,
node->value->dtype, targetRef, valueRef))
                    assignIncompatibleError(node->target->dtype,
node->value->dtype, varName);
            }
            else
            {
                assignIncompatibleError(node->target->dtype,
node->value->dtype, varName);
            }
        }
        node->dtype = DataType::VOID;
    }
}

```

2.9.6.10. visitBinOp()

visitBinOp(): adalah fungsi untuk mengevaluasi operasi biner (seperti aritmatika atau relasional). Fungsi ini mengunjungi sisi left dan right terlebih dahulu, lalu mendelegasikan penentuan tipe hasil operasi ke fungsi inferBinOpType di TypeSystem. Jika tipe operan tidak cocok, akan memicu error.

```

void SemanticAnalyzer::visitBinOp(BinOpNode *node)
{
    if (node->left) visitStatement(node->left.get());
    if (node->right) visitStatement(node->right.get());

    if (node->left && node->right)
    {
        DataType inferredType = inferBinOpType(node->op,
node->left->dtype, node->right->dtype);
        if (inferredType == DataType::UNKNOWN)
        {
            if (node->left->dtype != DataType::UNKNOWN &&
node->right->dtype != DataType::UNKNOWN)
            {
                invalidOperandError(node->op, node->left->dtype,
node->right->dtype);
            }
        }
        node->dtype = inferredType;
    }
}

```

```

    }
    else
    {
        node->dtype = DataType::UNKNOWN;
    }
}

```

2.9.6.11. visitVar()

visitVar() adalah fungsi untuk memproses penggunaan nama identifier (variabel, konstanta, atau fungsi). Fungsi ini melakukan lookup ke Symbol Table, serta mengekstrak dan menyimpan informasi (seperti indeks, lexical level, typeRef, dan tipe data) ke dalam node.

```

void SemanticAnalyzer::visitVar(VarNode *node)
{
    int idx = st.lookup(node->varName);
    if (idx == -1)
    {
        undeclaredError(node->varName);
        node->dtype = DataType::UNKNOWN;
        return;
    }

    AllowedObj obj = st.tab[idx].obj;
    if (obj != AllowedObj::VARIABLE && obj != AllowedObj::CONSTANT &&
obj != AllowedObj::FUNCTION)
    {
        wrongObjectKindError(node->varName, AllowedObj::VARIABLE,
obj);
        node->dtype = DataType::UNKNOWN;
        return;
    }

    node->tabIndex = idx;
    node->lexLevel = st.tab[idx].lev;
    node->dtype = st.tab[idx].type;
    node->typeRef = st.tab[idx].ref;
}

```

2.9.6.12. visitIf()

visitIf() adalah fungsi untuk mengevaluasi struktur if-then-else. Fungsi ini melakukan pengecekan conditional statement, memastikan ekspresi blok if merupakan boolean, dan melakukan visit pada blok then-else.

```

void SemanticAnalyzer::visitIf(IfNode *node)
{
    if (node->condition)

```

```

    {
        visitStatement(node->condition.get());
        if (node->condition->dtype != DataType::BOOLEAN &&
node->condition->dtype != DataType::UNKNOWN)
        {
            nonBooleanConditionError("if", node->condition->dtype);
        }
    }

    if (node->thenBranch) visitStatement(node->thenBranch.get());
    if (node->elseBranch) visitStatement(node->elseBranch.get());

    node->dtype = DataType::VOID;
}

```

2.9.6.13. visitWhile()

visitWhile() adalah fungsi untuk mengevaluasi struktur while-do. Fungsi ini melakukan pengecekan conditional statement, memastikan ekspresi blok while merupakan boolean, dan melakukan visit pada blok do.

```

void SemanticAnalyzer::visitWhile(WhileNode *node)
{
    if (node->condition)
    {
        visitStatement(node->condition.get());
        if (node->condition->dtype != DataType::BOOLEAN &&
node->condition->dtype != DataType::UNKNOWN)
        {
            nonBooleanConditionError("while",
node->condition->dtype);
        }
    }

    if (node->body) visitStatement(node->body.get());

    node->dtype = DataType::VOID;
}

```

2.9.6.14. visitFor()

visitFor() adalah fungsi untuk mengevaluasi struktur for-loop. Fungsi ini melakukan pengecekan conditional statement, memastikan ekspresi pada blok for (from, to) merupakan boolean, dan melakukan visit pada body dari for-loop.

```

void SemanticAnalyzer::visitFor(ForNode *node)

```

```

{
    int idx = st.lookup(node->varName);
    DataType controlType = DataType::UNKNOWN;

    if (idx == -1) undeclaredError(node->varName);
    else
    {
        if (st.tab[idx].obj != AllowedObj::VARIABLE)
        {
            wrongObjectKindError(node->varName, AllowedObj::VARIABLE,
st.tab[idx].obj);
        }
        else
        {
            controlType = st.tab[idx].type;
        }
    }

    if (node->fromExpr)
    {
        visitStatement(node->fromExpr.get());
        if (controlType != DataType::UNKNOWN && node->fromExpr->dtype
!= DataType::UNKNOWN)
        {
            if (!isAssignCompatible(controlType,
node->fromExpr->dtype))
            {
                assignIncompatibleError(controlType,
node->fromExpr->dtype, node->varName);
            }
        }
    }

    if (node->toExpr)
    {
        visitStatement(node->toExpr.get());
        if (controlType != DataType::UNKNOWN && node->toExpr->dtype
!= DataType::UNKNOWN)
        {
            if (!isAssignCompatible(controlType,
node->toExpr->dtype))
            {
                assignIncompatibleError(controlType,
node->toExpr->dtype, node->varName);
            }
        }
    }

    if (node->body) visitStatement(node->body.get());

    node->dtype = DataType::VOID;
}

```


2.9.6.15. visitRepeat()

visitRepeat() adalah fungsi untuk mengevaluasi struktur repeat-until. Fungsi ini melakukan visit pada blok statement, kemudian melakukan pengecekan conditional statement, memastikan ekspresi blok while (condition) merupakan boolean yang valid.

```
void SemanticAnalyzer::visitRepeat(RepeatNode *node)
{
    for (auto &stmt : node->statements)
    {
        if (stmt) visitStatement(stmt.get());
    }

    if (node->condition)
    {
        visitStatement(node->condition.get());
        if (node->condition->dtype != DataType::BOOLEAN &&
            node->condition->dtype != DataType::UNKNOWN)
        {
            nonBooleanConditionError("repeat",
node->condition->dtype);
        }
    }

    node->dtype = DataType::VOID;
}
```

2.9.6.16. visitCase()

visitCase() adalah fungsi untuk mengevaluasi struktur switch/case. Fungsi ini mengevaluasi ekspresi selector dan memvalidasi secara ketat berdasarkan tipe data tipe ordinal (seperti integer, char, boolean, subrange, atau enumerated), kemudian fungsi memproses setiap branch di dalamnya.

```
void SemanticAnalyzer::visitCase(CaseNode *node)
{
    if (node->selector)
    {
        visitStatement(node->selector.get());

        DataType stype = node->selector->dtype;
```

```

        if (stype != DataType::UNKNOWN &&
            stype != DataType::INTEGER &&
            stype != DataType::CHAR &&
            stype != DataType::BOOLEAN &&
            stype != DataType::SUBRANGE &&
            stype != DataType::ENUMERATED)
        {
            semanticError("Case selector must be of ordinal type, got " +
                dataTypeToString(stype) + "");
        }
    }

    for (auto &branch : node->branches)
    {
        if (branch.second)
            visitStatement(branch.second.get());
    }

    node->dtype = DataType::VOID;
}

```

2.9.6.17. visitProcCall()

visitProcCall() adalah fungsi untuk mengevaluasi pemanggilan fungsi atau prosedur. Fungsi ini memvalidasi bahwa nama identfier adalah PROCEDURE atau FUNCTION, menelusuri Block Table (btb) untuk menghitung jumlah parameter asli, membandingkannya dengan argumen yang diberikan, dan memverifikasi kompatibilitas tipe data dari setiap argumen satu per satu.

```

void SemanticAnalyzer::visitProcCall(ProcCallNode *node)
{
    int idx = st.lookup(node->procName);
    if (idx == -1)
    {
        undeclaredError(node->procName);
        node->dtype = DataType::UNKNOWN;
        return;
    }

    AllowedObj obj = st.tab[idx].obj;
    if (obj != AllowedObj::PROCEDURE && obj !=
AllowedObj::FUNCTION)
    {
        wrongObjectKindError(node->procName,
AllowedObj::PROCEDURE, obj);
    }
}

```

```

        node->dtype = DataType::UNKNOWN;
        return;
    }

    for (auto &arg : node->args)
        if (arg)
            visitStatement(arg.get());

        if (idx == PredefinedIdx::PROC_WRITELN || idx ==
PredefinedIdx::PROC_READLN)
    {
        node->dtype = DataType::VOID;
        return;
    }

    int ref = st.tab[idx].ref;

    std::vector<int> params;
    int p = st.btab[ref].lpar;
    while (p > 0)
    {
        params.push_back(p);
        p = st.tab[p].link;
    }
    std::reverse(params.begin(), params.end());

    int paramCount = (int)params.size();

    if ((int)node->args.size() != paramCount)
    {
        wrongArgCountError(node->procName, paramCount,
(int)node->args.size());
    }
    else
    {
        for (size_t i = 0; i < node->args.size(); i++)
        {
            if (!node->args[i])
                continue;
            if (node->args[i]->dtype == DataType::UNKNOWN)
                continue;

            DataType expectedType = st.tab[params[i]].type;
            int expectedRef = st.tab[params[i]].ref;

            int argRef = node->args[i]->typeRef;
            if (auto *v = dynamic_cast<VarNode
*>(node->args[i].get()))
                argRef = (v->tabIndex >= 0) ?
st.tab[v->tabIndex].ref : 0;

```

```

        if (!isAssignCompatible(expectedType,
node->args[i]->dtype,
                                expectedRef, argRef))
        {
            assignIncompatibleError(expectedType,
node->args[i]->dtype,
                                node->procName);
        }
    }
}

node->dtype = (st.tab[idx].obj == AllowedObj::FUNCTION)
              ? st.tab[idx].type
              : DataType::VOID;
}

```

2.9.6.18. visitArrayAccess()

visitArrayAccess() adalah fungsi untuk mengevaluasi pengaksesan elemen array, termasuk array multi-dimensi (seperti A[1, 2] atau A[1][2]). Fungsi ini mengevaluasi array dasar dan seluruh indeksinya, lalu melakukan iterasi menelusuri Array Table (atab) menggunakan typeRef untuk mendeduksi tipe elemen secara bertahap, sekaligus memvalidasi bahwa setiap indeks menggunakan tipe data ordinal yang sah.

```

void SemanticAnalyzer::visitArrayAccess(ArrayAccessNode *node) {
    if (node->array) visitStatement(node->array.get());

    for (auto &idx : node->indices)
        if (idx) visitStatement(idx.get());

    DataType currentType = node->array ? node->array->dtype :
    DataType::UNKNOWN;
    int currentRef = node->array ? node->array->typeRef : 0;

    for (size_t i = 0; i < node->indices.size(); i++)
    {
        if (currentType == DataType::ARRAY && currentRef >= 0 &&
currentRef < (int)st.atab.size())
        {
            // Validasi tipe indeks
            if (node->indices[i])
            {
                DataType idxType = node->indices[i]->dtype;
                if (idxType != DataType::UNKNOWN &&
!isValidIndexType(idxType))
                    invalidIndexTypeError(idxType);
            }
        }
    }
}

```

```

    }

    currentType = st.atab[currentRef].etyp;
    currentRef = st.atab[currentRef].eref;
}
else
{
    if (currentType != DataType::UNKNOWN)
        semanticError("Cannot index a non-array type.");
    currentType = DataType::UNKNOWN;
    currentRef = 0;
    break;
}
}

node->dtype = currentType;
node->typeRef = currentRef;
}

```

2.9.6.19. visitUnaryOp()

visitUnaryOp() adalah fungsi untuk mengevaluasi operasi dengan satu operan (seperti +, -, atau not). Fungsi ini mengevaluasi operan tersebut terlebih dahulu, lalu mendelegasikan penentuan dan validasi tipe data ke fungsi inferUnaryOpType di TypeSystem untuk memastikan bahwa operan memiliki tipe data yang kompatibel dengan operator yang digunakan.

```

void SemanticAnalyzer::visitUnaryOp(UnaryOpNode *node)
{
    if (node->operand)
        visitStatement(node->operand.get());

    DataType opType = node->operand ? node->operand->dtype :
    DataType::UNKNOWN;

    DataType inferred = inferUnaryOpType(node->op, opType);

    if (inferred == DataType::UNKNOWN && opType != DataType::UNKNOWN)
        invalidUnaryOperandError(node->op, opType);

    node->dtype = inferred;
}

```


BAB III

PENGUJIAN

3.1. Metode Pengujian

Pengujian dilakukan dengan menjalankan program compiler Arion menggunakan command:

```
./bin/program <input.txt> <output.txt>
```

Program akan mencetak Parse Tree dan Symbol Table ke terminal, serta menyimpan Decorated AST ke file output.

3.2. Hasil Pengujian

3.2.1. Kasus Uji 1

- Kasus: Token berupa angka yang langsung diikuti huruf (6e) harus dipisah menjadi dua token terpisah, bukan satu token unknown.
- Tujuan: Menguji bahwa lexer memisah NonSymbol yang berdekatan dan beda tipe secara benar.

- Input:

```
program TestNonSymbol;  
var  
  e: integer;  
begin  
  e := 6e;  
end.
```

- Output:

```
Syntax error at line 5, col 10: unexpected token 'ident(e)',  
expected endsy
```

- Analisis: Error unexpected token 'ident(e)' membuktikan 6e berhasil dipisah menjadi intcon(6) dan ident(e). Jika belum dipisah, error yang muncul adalah unknown(6e).

3.2.2. Kasus Uji 2

- Kasus: Komentar dalam bentuk { } dan (* *) tidak boleh menjadi node dalam parse tree.
- Tujuan: Menguji bahwa parser mengabaikan komentar dan tidak menyertakannya sebagai node.
- Input:

```
program TestComment;
var
  x: integer;
begin
  { ini komentar }
  x := 5; { komentar inline }
  (* komentar style 2 *)
  x := x + 1;
end.
```

- Output:

=== Symbol Table ===

TAB (Identifier Table)

idx	name	obj	type	ref	nrm	lev	adr	link
1	AND	keyword	Unknown	0	1	0	0	0
2	ARRAY	keyword	Unknown	0	1	0	0	1
3	BEGIN	keyword	Unknown	0	1	0	0	2
4	CASE	keyword	Unknown	0	1	0	0	3
5	CONST	keyword	Unknown	0	1	0	0	4
6	DIV	keyword	Unknown	0	1	0	0	5
7	DOWNTTO	keyword	Unknown	0	1	0	0	6
8	DO	keyword	Unknown	0	1	0	0	7
9	ELSE	keyword	Unknown	0	1	0	0	8
10	END	keyword	Unknown	0	1	0	0	9
11	FOR	keyword	Unknown	0	1	0	0	10
12	FUNCTION	keyword	Unknown	0	1	0	0	11
13	IF	keyword	Unknown	0	1	0	0	12
14	MOD	keyword	Unknown	0	1	0	0	13
15	NOT	keyword	Unknown	0	1	0	0	14
16	OF	keyword	Unknown	0	1	0	0	15

17	OR	keyword	Unknown	0	1	0	0	16
18	PROCEDURE	keyword	Unknown	0	1	0	0	17
19	PROGRAM	keyword	Unknown	0	1	0	0	18
20	RECORD	keyword	Unknown	0	1	0	0	19
21	REPEAT	keyword	Unknown	0	1	0	0	20
22	INTEGER	type	Integer	0	1	0	0	21
23	REAL	type	Real	0	1	0	0	22
24	BOOLEAN	type	Boolean	0	1	0	0	23
25	CHAR	type	Char	0	1	0	0	24
26	STRING	type	String	0	1	0	0	25
27	THEN	keyword	Unknown	0	1	0	0	26
28	TO	keyword	Unknown	0	1	0	0	27
29	TYPE	keyword	Unknown	0	1	0	0	28
30	UNTIL	keyword	Unknown	0	1	0	0	29
31	VAR	keyword	Unknown	0	1	0	0	30
32	WHILE	keyword	Unknown	0	1	0	0	31
33	true	constant	Boolean	0	1	0	1	32
34	false	constant	Boolean	0	1	0	0	33
35	writeln	procedure	Void	0	1	0	0	34
36	readln	procedure	Void	0	1	0	0	35
37	TestComment	program	Void	1	1	0	0	36
38	x	variable	Integer	0	1	1	0	0

BTAB (Block Table)

idx	last	lpar	psze	vsze

0	37	0	0	0
1	38	0	0	1

ATAB (Array Table)

(kosong)

=== Semantic Analysis ===

Errors : 0

Warnings: 0

Status : OK

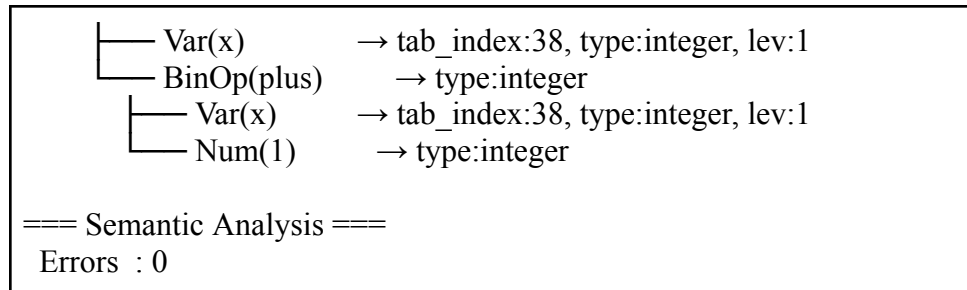
=== Decorated AST ===

ProgramNode(name: 'TestComment')→ tab_index:37, type:void, lev:0

```

├── Declarations
│   └── VarDecl('x') → tab_index:38, type:integer, lev:1
├── CompoundStmt → type:void
│   ├── Assign(x := 5) → type:void
│   │   ├── Var(x) → tab_index:38, type:integer, lev:1
│   │   └── Num(5) → type:integer
│   └── Assign(x := x + 1) → type:void

```



- Analisis: Tidak ada node comment dalam parse tree maupun Decorated AST. Program berjalan normal tanpa error.

3.2.3. Kasus Uji 3

- Kasus: Statement kosong (hanya semicolon) dalam blok begin...end harus dianggap valid.
- Tujuan: Menguji bahwa parser menerima empty statement sebagai statement yang valid.
- Input:

```

program TestEmpty;
begin
;
end.

```

- Output:

=== Symbol Table ===

TAB (Identifier Table)

idx	name	obj	type	ref	nrm	lev	adr	link

1	AND	keyword	Unknown	0	1	0	0	0
2	ARRAY	keyword	Unknown	0	1	0	0	1
3	BEGIN	keyword	Unknown	0	1	0	0	2
4	CASE	keyword	Unknown	0	1	0	0	3
5	CONST	keyword	Unknown	0	1	0	0	4
6	DIV	keyword	Unknown	0	1	0	0	5
7	DOWNT0	keyword	Unknown	0	1	0	0	6
8	DO	keyword	Unknown	0	1	0	0	7
9	ELSE	keyword	Unknown	0	1	0	0	8
10	END	keyword	Unknown	0	1	0	0	9

11	FOR	keyword	Unknown	0	1	0	0	10
12	FUNCTION	keyword	Unknown	0	1	0	0	11
13	IF	keyword	Unknown	0	1	0	0	12
14	MOD	keyword	Unknown	0	1	0	0	13
15	NOT	keyword	Unknown	0	1	0	0	14
16	OF	keyword	Unknown	0	1	0	0	15
17	OR	keyword	Unknown	0	1	0	0	16
18	PROCEDURE	keyword	Unknown	0	1	0	0	17
19	PROGRAM	keyword	Unknown	0	1	0	0	18
20	RECORD	keyword	Unknown	0	1	0	0	19
21	REPEAT	keyword	Unknown	0	1	0	0	20
22	INTEGER	type	Integer	0	1	0	0	21
23	REAL	type	Real	0	1	0	0	22
24	BOOLEAN	type	Boolean	0	1	0	0	23
25	CHAR	type	Char	0	1	0	0	24
26	STRING	type	String	0	1	0	0	25
27	THEN	keyword	Unknown	0	1	0	0	26
28	TO	keyword	Unknown	0	1	0	0	27
29	TYPE	keyword	Unknown	0	1	0	0	28
30	UNTIL	keyword	Unknown	0	1	0	0	29
31	VAR	keyword	Unknown	0	1	0	0	30
32	WHILE	keyword	Unknown	0	1	0	0	31
33	true	constant	Boolean	0	1	0	1	32
34	false	constant	Boolean	0	1	0	0	33
35	writeln	procedure	Void	0	1	0	0	34
36	readln	procedure	Void	0	1	0	0	35
37	TestEmpty	program	Void	1	1	0	0	36

BTAB (Block Table)

idx	last	lpar	psze	vsze

0	37	0	0	0
1	0	0	0	0

ATAB (Array Table)

(kosong)

=== Semantic Analysis ===

Errors : 0

Warnings: 0

Status : OK

=== Decorated AST ===

ProgramNode(name: 'TestEmpty') → tab_index:37, type:void,
lev:0

└─ CompoundStmt → type:void

=== Semantic Analysis ===

Errors : 0

Warnings: 0

Status : OK

- Analisis: Parse tree berhasil terbentuk dengan <statement> kosong tanpa error

3.2.4. Kasus Uji 4

- Kasus: while tanpa begin...end harus menghasilkan syntax error.
- Tujuan: Menguji bahwa parser mewajibkan compound-statement pada body while.
- Input:

```
program TestWhileInvalid;  
var  
  i: integer;  
begin  
  i := 5;  
  while i > 0 do  
    i := i - 1;  
end.
```

- Output:

```
Syntax error at line 7, col 6: unexpected token 'ident(i)', expected  
beginsy
```

- Analisis: Parser berhasil mendeteksi bahwa body while bukan compound-statement dan memberikan pesan error yang informatif.

3.2.5. Kasus Uji 5

- Kasus: while dengan begin...end harus valid dan <compound-statement> langsung menjadi child dari <while-statement>.

- Tujuan: Menguji bahwa while dengan compound-statement ter-parse dan ter-annotate dengan benar.
- Input:

```

program TestWhileValid;
var
  i: integer;
begin
  i := 5;
  while i > 0 do
  begin
    i := i - 1;
  end;
end.

```

- Output:

=== Symbol Table ===

TAB (Identifier Table)

idx	name	obj	type	ref	nrm	lev	adr	link
1	AND	keyword	Unknown	0	1	0	0	0
2	ARRAY	keyword	Unknown	0	1	0	0	1
3	BEGIN	keyword	Unknown	0	1	0	0	2
4	CASE	keyword	Unknown	0	1	0	0	3
5	CONST	keyword	Unknown	0	1	0	0	4
6	DIV	keyword	Unknown	0	1	0	0	5
7	DOWNTO	keyword	Unknown	0	1	0	0	6
8	DO	keyword	Unknown	0	1	0	0	7
9	ELSE	keyword	Unknown	0	1	0	0	8
10	END	keyword	Unknown	0	1	0	0	9
11	FOR	keyword	Unknown	0	1	0	0	10
12	FUNCTION	keyword	Unknown	0	1	0	0	11
13	IF	keyword	Unknown	0	1	0	0	12
14	MOD	keyword	Unknown	0	1	0	0	13
15	NOT	keyword	Unknown	0	1	0	0	14
16	OF	keyword	Unknown	0	1	0	0	15
17	OR	keyword	Unknown	0	1	0	0	16
18	PROCEDURE	keyword	Unknown	0	1	0	0	17
19	PROGRAM	keyword	Unknown	0	1	0	0	18
20	RECORD	keyword	Unknown	0	1	0	0	19
21	REPEAT	keyword	Unknown	0	1	0	0	20
22	INTEGER	type	Integer	0	1	0	0	21

23	REAL	type	Real	0	1	0	0	22
24	BOOLEAN	type	Boolean	0	1	0	0	23
25	CHAR	type	Char	0	1	0	0	24
26	STRING	type	String	0	1	0	0	25
27	THEN	keyword	Unknown	0	1	0	0	26
28	TO	keyword	Unknown	0	1	0	0	27
29	TYPE	keyword	Unknown	0	1	0	0	28
30	UNTIL	keyword	Unknown	0	1	0	0	29
31	VAR	keyword	Unknown	0	1	0	0	30
32	WHILE	keyword	Unknown	0	1	0	0	31
33	true	constant	Boolean	0	1	0	1	32
34	false	constant	Boolean	0	1	0	0	33
35	writeln	procedure	Void	0	1	0	0	34
36	readln	procedure	Void	0	1	0	0	35
37	TestWhileValid	program	Void	1	1	0	0	36
38	i	variable	Integer	0	1	1	0	0

BTAB (Block Table)

idx	last	lpar	psze	vsze
-----	------	------	------	------

0	37	0	0	0
1	38	0	0	1

ATAB (Array Table)

(kosong)

=== Semantic Analysis ===

Errors : 0

Warnings: 0

Status : OK

=== Decorated AST ===

ProgramNode(name: 'TestWhileValid') → tab_index:37,

type:void, lev:0

```

├── Declarations
│   └── VarDecl('i') → tab_index:38, type:integer, lev:1
├── CompoundStmt → type:void
│   ├── Assign(i := 5) → type:void
│   │   ├── Var(i) → tab_index:38, type:integer, lev:1
│   │   └── Num(5) → type:integer
│   ├── While → type:void
│   │   ├── BinOp(gtr) → type:boolean
│   │   │   ├── Var(i) → tab_index:38, type:integer, lev:1
│   │   │   └── Num(0) → type:integer
│   │   └── CompoundStmt → type:void
│   │       └── Assign(i := i - 1) → type:void

```

```

└─ Var(i)      → tab_index:38, type:integer, lev:1
  └─ BinOp(minus) → type:integer
    └─ Var(i)    → tab_index:38, type:integer, lev:1
      └─ Num(1)   → type:integer

```

=== Semantic Analysis ===

Errors : 0

Warnings: 0

Status : OK

- Analisis: <compound-statement> langsung menjadi child while, body while terannotasi dengan benar.

3.2.6. Kasus Uji 6

- Kasus: Program dengan nested scope, procedure dengan parameter, dan function dengan return value.
- Tujuan: Menguji bahwa tab dan btab terisi benar untuk semua identifier pada level leksikal yang tepat.
- Input:

```

program TC6;
var
  x, y: integer;
  flag: boolean;
procedure Swap(a, b: integer);
var
  tmp: integer;
begin
  tmp := a; a := b; b := tmp;
end;
function Max(p, q: integer): integer;
begin
  if p > q then Max := p else Max := q;
end;
begin
  x := 10; y := 20;
  Swap(x, y);
  x := Max(x, y);
end.

```

- Output:

=== Symbol Table ===

TAB (Identifier Table)

idx	name	obj	type	ref	nrm	lev	adr	link

1	AND	keyword	Unknown	0	1	0	0	0
2	ARRAY	keyword	Unknown	0	1	0	0	1
3	BEGIN	keyword	Unknown	0	1	0	0	2
4	CASE	keyword	Unknown	0	1	0	0	3
5	CONST	keyword	Unknown	0	1	0	0	4
6	DIV	keyword	Unknown	0	1	0	0	5
7	DOWNT0	keyword	Unknown	0	1	0	0	6
8	DO	keyword	Unknown	0	1	0	0	7
9	ELSE	keyword	Unknown	0	1	0	0	8
10	END	keyword	Unknown	0	1	0	0	9
11	FOR	keyword	Unknown	0	1	0	0	10
12	FUNCTION	keyword	Unknown	0	1	0	0	11
13	IF	keyword	Unknown	0	1	0	0	12
14	MOD	keyword	Unknown	0	1	0	0	13
15	NOT	keyword	Unknown	0	1	0	0	14
16	OF	keyword	Unknown	0	1	0	0	15
17	OR	keyword	Unknown	0	1	0	0	16
18	PROCEDURE	keyword	Unknown	0	1	0	0	17
19	PROGRAM	keyword	Unknown	0	1	0	0	18
20	RECORD	keyword	Unknown	0	1	0	0	19
21	REPEAT	keyword	Unknown	0	1	0	0	20
22	INTEGER	type	Integer	0	1	0	0	21
23	REAL	type	Real	0	1	0	0	22
24	BOOLEAN	type	Boolean	0	1	0	0	23
25	CHAR	type	Char	0	1	0	0	24
26	STRING	type	String	0	1	0	0	25
27	THEN	keyword	Unknown	0	1	0	0	26
28	TO	keyword	Unknown	0	1	0	0	27
29	TYPE	keyword	Unknown	0	1	0	0	28
30	UNTIL	keyword	Unknown	0	1	0	0	29
31	VAR	keyword	Unknown	0	1	0	0	30
32	WHILE	keyword	Unknown	0	1	0	0	31
33	true	constant	Boolean	0	1	0	1	32
34	false	constant	Boolean	0	1	0	0	33
35	writeln	procedure	Void	0	1	0	0	34
36	readln	procedure	Void	0	1	0	0	35
37	TC6	program	Void	1	1	0	0	36
38	x	variable	Integer	0	1	1	0	0
39	y	variable	Integer	0	1	1	1	38

40	flag	variable	Boolean	0	1	1	2	39
41	Swap	procedure	Void	2	1	1	0	40
42	a	variable	Integer	0	1	2	0	0
43	b	variable	Integer	0	1	2	1	42
44	tmp	variable	Integer	0	1	2	0	43
45	Max	function	Integer	3	1	1	0	41
46	p	variable	Integer	0	1	2	0	0
47	q	variable	Integer	0	1	2	1	46

BTAB (Block Table)

idx	last	lpar	psze	vsze

0	37	0	0	0
1	45	0	0	3
2	44	43	2	1
3	47	47	2	0

ATAB (Array Table)

(kosong)

=== Semantic Analysis ===

Errors : 0

Warnings: 0

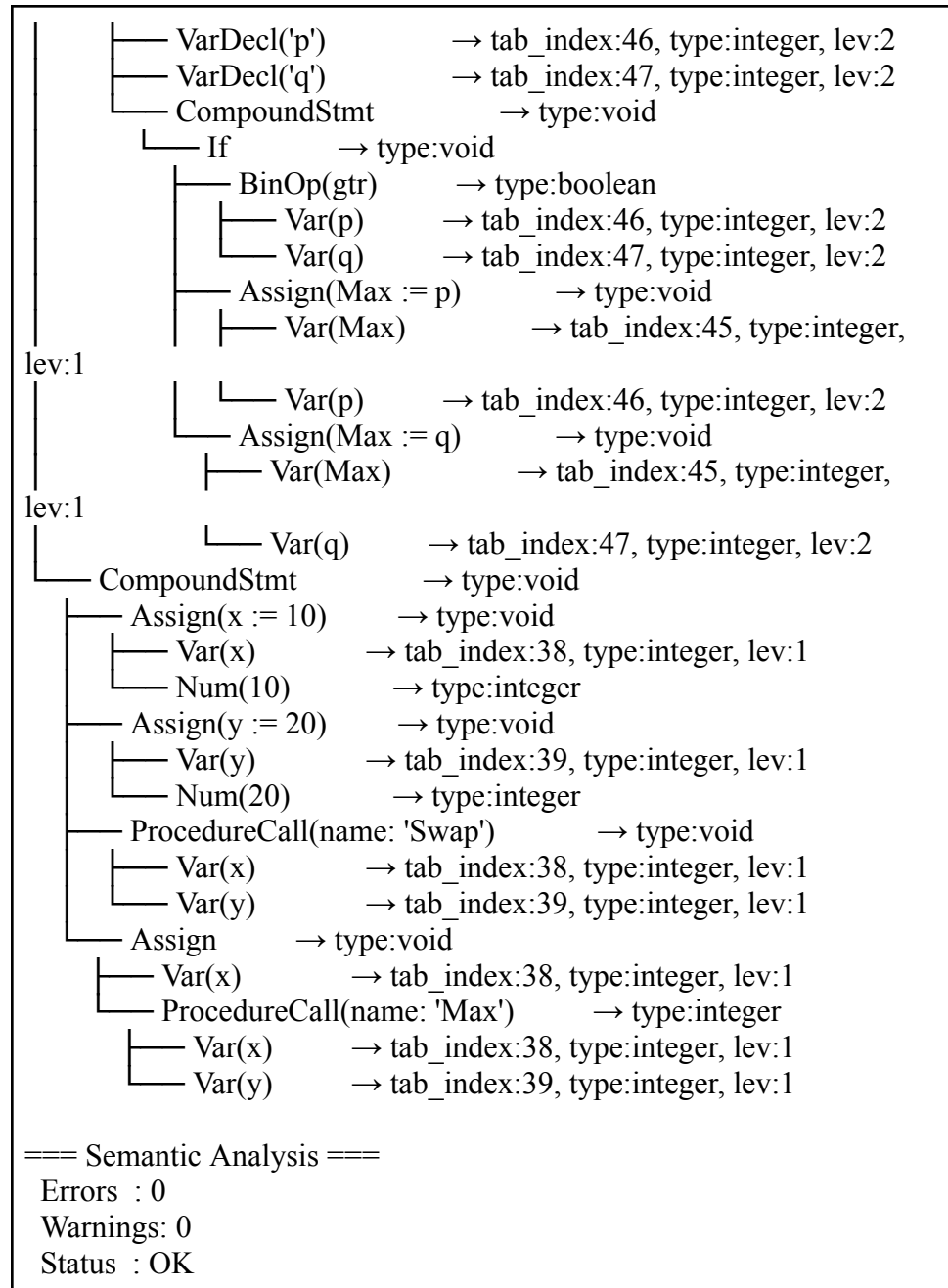
Status : OK

=== Decorated AST ===

```

ProgramNode(name: 'TC6')      → tab_index:37, type:void, lev:0
├── Declarations
│   ├── VarDecl('x')          → tab_index:38, type:integer, lev:1
│   ├── VarDecl('y')          → tab_index:39, type:integer, lev:1
│   ├── VarDecl('flag')       → tab_index:40, type:boolean, lev:1
│   ├── ProcDecl(Swap)        → tab_index:41, type:void, lev:1
│   │   ├── VarDecl('a')      → tab_index:42, type:integer, lev:2
│   │   ├── VarDecl('b')      → tab_index:43, type:integer, lev:2
│   │   └── CompoundStmt      → type:void
│   │       ├── VarDecl('tmp') → tab_index:44, type:integer, lev:2
│   │       ├── Assign(tmp := a) → type:void
│   │       │   ├── Var(tmp) → tab_index:44, type:integer, lev:2
│   │       │   └── Var(a) → tab_index:42, type:integer, lev:2
│   │       ├── Assign(a := b) → type:void
│   │       │   ├── Var(a) → tab_index:42, type:integer, lev:2
│   │       │   └── Var(b) → tab_index:43, type:integer, lev:2
│   │       ├── Assign(b := tmp) → type:void
│   │       │   ├── Var(b) → tab_index:43, type:integer, lev:2
│   │       │   └── Var(tmp) → tab_index:44, type:integer, lev:2
│   └── FuncDecl(Max)         → tab_index:45, type:integer, lev:1

```



- Analisis: Tab terisi dengan semua variabel, parameter, dan subprogram pada level leksikal yang tepat. Btab memiliki entry untuk global block, Swap, dan Max dengan lpar dan psze yang benar. Function Max return type:integer dengan benar.

3.2.7. Kasus Uji 7

- Kasus: Dua anonymous array dengan range berbeda harus menghasilkan error incompatible.
- Tujuan: Menguji bahwa semantic analyzer mendeteksi incompatibility antara dua anonymous array yang berbeda entry di atas.
- Input:

```
program TC2;
var
  nums: array[1..10] of integer;
  grid: array[1..5] of array[1..5] of real;
  i, j: integer;
procedure FillRow(row: array[1..5] of integer; val: integer);
var
  k: integer;
begin
  k := 1;
end;
begin
  nums[1] := 100;
  grid[1][1] := 3.14;
  i := nums[3];
  j := 0;
  FillRow(nums, j);
end.
```

- Output:

=== Symbol Table ===

TAB (Identifier Table)

idx	name	obj	type	ref	nrm	lev	adr	link
1	AND	keyword	Unknown	0	1	0	0	0
2	ARRAY	keyword	Unknown	0	1	0	0	1
3	BEGIN	keyword	Unknown	0	1	0	0	2
4	CASE	keyword	Unknown	0	1	0	0	3
5	CONST	keyword	Unknown	0	1	0	0	4
6	DIV	keyword	Unknown	0	1	0	0	5
7	DOWNTON	keyword	Unknown	0	1	0	0	6

8	DO	keyword	Unknown	0	1	0	0	7
9	ELSE	keyword	Unknown	0	1	0	0	8
10	END	keyword	Unknown	0	1	0	0	9
11	FOR	keyword	Unknown	0	1	0	0	10
12	FUNCTION	keyword	Unknown	0	1	0	0	11
13	IF	keyword	Unknown	0	1	0	0	12
14	MOD	keyword	Unknown	0	1	0	0	13
15	NOT	keyword	Unknown	0	1	0	0	14
16	OF	keyword	Unknown	0	1	0	0	15
17	OR	keyword	Unknown	0	1	0	0	16
18	PROCEDURE	keyword	Unknown	0	1	0	0	17
19	PROGRAM	keyword	Unknown	0	1	0	0	18
20	RECORD	keyword	Unknown	0	1	0	0	19
21	REPEAT	keyword	Unknown	0	1	0	0	20
22	INTEGER	type	Integer	0	1	0	0	21
23	REAL	type	Real	0	1	0	0	22
24	BOOLEAN	type	Boolean	0	1	0	0	23
25	CHAR	type	Char	0	1	0	0	24
26	STRING	type	String	0	1	0	0	25
27	THEN	keyword	Unknown	0	1	0	0	26
28	TO	keyword	Unknown	0	1	0	0	27
29	TYPE	keyword	Unknown	0	1	0	0	28
30	UNTIL	keyword	Unknown	0	1	0	0	29
31	VAR	keyword	Unknown	0	1	0	0	30
32	WHILE	keyword	Unknown	0	1	0	0	31
33	true	constant	Boolean	0	1	0	1	32
34	false	constant	Boolean	0	1	0	0	33
35	writeln	procedure	Void	0	1	0	0	34
36	readln	procedure	Void	0	1	0	0	35
37	TC7	program	Void	1	1	0	0	36
38	nums	variable	Array	0	1	1	0	0
39	grid	variable	Array	2	1	1	10	38
40	i	variable	Integer	0	1	1	60	39
41	j	variable	Integer	0	1	1	61	40
42	FillRow	procedure	Void	2	1	1	0	41
43	row	variable	Array	3	1	2	0	0
44	val	variable	Integer	0	1	2	5	43
45	k	variable	Integer	0	1	2	0	44

BTAB (Block Table)

idx	last	lpar	psze	vsze
-----	------	------	------	------

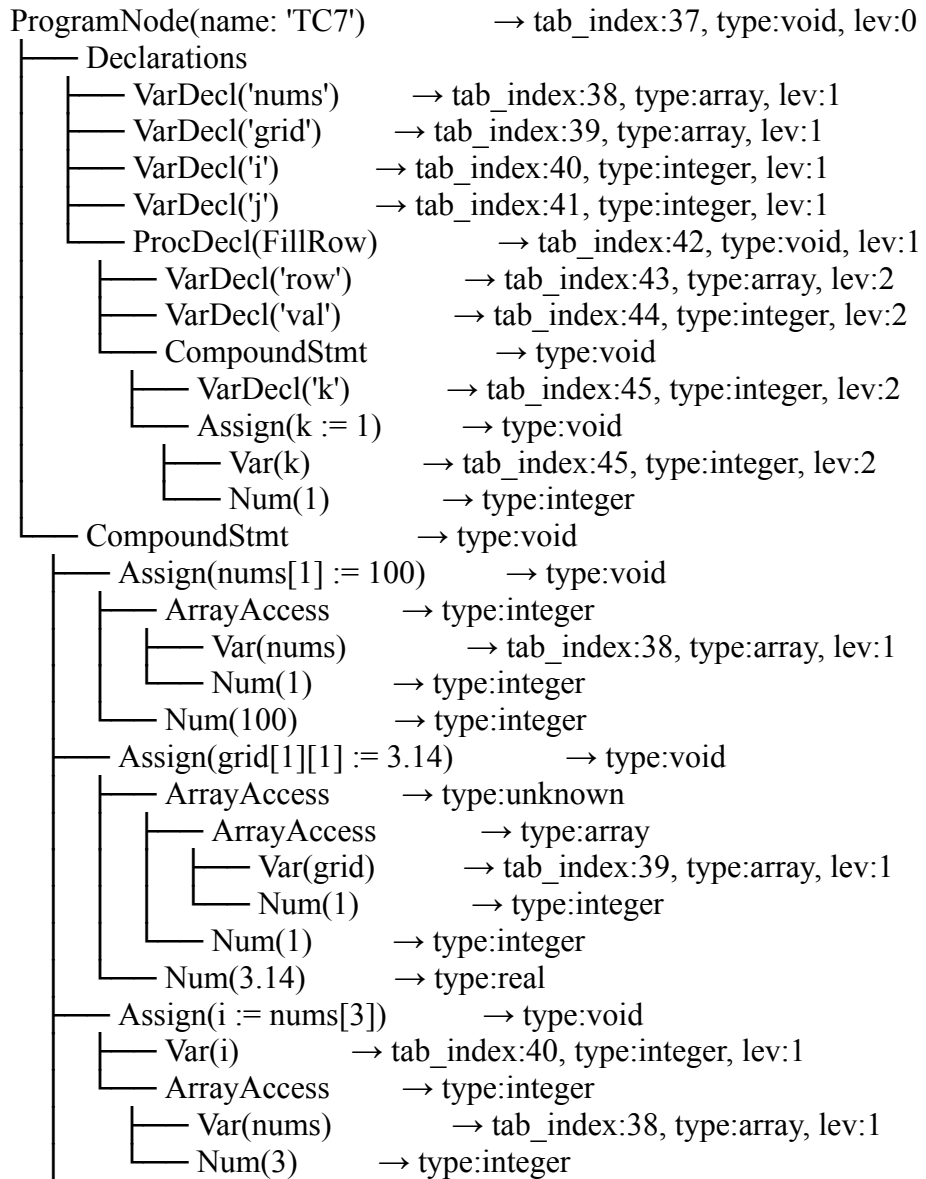
0	37	0	0	0
1	42	0	0	62
2	45	44	6	1

ATAB (Array Table)							
idx	xtyp	etyp	eref	low	high	elsz	size
0	Integer	Integer	0	1	10	1	10
1	Integer	Real	0	1	5	2	10
2	Integer	Array	1	1	5	10	50
3	Integer	Integer	0	1	5	1	5

=== Semantic Analysis ===

Errors : 1
Warnings: 0
Status : FAILED

=== Decorated AST ===



```

├── Assign(j := 0)    → type:void
│   ├── Var(j)       → tab_index:41, type:integer, lev:1
│   └── Num(0)       → type:integer
└── ProcedureCall(name: 'FillRow') → type:void
    ├── Var(nums)    → tab_index:38, type:array, lev:1
    └── Var(j)       → tab_index:41, type:integer, lev:1

```

=== Semantic Analysis ===

Errors : 1

Warnings: 0

Status : FAILED

- Analisis: “[ERROR] Assignment incompatible: cannot assign 'Array' to 'Array’” Muncul di terminal yang artinya Semantic analyzer berhasil mendeteksi bahwa nums (array[1..10]) dan parameter row (array[1..5]) adalah dua anonymous array dengan atab entry berbeda sehingga incompatible.

3.2.8. Kasus Uji 8

- Kasus: Kasus normal untuk program arion symbol table identifier untuk fungsi visit yang sesuai
- Tujuan: uji visitAssign, visitBinOp, visitVar, visitIf
- Input:

```

program TC8;
var
  a, b, c: Integer;
  x: Real;
  flag: Boolean;
begin
  a := 10;
  b := 3;
  c := a + b;
  c := a - b;
  c := a * b;
  c := a div b;
  c := a mod b;
  x := a / b;
  flag := a > b;
  flag := a < b;

```

```

flag := a >= b;
flag := a <= b;
flag := a <> b;
if flag then
  c := a
else
  c := b;
end.

```

- Output:

=== Symbol Table ===

TAB (Identifier Table)

idx	name	obj	type	ref	nrm	lev	adr	link

1	AND	keyword	Unknown	0	1	0	0	0
2	ARRAY	keyword	Unknown	0	1	0	0	1
3	BEGIN	keyword	Unknown	0	1	0	0	2
4	CASE	keyword	Unknown	0	1	0	0	3
5	CONST	keyword	Unknown	0	1	0	0	4
6	DIV	keyword	Unknown	0	1	0	0	5
7	DOWNT0	keyword	Unknown	0	1	0	0	6
8	DO	keyword	Unknown	0	1	0	0	7
9	ELSE	keyword	Unknown	0	1	0	0	8
10	END	keyword	Unknown	0	1	0	0	9
11	FOR	keyword	Unknown	0	1	0	0	10
12	FUNCTION	keyword	Unknown	0	1	0	0	11
13	IF	keyword	Unknown	0	1	0	0	12
14	MOD	keyword	Unknown	0	1	0	0	13
15	NOT	keyword	Unknown	0	1	0	0	14
16	OF	keyword	Unknown	0	1	0	0	15
17	OR	keyword	Unknown	0	1	0	0	16
18	PROCEDURE	keyword	Unknown	0	1	0	0	17
19	PROGRAM	keyword	Unknown	0	1	0	0	18
20	RECORD	keyword	Unknown	0	1	0	0	19
21	REPEAT	keyword	Unknown	0	1	0	0	20
22	INTEGER	type	Integer	0	1	0	0	21
23	REAL	type	Real	0	1	0	0	22
24	BOOLEAN	type	Boolean	0	1	0	0	23
25	CHAR	type	Char	0	1	0	0	24
26	STRING	type	String	0	1	0	0	25
27	THEN	keyword	Unknown	0	1	0	0	26
28	TO	keyword	Unknown	0	1	0	0	27
29	TYPE	keyword	Unknown	0	1	0	0	28

30	UNTIL	keyword	Unknown	0	1	0	0	29
31	VAR	keyword	Unknown	0	1	0	0	30
32	WHILE	keyword	Unknown	0	1	0	0	31
33	true	constant	Boolean	0	1	0	1	32
34	false	constant	Boolean	0	1	0	0	33
35	writeln	procedure	Void	0	1	0	0	34
36	readln	procedure	Void	0	1	0	0	35
37	TC18	program	Void	1	1	0	0	36
38	a	variable	Integer	0	1	1	0	0
39	b	variable	Integer	0	1	1	1	38
40	c	variable	Integer	0	1	1	2	39
41	x	variable	Real	0	1	1	3	40
42	flag	variable	Boolean	0	1	1	5	41

BTAB (Block Table)

idx	last	lpar	psze	vsze
-----	------	------	------	------

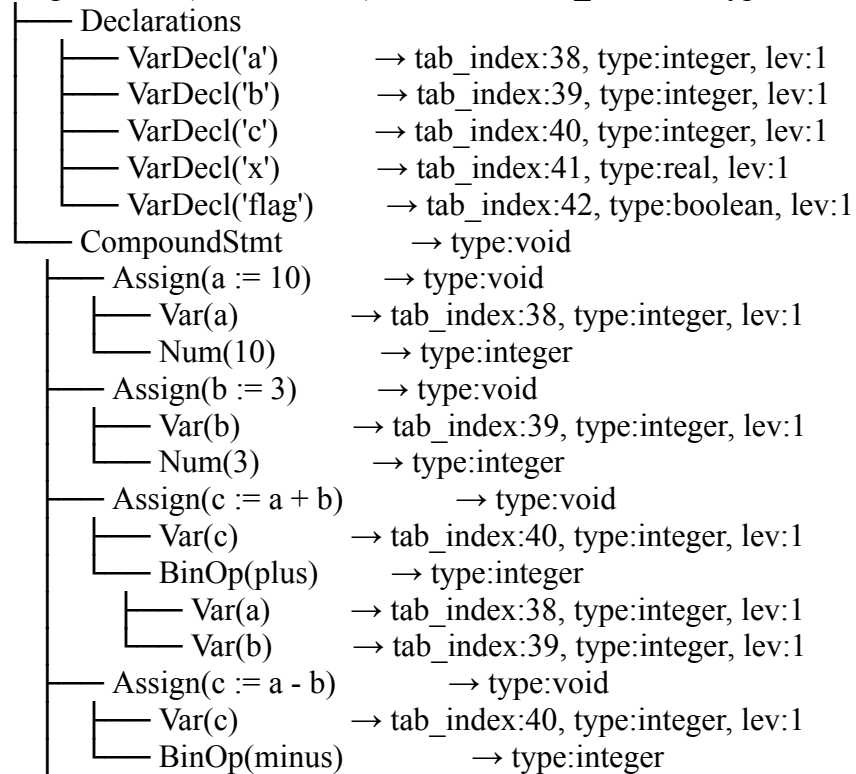
0	37	0	0	0
1	42	0	0	6

ATAB (Array Table)

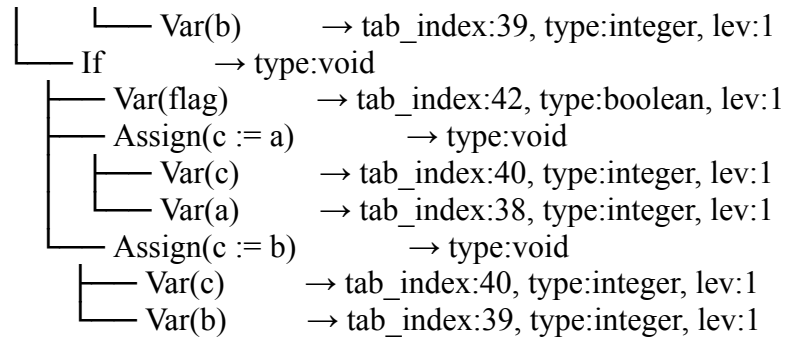
(kosong)

=== Decorated AST ===

ProgramNode(name: 'TC18') → tab_index:37, type:void, lev:0



└─	Var(a)	→ tab_index:38, type:integer, lev:1
└─	Var(b)	→ tab_index:39, type:integer, lev:1
└─	Assign(c := atimesb)	→ type:void
└─	Var(c)	→ tab_index:40, type:integer, lev:1
└─	BinOp(times)	→ type:integer
└─	Var(a)	→ tab_index:38, type:integer, lev:1
└─	Var(b)	→ tab_index:39, type:integer, lev:1
└─	Assign(c := adivb)	→ type:void
└─	Var(c)	→ tab_index:40, type:integer, lev:1
└─	BinOp(idiv)	→ type:integer
└─	Var(a)	→ tab_index:38, type:integer, lev:1
└─	Var(b)	→ tab_index:39, type:integer, lev:1
└─	Assign(c := aimodb)	→ type:void
└─	Var(c)	→ tab_index:40, type:integer, lev:1
└─	BinOp(imod)	→ type:integer
└─	Var(a)	→ tab_index:38, type:integer, lev:1
└─	Var(b)	→ tab_index:39, type:integer, lev:1
└─	Assign(x := ardivb)	→ type:void
└─	Var(x)	→ tab_index:41, type:real, lev:1
└─	BinOp(rdiv)	→ type:real
└─	Var(a)	→ tab_index:38, type:integer, lev:1
└─	Var(b)	→ tab_index:39, type:integer, lev:1
└─	Assign(flag := agrb)	→ type:void
└─	Var(flag)	→ tab_index:42, type:boolean, lev:1
└─	BinOp(gtr)	→ type:boolean
└─	Var(a)	→ tab_index:38, type:integer, lev:1
└─	Var(b)	→ tab_index:39, type:integer, lev:1
└─	Assign(flag := alssb)	→ type:void
└─	Var(flag)	→ tab_index:42, type:boolean, lev:1
└─	BinOp(lss)	→ type:boolean
└─	Var(a)	→ tab_index:38, type:integer, lev:1
└─	Var(b)	→ tab_index:39, type:integer, lev:1
└─	Assign(flag := ageqb)	→ type:void
└─	Var(flag)	→ tab_index:42, type:boolean, lev:1
└─	BinOp(geq)	→ type:boolean
└─	Var(a)	→ tab_index:38, type:integer, lev:1
└─	Var(b)	→ tab_index:39, type:integer, lev:1
└─	Assign(flag := aleqb)	→ type:void
└─	Var(flag)	→ tab_index:42, type:boolean, lev:1
└─	BinOp(leq)	→ type:boolean
└─	Var(a)	→ tab_index:38, type:integer, lev:1
└─	Var(b)	→ tab_index:39, type:integer, lev:1
└─	Assign(flag := aneqb)	→ type:void
└─	Var(flag)	→ tab_index:42, type:boolean, lev:1
└─	BinOp(neq)	→ type:boolean
└─	Var(a)	→ tab_index:38, type:integer, lev:1



=== Semantic Analysis ===

Errors : 0

Warnings: 0

Status : OK

- Analisis

Program berhasil menjalankan test case. Symbol Table berhasil memuat identifier yang sesuai ketika melakukan visitAssign, visitBinOp, visitVar, visitIf.

3.2.9. Kasus Uji 9

- Kasus: Kasus normal untuk program arion symbol table identifier untuk fungsi visit yang sesuai
- Tujuan: uji visitWhile, visitFor, visitRepeat, visitProcCall
- Input:

```

program TC9;
var
  i, total: Integer;
begin
  total := 0;
  i := 1;
  while i <= 5 do
  begin
    total := total + i;
    i := i + 1;
  end;

  total := 0;
  for i := 1 to 5 do

```

```

begin
  total := total + i;
end;

i := 10;
repeat
  i := i - 1;
until i < 1;

writeln(total);
readln(i);
end.

```

- Output:

=== Symbol Table ===

TAB (Identifier Table)

idx	name	obj	type	ref	nrm	lev	adr	link
1	AND	keyword	Unknown	0	1	0	0	0
2	ARRAY	keyword	Unknown	0	1	0	0	1
3	BEGIN	keyword	Unknown	0	1	0	0	2
4	CASE	keyword	Unknown	0	1	0	0	3
5	CONST	keyword	Unknown	0	1	0	0	4
6	DIV	keyword	Unknown	0	1	0	0	5
7	DOWNTO	keyword	Unknown	0	1	0	0	6
8	DO	keyword	Unknown	0	1	0	0	7
9	ELSE	keyword	Unknown	0	1	0	0	8
10	END	keyword	Unknown	0	1	0	0	9
11	FOR	keyword	Unknown	0	1	0	0	10
12	FUNCTION	keyword	Unknown	0	1	0	0	11
13	IF	keyword	Unknown	0	1	0	0	12
14	MOD	keyword	Unknown	0	1	0	0	13
15	NOT	keyword	Unknown	0	1	0	0	14
16	OF	keyword	Unknown	0	1	0	0	15
17	OR	keyword	Unknown	0	1	0	0	16
18	PROCEDURE	keyword	Unknown	0	1	0	0	17
19	PROGRAM	keyword	Unknown	0	1	0	0	18
20	RECORD	keyword	Unknown	0	1	0	0	19
21	REPEAT	keyword	Unknown	0	1	0	0	20
22	INTEGER	type	Integer	0	1	0	0	21
23	REAL	type	Real	0	1	0	0	22
24	BOOLEAN	type	Boolean	0	1	0	0	23
25	CHAR	type	Char	0	1	0	0	24

26	STRING	type	String	0	1	0	0	25
27	THEN	keyword	Unknown	0	1	0	0	26
28	TO	keyword	Unknown	0	1	0	0	27
29	TYPE	keyword	Unknown	0	1	0	0	28
30	UNTIL	keyword	Unknown	0	1	0	0	29
31	VAR	keyword	Unknown	0	1	0	0	30
32	WHILE	keyword	Unknown	0	1	0	0	31
33	true	constant	Boolean	0	1	0	1	32
34	false	constant	Boolean	0	1	0	0	33
35	writeln	procedure	Void	0	1	0	0	34
36	readln	procedure	Void	0	1	0	0	35
37	TC19	program	Void	1	1	0	0	36
38	i	variable	Integer	0	1	1	0	0
39	total	variable	Integer	0	1	1	1	38

BTAB (Block Table)

idx	last	lpar	psze	vsze
0	37	0	0	0
1	39	0	0	2

ATAB (Array Table)

(kosong)

=== Semantic Analysis ===

Errors : 0
Warnings: 0
Status : OK

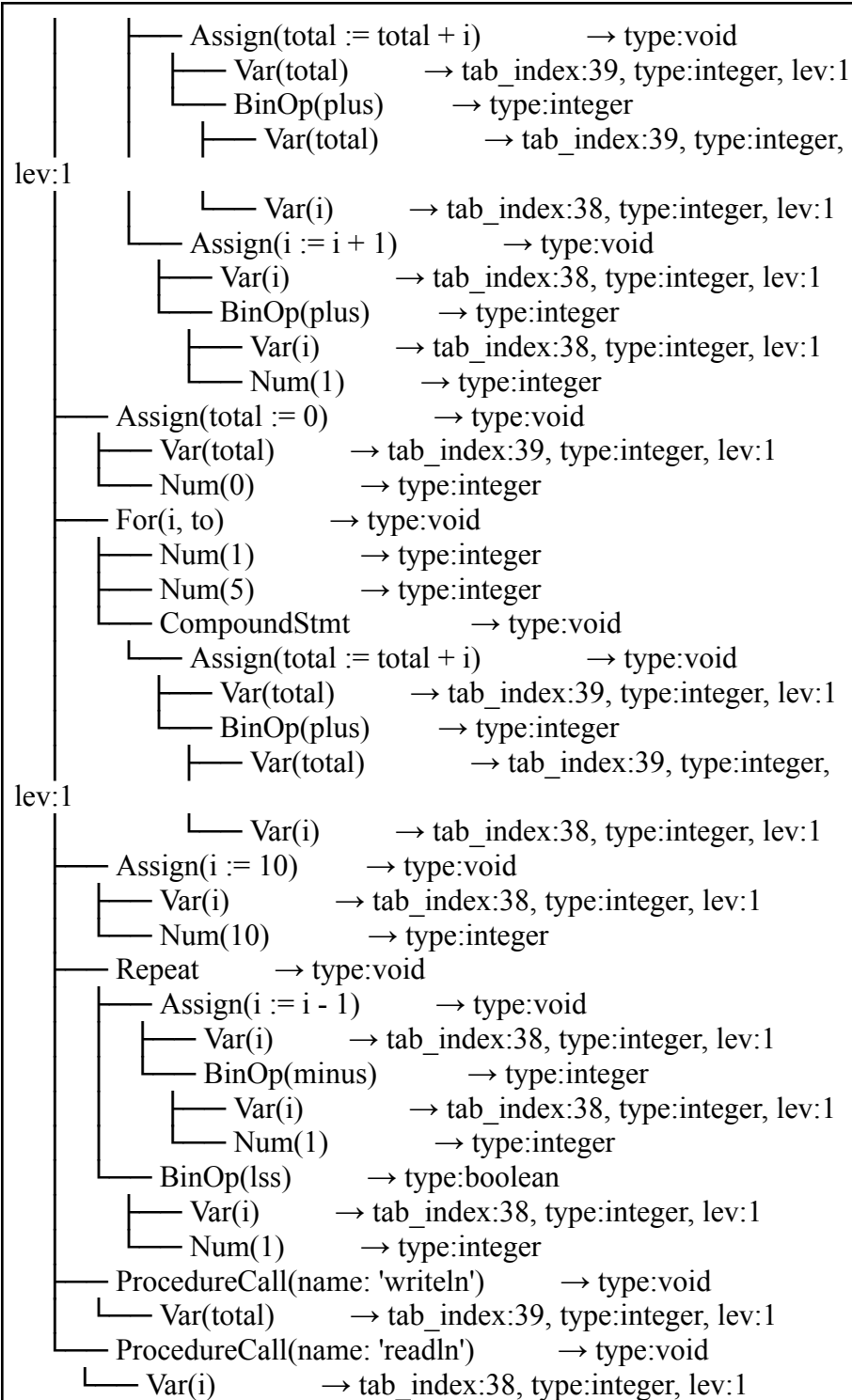
=== Decorated AST ===

ProgramNode(name: 'TC19') → tab_index:37, type:void, lev:0

```

├── Declarations
│   ├── VarDecl('i') → tab_index:38, type:integer, lev:1
│   └── VarDecl('total') → tab_index:39, type:integer, lev:1
├── CompoundStmt → type:void
│   ├── Assign(total := 0) → type:void
│   │   ├── Var(total) → tab_index:39, type:integer, lev:1
│   │   └── Num(0) → type:integer
│   ├── Assign(i := 1) → type:void
│   │   ├── Var(i) → tab_index:38, type:integer, lev:1
│   │   └── Num(1) → type:integer
│   ├── While → type:void
│   │   ├── BinOp(leq) → type:boolean
│   │   │   ├── Var(i) → tab_index:38, type:integer, lev:1
│   │   │   └── Num(5) → type:integer
│   │   └── CompoundStmt → type:void

```



=== Semantic Analysis ===

Errors : 0

Warnings: 0

Status : OK

- Analisis

Program berhasil menjalankan test case. Symbol Table berhasil memuat identifier yang sesuai ketika melakukan visitWhile, visitFor, visitRepeat, visitProcCall.

3.2.10. Kasus Uji 10

Kasus: Kasus normal untuk program arion symbol table identifier untuk fungsi visit yang sesuai

- Tujuan: uji visitCase, visitBinOp, visitVar
- Input:

```
program TC10;
var
score: Integer;
grade: Integer;
passed: Boolean;
begin
score := 85;
grade := score div 10;
passed := score >= 60;
case grade of
  10: writeln('Sempurnanya');
  9: writeln('Aku');
  8: writeln('Untuk');
  7: writeln('Dirimu');
  6: writeln('Aku');
  5: writeln('Banyak');
  4: writeln('Gamau');
end;
end.
```

- Output:

=== Symbol Table ===						
TAB (Identifier Table)						
adr link		idx	name	obj	type	ref nrm lev

-----		1	AND	keyword	Unknown	0 1

0 0 0	2	ARRAY	keyword	Unknown	0	1
0 0 1	3	BEGIN	keyword	Unknown	0	1
0 0 2	4	CASE	keyword	Unknown	0	1
0 0 3	5	CONST	keyword	Unknown	0	1
0 0 4	6	DIV	keyword	Unknown	0	1 0
0 5	7	DOWNT0	keyword	Unknown	0	
1 0 0 6	8	DO	keyword	Unknown	0	1 0
0 7	9	ELSE	keyword	Unknown	0	1
0 0 8	10	END	keyword	Unknown	0	1
0 0 9	11	FOR	keyword	Unknown	0	1
0 0 10	12	FUNCTION	keyword	Unknown	0	
1 0 0 11	13	IF	keyword	Unknown	0	1 0
0 12	14	MOD	keyword	Unknown	0	1
0 0 13	15	NOT	keyword	Unknown	0	1
0 0 14	16	OF	keyword	Unknown	0	1 0
0 15	17	OR	keyword	Unknown	0	1 0
0 16	18	PROCEDURE	keyword	Unknown	0	
1 0 0 17	19	PROGRAM	keyword	Unknown	0	
1 0 0 18	20	RECORD	keyword	Unknown	0	
1 0 0 19	21	REPEAT	keyword	Unknown	0	1
0 0 20	22	INTEGER	type	Integer	0	1 0
0 21	23	REAL	type	Real	0	1 0 0
22	24	BOOLEAN	type	Boolean	0	1

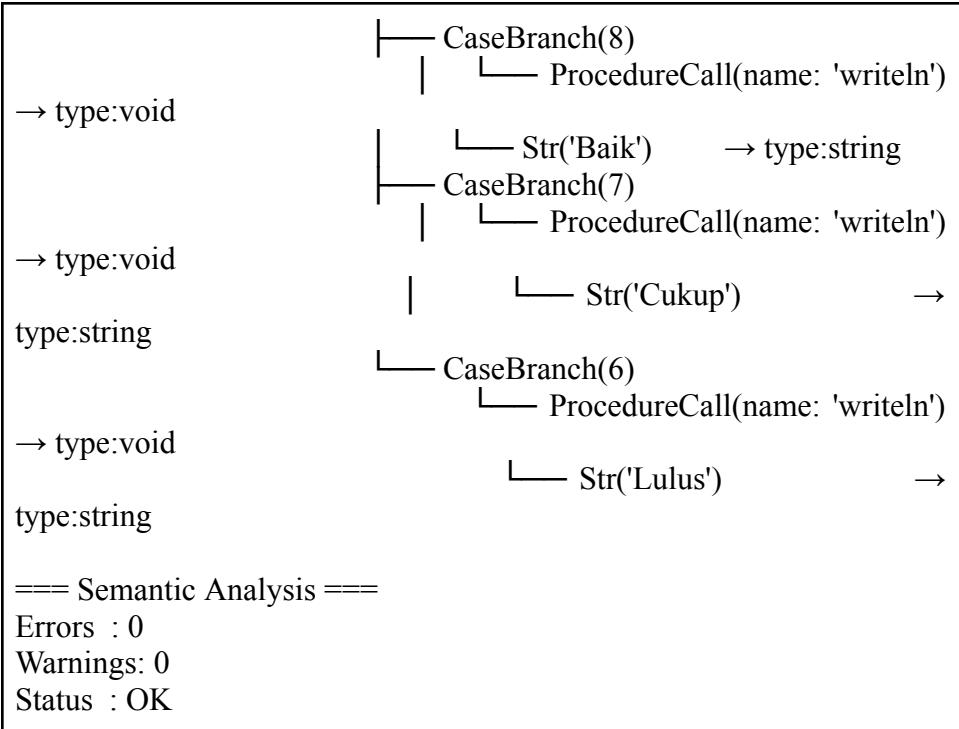
0	0	23	25	CHAR	type	Char	0	1	0
0	0	24	26	STRING	type	String	0	1	0
0	0	25	27	THEN	keyword	Unknown	0	1	
0	0	26	28	TO	keyword	Unknown	0	1	0
0	0	27	29	TYPE	keyword	Unknown	0	1	
0	0	28	30	UNTIL	keyword	Unknown	0	1	
0	0	29	31	VAR	keyword	Unknown	0	1	
0	0	30	32	WHILE	keyword	Unknown	0	1	
0	0	31	33	true	constant	Boolean	0	1	0
1	0	32	34	false	constant	Boolean	0	1	0
0	0	33	35	writeln	procedure	Void	0	1	0
0	0	34	36	readln	procedure	Void	0	1	0
0	0	35	37	TC20	program	Void	1	1	0
0	0	36	38	score	variable	Integer	0	1	1
0	0	37	39	grade	variable	Integer	0	1	1
2	0	38	40	passed	variable	Boolean	0	1	1
BTAB (Block Table)									
idx last lpar psze vsze									

0	37	0	0	0					
1	40	0	0	3					
ATAB (Array Table)									
(kosong)									
=== Semantic Analysis ===									
Errors : 0									
Warnings: 0									

Status : OK

=== Decorated AST ===

```
ProgramNode(name: 'TC20') →
tab_index:37, type:void, lev:0
├── Declarations
│   ├── VarDecl('score') →
│   │   tab_index:38, type:integer, lev:1
│   │   ├── VarDecl('grade') →
│   │   │   tab_index:39, type:integer, lev:1
│   │   │   └── VarDecl('passed') →
│   │   │       tab_index:40, type:boolean, lev:1
│   └── CompoundStmt → type:void
│       ├── Assign(score := 85) → type:void
│       │   ├── Var(score) → tab_index:38,
│       │   │   type:integer, lev:1
│       │   └── Num(85) → type:integer
│       ├── Assign(grade := scoreidiv10) →
│       │   type:void
│       │   ├── Var(grade) → tab_index:39,
│       │   │   type:integer, lev:1
│       │   └── BinOp(idiv) → type:integer
│       │       └── Var(score) →
│       │           tab_index:38, type:integer, lev:1
│       │           ├── Num(10) → type:integer
│       │           └── Assign(passed := scoregeq60) →
│       │               type:void
│       │               ├── Var(passed) → tab_index:40,
│       │               │   type:boolean, lev:1
│       │               └── BinOp(geq) → type:boolean
│       │                   └── Var(score) →
│       │                       tab_index:38, type:integer, lev:1
│       └── Case → type:void
│           ├── Num(60) → type:integer
│           └── Var(grade) → tab_index:39,
│               type:integer, lev:1
│               ├── CaseBranch(10)
│               │   ├── ProcedureCall(name: 'writeln')
│               │   │   → type:void
│               │   └── Str('Sempurna') →
│               │       type:string
│               └── CaseBranch(9)
│                   ├── ProcedureCall(name: 'writeln')
│                   │   → type:void
│                   └── Str('Sangat Baik') →
│                       type:string
```



- Analisis

Program berhasil menjalankan test case. Symbol Table berhasil memuat identifier yang sesuai ketika melakukan visitCase, visitBinOp, visitVar.

3.2.11. Kasus Uji 11

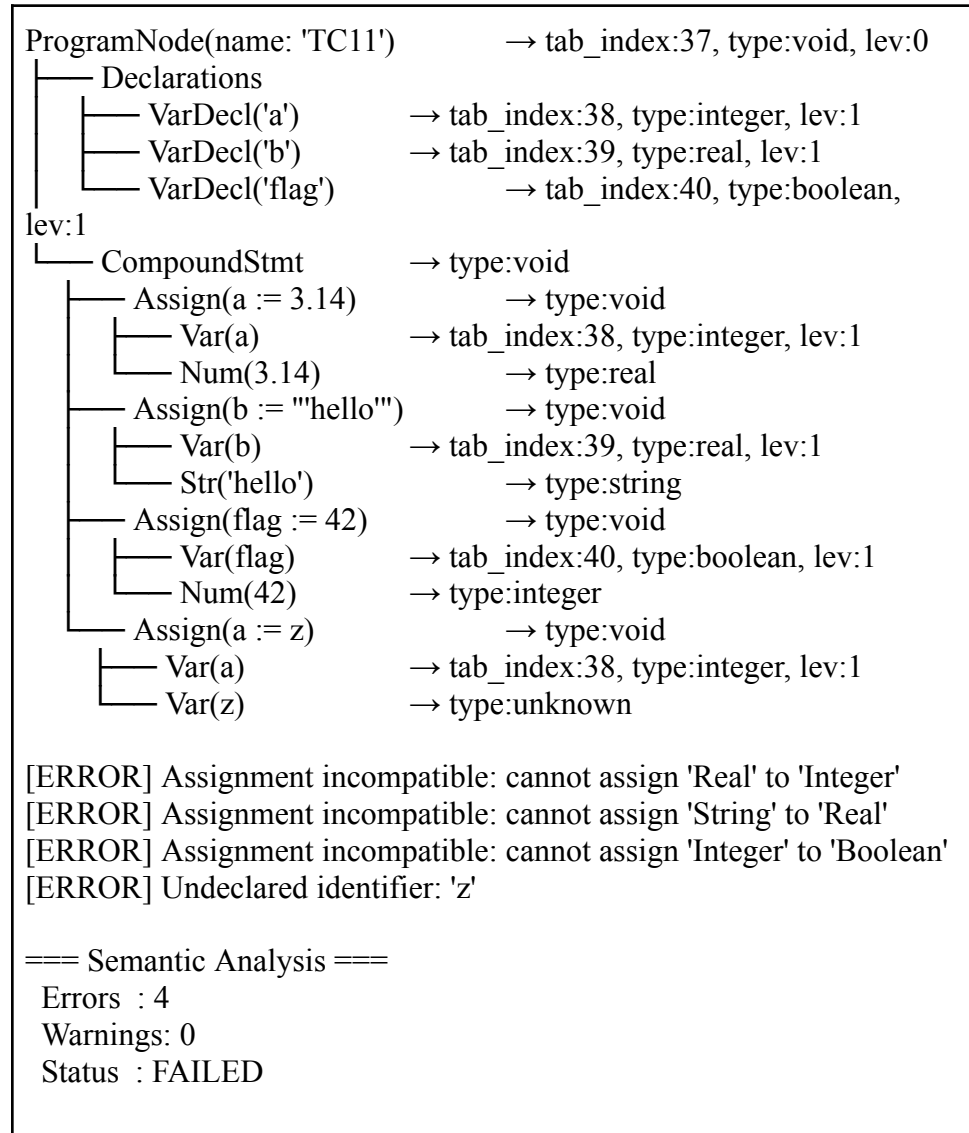
- Kasus: Type Mismatch dan Undeclared Identifier
- Tujuan: Uji typeMismatchError, assignIncompatibleError, undeclaredError
- Input:

```

program TC11;
var
  a: Integer;
  b: Real;
  flag: Boolean;
begin
  a := 3.14;
  b := 'hello';
  flag := 42;
  a := z;
end.

```

- Output:



- Analisis

Memverifikasi bahwa assignIncompatibleError dipanggil untuk narrowing (Real → Integer), tipe tidak kompatibel (String → Real, Integer → Boolean), dan undeclaredError untuk identifier yang tidak dideklarasikan. Program tidak crash meskipun ada 4 error (error recovery bekerja).

3.2.12. Kasus Uji 12

- Kasus: Redeclaration dan Non-Boolean Condition

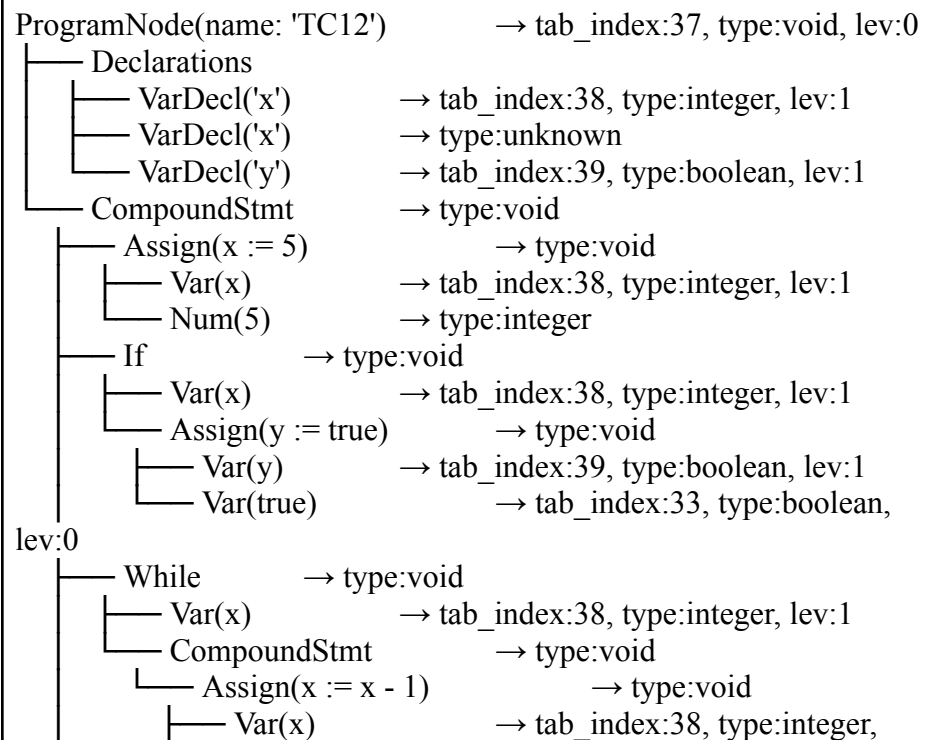
- Tujuan: Uji redeclarationError, nonBooleanConditionError pada if, while, repeat
- Input:

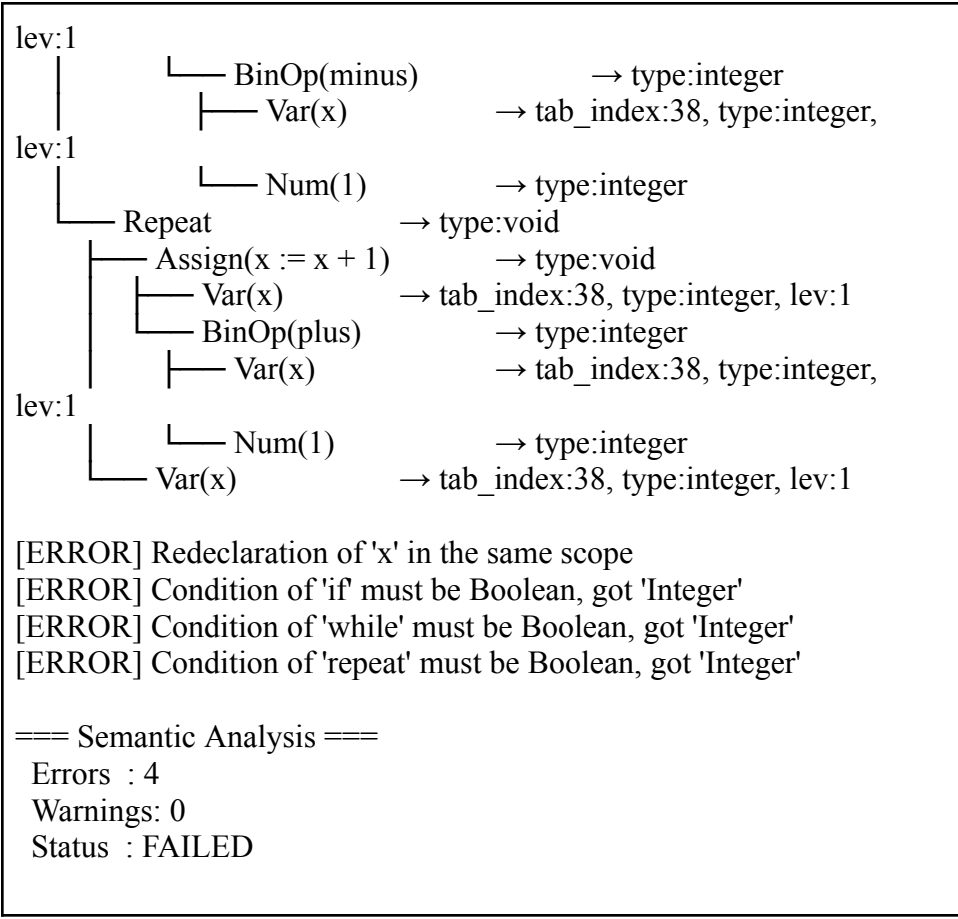
```

program TC12;
var
  x: Integer;
  x: Real;
  y: Boolean;
begin
  x := 5;
  if x then
    y := true;
  while x do
  begin
    x := x - 1;
  end;
  repeat
    x := x + 1;
  until x;
end.

```

- Output:





- Analisis

Memverifikasi `redeclarationError` untuk `x` yang dideklarasikan dua kali di scope yang sama, dan `nonBooleanConditionError` untuk ketiga jenis loop/kondisional yang conditionnya bukan Boolean.

3.2.13. Kasus Uji 13

- Kasus: Invalid Array Index Type dan Subrange Invalid
- Tujuan: Uji `invalidIndexTypeError`, `invalidSubrangeError`, `realSubrangeError`
- Input:

```
program TC13;  
type  
  BadArray = array[1.0..5.0] of Integer;  
  BadRange = array[10..1] of Integer;
```

```

var
  a: array[1..5] of Integer;
  b: BadArray;
begin
  a[1] := 10;
end.

```

- Output:

```

ProgramNode(name: 'TC13')      → tab_index:37, type:void, lev:0
├── Declarations
│   ├── TypeDecl(BadArray)      → tab_index:38,
│   │   type:array, lev:1
│   ├── TypeDecl(BadRange)      → tab_index:39,
│   │   type:array, lev:1
│   ├── VarDecl('a')            → tab_index:40, type:array, lev:1
│   ├── VarDecl('b')            → tab_index:41, type:unknown, lev:1
│   └── CompoundStmnt           → type:void
│       ├── Assign(a[1] := 10)   → type:void
│       │   ├── ArrayAccess      → type:integer
│       │   │   ├── Var(a)       → tab_index:40, type:array, lev:1
│       │   │   ├── Num(1)       → type:integer
│       │   └── Num(10)          → type:integer

```

[ERROR] Invalid array index type: 'Real' is not allowed as index type (Real is forbidden)

[ERROR] Subrange type cannot be Real

[ERROR] Invalid subrange: lower bound 10 > upper bound 1

=== Semantic Analysis ===

Errors : 3

Warnings: 0

Status : FAILED

- Analisis

Memverifikasi tiga error berbeda terkait array: index type Real dilarang, subrange tidak boleh bertipe Real, dan lower bound tidak boleh lebih besar dari upper bound.

3.2.14. Kasus Uji 14

- Kasus: Const Declaration dan Unary Operator

- Tujuan: Uji visitConstDecl, unary not, unary minus, dan penggunaan konstanta dalam ekspresi
- Input:

```

program TC14;
const
  MAX == 100;
  PI == 3.14;
  FLAG == true;
var
  a: Integer;
  b: Real;
  c: Boolean;
begin
  a := MAX;
  b := PI;
  c := not FLAG;
  a := -MAX;
  c := not a;
end.

```

- Output:

```

ProgramNode(name: 'TC14')      → tab_index:37, type:void, lev:0
├── Declarations
│   ├── ConstDecl(MAX = 100)    →
│   │   tab_index:38, type:integer, lev:1
│   │   ├── ConstDecl(PI = 3.14) →
│   │   │   tab_index:39, type:real, lev:1
│   │   │   └── ConstDecl(FLAG = true) →
│   │   │       tab_index:40, type:unknown, lev:1
│   │   └── VarDecl('a')        → tab_index:41,
│   │       type:integer, lev:1
│   │   └── VarDecl('b')        → tab_index:42,
│   │       type:real, lev:1
│   │   └── VarDecl('c')        → tab_index:43,
│   │       type:boolean, lev:1
│   └── CompoundStmt            → type:void
│       ├── Assign(a := MAX)    → type:void
│       │   └── Var(a)          → tab_index:41,
│       │       type:integer, lev:1
│       └── Var(MAX)            →
│           tab_index:38, type:integer, lev:1
│       └── Assign(b := PI)     → type:void

```

type:real, lev:1		└─ Var(b)	→ tab_index:42,
type:real, lev:1		└─ Var(PI)	→ tab_index:39,
type:void		└─ Assign(c := notsyFLAG)	→
type:boolean, lev:1		└─ Var(c)	→ tab_index:43,
type:boolean		└─ UnaryOp(notsy)	→
tab_index:40, type:unknown, lev:1		└─ Var(FLAG)	→
		└─ Assign(a := -MAX)	→ type:void
type:integer, lev:1		└─ Var(a)	→ tab_index:41,
type:integer		└─ UnaryOp(minus)	→
tab_index:38, type:integer, lev:1		└─ Var(MAX)	→
		└─ Assign(c := notsya)	→ type:void
type:boolean, lev:1		└─ Var(c)	→ tab_index:43,
type:boolean		└─ UnaryOp(notsy)	→
type:integer, lev:1		└─ Var(a)	→ tab_index:41,
[ERROR] Operand of 'not' must be Boolean			
=== Semantic Analysis ===			
Errors : 1			
Warnings: 0			
Status : FAILED			

- Analisis

Memverifikasi visitConstDecl mengisi adr dengan benar (100 untuk MAX, 1 untuk FLAG), unary not valid pada Boolean namun error pada Integer, dan unary minus valid pada Integer.

3.2.15. Kasus Uji 15

- Kasus: Wrong Arg Count dan Wrong Object Kind

- Tujuan: Uji wrongArgCountError, wrongObjectKindError, penggunaan nama prosedur sebagai variabel
- Input:

```

program TC15;
var
  x, y: Integer;
procedure Add(a, b: Integer);
begin
  x := a + b;
end;
begin
  x := 5;
  y := 3;
  Add(x, y, x);
  Add();
  x := Add;
  writeln();
end.

```

- Output:

```

ProgramNode(name: 'TC15')      → tab_index:37, type:void, lev:0
├── Declarations
│   ├── VarDecl('x')          → tab_index:38,
│   │   type:integer, lev:1
│   │   ├── VarDecl('y')      → tab_index:39,
│   │   │   type:integer, lev:1
│   │   └── ProcDecl(Add)      →
│   │       tab_index:40, type:void, lev:1
│   │       ├── VarDecl('a')   →
│   │       │   tab_index:41, type:integer, lev:2
│   │       │   ├── VarDecl('b') →
│   │       │   │   tab_index:42, type:integer, lev:2
│   │       │   │   └── CompoundStmt → type:void
│   │       │   │       └── Assign(x := a + b) →
│   │       │   type:void
│   │       └── Var(x)          →
│   │           tab_index:38, type:integer, lev:1
│   │           └── BinOp(plus) →
│   │               type:integer
│   └── Var(a)                  →
│       tab_index:41, type:integer, lev:2

```

		└─ Var(b)	→
tab_index:42, type:integer, lev:2		└─ CompoundStmt	→ type:void
		└─ Assign(x := 5)	→ type:void
		└─ Var(x)	→ tab_index:38,
type:integer, lev:1		└─ Num(5)	→ type:integer
		└─ Assign(y := 3)	→ type:void
		└─ Var(y)	→ tab_index:39,
type:integer, lev:1		└─ Num(3)	→ type:integer
		└─ ProcedureCall(name: 'Add')	→
type:void		└─ Var(x)	→ tab_index:38,
type:integer, lev:1		└─ Var(y)	→ tab_index:39,
type:integer, lev:1		└─ Var(x)	→ tab_index:38,
type:integer, lev:1		└─ ProcedureCall(name: 'Add')	→
type:void		└─ Assign(x := Add)	→ type:void
		└─ Var(x)	→ tab_index:38,
type:integer, lev:1		└─ Var(Add)	→ type:unknown
		└─ ProcedureCall(name: 'writeln')	
→ type:void			
expected 2, got 3			[ERROR] Wrong number of arguments for 'Add':
expected 2, got 0			[ERROR] Wrong number of arguments for 'Add':
variable			[ERROR] 'Add' is a procedure, expected a
			=== Semantic Analysis ===
			Errors : 3
			Warnings: 0
			Status : FAILED

- Analisis

wrongObjectKindError saat prosedur digunakan sebagai nilai ekspresi di sisi kanan assignment

3.2.16. Kasus Uji 16

- Kasus: Happy Path untuk Semua Jenis Deklarasi

- Tujuan: Uji semua visit functions berjalan tanpa error, semua identifier masuk tab dengan atribut yang benar
- Input:

```

program TC16;
const
  MAX == 100;
  PI == 3;
type
  Score = integer;
var
  x: integer;
  flag: boolean;
function max(a: integer; b: integer): integer;
begin
end;
procedure cetak(val: integer);
begin
end;
begin
end.

```

- Output:

=== Symbol Table ===

TAB (Identifier Table)

idx	name	obj	type	ref	nrm	lev	adr	link
1	AND	keyword	Unknown	0	1	0	0	0
2	ARRAY	keyword	Unknown	0	1	0	0	1
3	BEGIN	keyword	Unknown	0	1	0	0	2
4	CASE	keyword	Unknown	0	1	0	0	3
5	CONST	keyword	Unknown	0	1	0	0	4
6	DIV	keyword	Unknown	0	1	0	0	5
7	DOWNT0	keyword	Unknown	0	1	0	0	6
8	DO	keyword	Unknown	0	1	0	0	7
9	ELSE	keyword	Unknown	0	1	0	0	8
10	END	keyword	Unknown	0	1	0	0	9
11	FOR	keyword	Unknown	0	1	0	0	10
12	FUNCTION	keyword	Unknown	0	1	0	0	11
13	IF	keyword	Unknown	0	1	0	0	12
14	MOD	keyword	Unknown	0	1	0	0	13
15	NOT	keyword	Unknown	0	1	0	0	14

16	OF	keyword	Unknown	0	1	0	0	15
17	OR	keyword	Unknown	0	1	0	0	16
18	PROCEDURE	keyword	Unknown	0	1	0	0	17
19	PROGRAM	keyword	Unknown	0	1	0	0	18
20	RECORD	keyword	Unknown	0	1	0	0	19
21	REPEAT	keyword	Unknown	0	1	0	0	20
22	INTEGER	type	Integer	0	1	0	0	21
23	REAL	type	Real	0	1	0	0	22
24	BOOLEAN	type	Boolean	0	1	0	0	23
25	CHAR	type	Char	0	1	0	0	24
26	STRING	type	String	0	1	0	0	25
27	THEN	keyword	Unknown	0	1	0	0	26
28	TO	keyword	Unknown	0	1	0	0	27
29	TYPE	keyword	Unknown	0	1	0	0	28
30	UNTIL	keyword	Unknown	0	1	0	0	29
31	VAR	keyword	Unknown	0	1	0	0	30
32	WHILE	keyword	Unknown	0	1	0	0	31
33	true	constant	Boolean	0	1	0	1	32
34	false	constant	Boolean	0	1	0	0	33
35	writeln	procedure	Void	0	1	0	0	34
36	readln	procedure	Void	0	1	0	0	35
37	TC18	program	Void	1	1	0	0	36
38	MAX	constant	Integer	0	1	1	100	0
39	PI	constant	Integer	0	1	1	3	38
40	Score	type	Integer	0	1	1	0	39
41	x	variable	Integer	0	1	1	0	40
42	flag	variable	Boolean	0	1	1	1	41
43	max	function	Integer	2	1	1	0	42
44	a	variable	Integer	0	1	2	0	0
45	b	variable	Integer	0	1	2	1	44
46	cetak	procedure	Void	3	1	1	0	43
47	val	variable	Integer	0	1	2	0	0

BTAB (Block Table)

idx	last	lpar	psze	vsze
-----	------	------	------	------

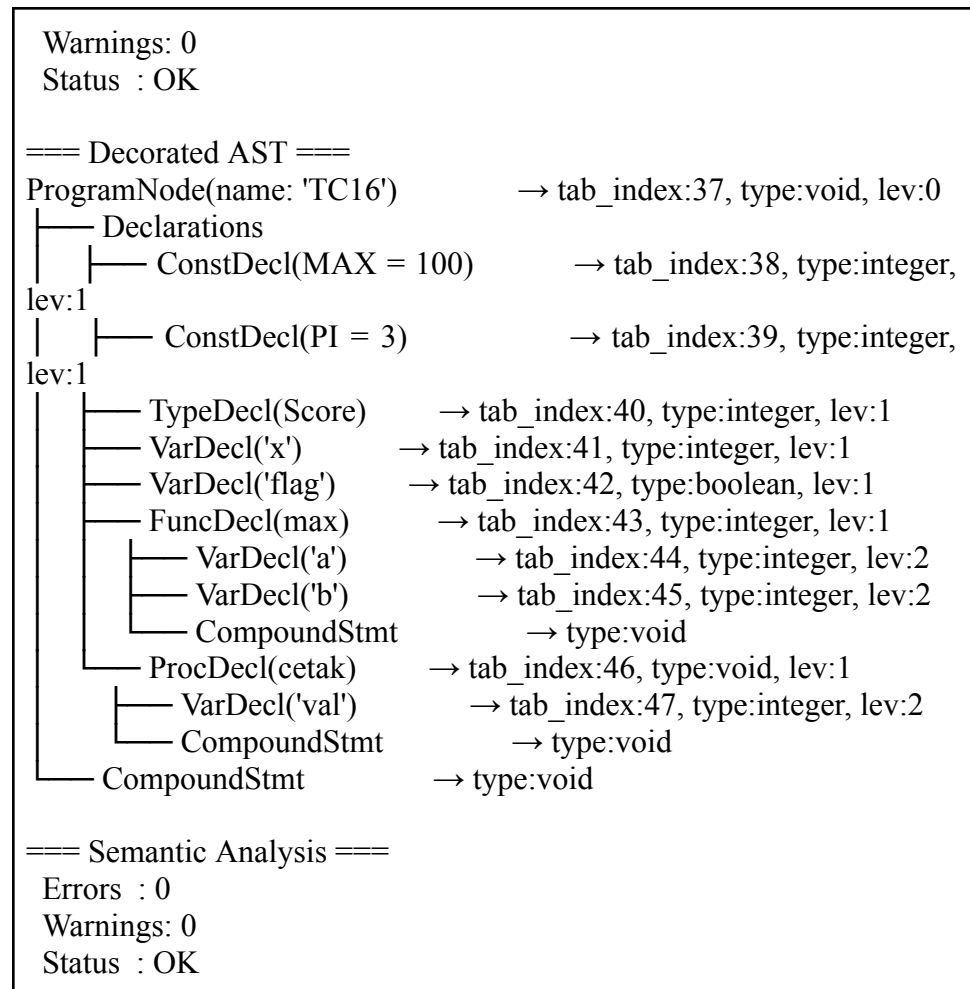
0	37	0	0	0
1	46	0	0	2
2	45	45	2	0
3	47	47	1	0

ATAB (Array Table)

(kosong)

=== Semantic Analysis ===

Errors : 0



- Analisis

Memverifikasi bahwa visitProgram(), visitConstDecl(), visitTypeDecl(), visitVarDecl(), visitFuncDecl(), dan visitProcDecl() berjalan dengan benar. Semua identifier masuk ke symbol table dengan lev, type, dan tab_index yang sesuai. Fungsi max terdaftar sebagai FUNCTION dengan return type integer, sedangkan cetak terdaftar sebagai PROCEDURE bertipe void. Parameter a, b, dan val berada di lev:2 sesuai scope masing-masing

3.2.17. Kasus Uji 17

- Kasus: Redeclaration Variable

- Tujuan: Uji redeclarationError pada visitVarDecl ketika variabel dideklarasikan dua kali di scope yang sama
- Input:

```
program TC17;
var
  a: integer;
  a: real;
  b: boolean;
  b: char;
begin
end.
```

- Output:

```
[ERROR] Redeclaration of 'a' in the same scope
[ERROR] Redeclaration of 'b' in the same scope
```

=== Symbol Table ===

TAB (Identifier Table)

idx	name	obj	type	ref	nrm	lev	adr	link
1	AND	keyword	Unknown	0	1	0	0	0
2	ARRAY	keyword	Unknown	0	1	0	0	1
3	BEGIN	keyword	Unknown	0	1	0	0	2
4	CASE	keyword	Unknown	0	1	0	0	3
5	CONST	keyword	Unknown	0	1	0	0	4
6	DIV	keyword	Unknown	0	1	0	0	5
7	DOWNTTO	keyword	Unknown	0	1	0	0	6
8	DO	keyword	Unknown	0	1	0	0	7
9	ELSE	keyword	Unknown	0	1	0	0	8
10	END	keyword	Unknown	0	1	0	0	9
11	FOR	keyword	Unknown	0	1	0	0	10
12	FUNCTION	keyword	Unknown	0	1	0	0	11
13	IF	keyword	Unknown	0	1	0	0	12
14	MOD	keyword	Unknown	0	1	0	0	13
15	NOT	keyword	Unknown	0	1	0	0	14
16	OF	keyword	Unknown	0	1	0	0	15
17	OR	keyword	Unknown	0	1	0	0	16
18	PROCEDURE	keyword	Unknown	0	1	0	0	17
19	PROGRAM	keyword	Unknown	0	1	0	0	18
20	RECORD	keyword	Unknown	0	1	0	0	19
21	REPEAT	keyword	Unknown	0	1	0	0	20

22	INTEGER	type	Integer	0	1	0	0	21
23	REAL	type	Real	0	1	0	0	22
24	BOOLEAN	type	Boolean	0	1	0	0	23
25	CHAR	type	Char	0	1	0	0	24
26	STRING	type	String	0	1	0	0	25
27	THEN	keyword	Unknown	0	1	0	0	26
28	TO	keyword	Unknown	0	1	0	0	27
29	TYPE	keyword	Unknown	0	1	0	0	28
30	UNTIL	keyword	Unknown	0	1	0	0	29
31	VAR	keyword	Unknown	0	1	0	0	30
32	WHILE	keyword	Unknown	0	1	0	0	31
33	true	constant	Boolean	0	1	0	1	32
34	false	constant	Boolean	0	1	0	0	33
35	writeln	procedure	Void	0	1	0	0	34
36	readln	procedure	Void	0	1	0	0	35
37	TC19	program	Void	1	1	0	0	36
38	a	variable	Integer	0	1	1	0	0
39	b	variable	Boolean	0	1	1	1	38

BTAB (Block Table)

idx	last	lpar	psze	vsze
-----	------	------	------	------

0	37	0	0	0
1	39	0	0	2

ATAB (Array Table)

(kosong)

=== Semantic Analysis ===

Errors : 2

Warnings: 0

Status : FAILED

=== Decorated AST ===

ProgramNode(name: 'TC17') → tab_index:37, type:void, lev:0

```

├── Declarations
│   ├── VarDecl('a') → tab_index:38, type:integer, lev:1
│   ├── VarDecl('a') → type:unknown
│   ├── VarDecl('b') → tab_index:39, type:boolean, lev:1
│   └── VarDecl('b') → type:unknown
└── CompoundStmt → type:void

```

=== Semantic Analysis ===

Errors : 2

Warnings: 0

Status : FAILED

- Analisis

Memverifikasi bahwa lookupCurrentScope() mendeteksi dua redeclaration secara independen. Identifier pertama dari masing-masing nama tetap valid di symbol table, sedangkan yang kedua ditolak dan node-nya dianotasi type:unknown tanpa tab_index. Error count tepat 2 sesuai jumlah redeclaration.

3.2.18. Kasus Uji 18

- Kasus: Redeclaration Konstanta
- Tujuan: Uji redeclarationError pada visitConstDecl ketika konstanta dideklarasikan dua kali di scope yang sama
- Input:

```
program TC18;
const
  MAX == 100;
  MAX == 200;
begin
end.
```

- Output:

[ERROR] Redeclaration of 'MAX' in the same scope

=== Symbol Table ===

TAB (Identifier Table)

idx	name	obj	type	ref	nrm	lev	adr	link
1	AND	keyword	Unknown	0	1	0	0	0
2	ARRAY	keyword	Unknown	0	1	0	0	1
3	BEGIN	keyword	Unknown	0	1	0	0	2
4	CASE	keyword	Unknown	0	1	0	0	3
5	CONST	keyword	Unknown	0	1	0	0	4
6	DIV	keyword	Unknown	0	1	0	0	5
7	DOWNT0	keyword	Unknown	0	1	0	0	6
8	DO	keyword	Unknown	0	1	0	0	7
9	ELSE	keyword	Unknown	0	1	0	0	8
10	END	keyword	Unknown	0	1	0	0	9
11	FOR	keyword	Unknown	0	1	0	0	10

12	FUNCTION	keyword	Unknown	0	1	0	0	11
13	IF	keyword	Unknown	0	1	0	0	12
14	MOD	keyword	Unknown	0	1	0	0	13
15	NOT	keyword	Unknown	0	1	0	0	14
16	OF	keyword	Unknown	0	1	0	0	15
17	OR	keyword	Unknown	0	1	0	0	16
18	PROCEDURE	keyword	Unknown	0	1	0	0	17
19	PROGRAM	keyword	Unknown	0	1	0	0	18
20	RECORD	keyword	Unknown	0	1	0	0	19
21	REPEAT	keyword	Unknown	0	1	0	0	20
22	INTEGER	type	Integer	0	1	0	0	21
23	REAL	type	Real	0	1	0	0	22
24	BOOLEAN	type	Boolean	0	1	0	0	23
25	CHAR	type	Char	0	1	0	0	24
26	STRING	type	String	0	1	0	0	25
27	THEN	keyword	Unknown	0	1	0	0	26
28	TO	keyword	Unknown	0	1	0	0	27
29	TYPE	keyword	Unknown	0	1	0	0	28
30	UNTIL	keyword	Unknown	0	1	0	0	29
31	VAR	keyword	Unknown	0	1	0	0	30
32	WHILE	keyword	Unknown	0	1	0	0	31
33	true	constant	Boolean	0	1	0	1	32
34	false	constant	Boolean	0	1	0	0	33
35	writeln	procedure	Void	0	1	0	0	34
36	readln	procedure	Void	0	1	0	0	35
37	TC20	program	Void	1	1	0	0	36
38	MAX	constant	Integer	0	1	1	100	0

BTAB (Block Table)

idx	last	lpar	psze	vsze

0	37	0	0	0
1	38	0	0	0

ATAB (Array Table)

(kosong)

=== Semantic Analysis ===

Errors : 1

Warnings: 0

Status : FAILED

=== Decorated AST ===

ProgramNode(name: 'TC18')

→ tab_index:37, type:void, lev:0

├── Declarations

│ └── ConstDecl(MAX = 100)

→ tab_index:38, type:integer,

```

lev:1
├── ConstDecl(MAX = 200)    → type:unknown
└── CompoundStmt           → type:void

```

=== Semantic Analysis ===

Errors : 1
Warnings: 0
Status : FAILED

- Analisis

Memverifikasi bahwa visitConstDecl() mendeteksi redeclaration konstanta. MAX pertama dengan nilai 100 berhasil masuk tab, sedangkan MAX kedua dengan nilai 200 ditolak dan dianotasi type:unknown. Field adr pada tab[38] terisi dengan nilai konstanta 100.

3.2.19. Kasus Uji 19

- Kasus: Redeclaration Prosedur
- Tujuan: Uji redeclarationError pada visitProcDecl ketika prosedur dideklarasikan dua kali di scope yang sama
- Input:

```

program TC19;
procedure hitung(x: integer);
begin
end;
procedure hitung(y: real);
begin
end;
begin
end.

```

- Output:

[ERROR] Redeclaration of 'hitung' in the same scope

=== Symbol Table ===

TAB (Identifier Table)

idx	name	obj	type	ref	nrm	lev	adr	link
-----	------	-----	------	-----	-----	-----	-----	------

1	AND	keyword	Unknown	0	1	0	0	0	
2	ARRAY	keyword	Unknown	0	1	0	0	1	
3	BEGIN	keyword	Unknown	0	1	0	0	2	
4	CASE	keyword	Unknown	0	1	0	0	3	
5	CONST	keyword	Unknown	0	1	0	0	4	
6	DIV	keyword	Unknown	0	1	0	0	5	
7	DOWNT0	keyword	Unknown	0	1	0	0	6	
8	DO	keyword	Unknown	0	1	0	0	7	
9	ELSE	keyword	Unknown	0	1	0	0	8	
10	END	keyword	Unknown	0	1	0	0	9	
11	FOR	keyword	Unknown	0	1	0	0	10	
12	FUNCTION	keyword	Unknown	0	1	0	0	11	
13	IF	keyword	Unknown	0	1	0	0	12	
14	MOD	keyword	Unknown	0	1	0	0	13	
15	NOT	keyword	Unknown	0	1	0	0	14	
16	OF	keyword	Unknown	0	1	0	0	15	
17	OR	keyword	Unknown	0	1	0	0	16	
18	PROCEDURE	keyword	Unknown	0	1	0	0	17	
19	PROGRAM	keyword	Unknown	0	1	0	0	18	
20	RECORD	keyword	Unknown	0	1	0	0	19	
21	REPEAT	keyword	Unknown	0	1	0	0	20	
22	INTEGER	type	Integer	0	1	0	0	21	
23	REAL	type	Real	0	1	0	0	22	
24	BOOLEAN	type	Boolean	0	1	0	0	23	
25	CHAR	type	Char	0	1	0	0	24	
26	STRING	type	String	0	1	0	0	25	
27	THEN	keyword	Unknown	0	1	0	0	26	
28	TO	keyword	Unknown	0	1	0	0	27	
29	TYPE	keyword	Unknown	0	1	0	0	28	
30	UNTIL	keyword	Unknown	0	1	0	0	29	
31	VAR	keyword	Unknown	0	1	0	0	30	
32	WHILE	keyword	Unknown	0	1	0	0	31	
33	true	constant	Boolean	0	1	0	1	32	
34	false	constant	Boolean	0	1	0	0	33	
35	writeln	procedure	Void	0	1	0	0	34	
36	readln	procedure	Void	0	1	0	0	35	
37	TC21	program	Void	1	1	0	0	36	
38	hitung	procedure	Void	2	1	1	0	0	
39	x	variable	Integer	0	1	2	0	0	

BTAB (Block Table)

idx last lpar psze vsze

0	37	0	0	0
1	38	0	0	0

```

2  39  39  1  0

ATAB (Array Table)
(kosong)

=== Semantic Analysis ===
Errors : 1
Warnings: 0
Status : FAILED

=== Decorated AST ===
ProgramNode(name: 'TC19')      → tab_index:37, type:void, lev:0
├── Declarations
│   ├── ProcDecl(hitung)      → tab_index:38, type:void, lev:1
│   │   ├── VarDecl('x')      → tab_index:39, type:integer, lev:2
│   │   └── CompoundStmt      → type:void
│   └── ProcDecl(hitung)      → type:void
│       ├── VarDecl('y')      → type:real
│       └── CompoundStmt      → type:void
└── CompoundStmt              → type:void

=== Semantic Analysis ===
Errors : 1
Warnings: 0
Status : FAILED

```

- Analisis

Memverifikasi bahwa visitProcDecl() mendeteksi redeclaration prosedur di scope yang sama. Prosedur hitung pertama beserta parameternya x berhasil masuk tab, sedangkan hitung kedua ditolak sebelum parameternya diproses. Node hitung kedua tidak memiliki tab_index karena enterTab() tidak dipanggil.

3.2.20. Kasus Uji 20

- Kasus: Variabel Lokal Tidak Konflik dengan Variabel Global
- Tujuan: Uji bahwa lookupCurrentScope() hanya memeriksa scope aktif sehingga variabel dengan nama sama di scope berbeda tidak dianggap redeclaration
- Input:

```

program TC20;
var
  x: integer;
procedure hitung;
var
  x: real;
begin
end;
begin
end.

```

- Output:

=== Symbol Table ===

TAB (Identifier Table)

idx	name	obj	type	ref	nrm	lev	adr	link
1	AND	keyword	Unknown	0	1	0	0	0
2	ARRAY	keyword	Unknown	0	1	0	0	1
3	BEGIN	keyword	Unknown	0	1	0	0	2
4	CASE	keyword	Unknown	0	1	0	0	3
5	CONST	keyword	Unknown	0	1	0	0	4
6	DIV	keyword	Unknown	0	1	0	0	5
7	DOWNT0	keyword	Unknown	0	1	0	0	6
8	DO	keyword	Unknown	0	1	0	0	7
9	ELSE	keyword	Unknown	0	1	0	0	8
10	END	keyword	Unknown	0	1	0	0	9
11	FOR	keyword	Unknown	0	1	0	0	10
12	FUNCTION	keyword	Unknown	0	1	0	0	11
13	IF	keyword	Unknown	0	1	0	0	12
14	MOD	keyword	Unknown	0	1	0	0	13
15	NOT	keyword	Unknown	0	1	0	0	14
16	OF	keyword	Unknown	0	1	0	0	15
17	OR	keyword	Unknown	0	1	0	0	16
18	PROCEDURE	keyword	Unknown	0	1	0	0	17
19	PROGRAM	keyword	Unknown	0	1	0	0	18
20	RECORD	keyword	Unknown	0	1	0	0	19
21	REPEAT	keyword	Unknown	0	1	0	0	20
22	INTEGER	type	Integer	0	1	0	0	21
23	REAL	type	Real	0	1	0	0	22
24	BOOLEAN	type	Boolean	0	1	0	0	23
25	CHAR	type	Char	0	1	0	0	24
26	STRING	type	String	0	1	0	0	25
27	THEN	keyword	Unknown	0	1	0	0	26
28	TO	keyword	Unknown	0	1	0	0	27

29	TYPE	keyword	Unknown	0	1	0	0	28
30	UNTIL	keyword	Unknown	0	1	0	0	29
31	VAR	keyword	Unknown	0	1	0	0	30
32	WHILE	keyword	Unknown	0	1	0	0	31
33	true	constant	Boolean	0	1	0	1	32
34	false	constant	Boolean	0	1	0	0	33
35	writeln	procedure	Void	0	1	0	0	34
36	readln	procedure	Void	0	1	0	0	35
37	TC22	program	Void	1	1	0	0	36
38	x	variable	Integer	0	1	1	0	0
39	hitung	procedure	Void	2	1	1	0	38
40	x	variable	Real	0	1	2	0	0

BTAB (Block Table)

idx	last	lpar	psze	vsze
-----	------	------	------	------

0	37	0	0	0
1	39	0	0	1
2	40	0	0	2

ATAB (Array Table)
(kosong)

=== Semantic Analysis ===
Errors : 0
Warnings: 0
Status : OK

=== Decorated AST ===
ProgramNode(name: 'TC20') → tab_index:37, type:void, lev:0

```

├── Declarations
│   ├── VarDecl('x') → tab_index:38, type:integer, lev:1
│   ├── ProcDecl(hitung) → tab_index:39, type:void, lev:1
│   │   └── CompoundStmt → type:void
│   │       └── VarDecl('x') → tab_index:40, type:real, lev:2
│   └── CompoundStmt → type:void

```

=== Semantic Analysis ===
Errors : 0
Warnings: 0
Status : OK

- Analisis:
Memverifikasi bahwa dua variabel bernama x di scope berbeda tidak menghasilkan error. x global terdaftar di tab[38] dengan lev:1 bertipe

integer, sedangkan x lokal di dalam prosedur hitung terdaftar di tab[40] dengan lev:2 bertipe real. Mekanisme pushScope() dan popScope() memastikan kedua identifier berada di blok yang berbeda sehingga lookupCurrentScope() tidak mendeteksi konflik.

3.2.21. Kasus Uji 21

- Kasus: Redeclaration Parameter
- Tujuan: Uji redeclarationError pada visitProcDecl ketika dua parameter memiliki nama yang sama
- Input:

```
program TC21;
procedure hitung(x: integer; x: real);
begin
end;
begin
end.
```

- Output:

[ERROR] Redeclaration of 'x' in the same scope

==== Symbol Table ====

TAB (Identifier Table)

idx	name	obj	type	ref	nrm	lev	adr	link
1	AND	keyword	Unknown	0	1	0	0	0
2	ARRAY	keyword	Unknown	0	1	0	0	1
3	BEGIN	keyword	Unknown	0	1	0	0	2
4	CASE	keyword	Unknown	0	1	0	0	3
5	CONST	keyword	Unknown	0	1	0	0	4
6	DIV	keyword	Unknown	0	1	0	0	5
7	DOWNTO	keyword	Unknown	0	1	0	0	6
8	DO	keyword	Unknown	0	1	0	0	7
9	ELSE	keyword	Unknown	0	1	0	0	8
10	END	keyword	Unknown	0	1	0	0	9
11	FOR	keyword	Unknown	0	1	0	0	10
12	FUNCTION	keyword	Unknown	0	1	0	0	11
13	IF	keyword	Unknown	0	1	0	0	12
14	MOD	keyword	Unknown	0	1	0	0	13

15	NOT	keyword	Unknown	0	1	0	0	14
16	OF	keyword	Unknown	0	1	0	0	15
17	OR	keyword	Unknown	0	1	0	0	16
18	PROCEDURE	keyword	Unknown	0	1	0	0	17
19	PROGRAM	keyword	Unknown	0	1	0	0	18
20	RECORD	keyword	Unknown	0	1	0	0	19
21	REPEAT	keyword	Unknown	0	1	0	0	20
22	INTEGER	type	Integer	0	1	0	0	21
23	REAL	type	Real	0	1	0	0	22
24	BOOLEAN	type	Boolean	0	1	0	0	23
25	CHAR	type	Char	0	1	0	0	24
26	STRING	type	String	0	1	0	0	25
27	THEN	keyword	Unknown	0	1	0	0	26
28	TO	keyword	Unknown	0	1	0	0	27
29	TYPE	keyword	Unknown	0	1	0	0	28
30	UNTIL	keyword	Unknown	0	1	0	0	29
31	VAR	keyword	Unknown	0	1	0	0	30
32	WHILE	keyword	Unknown	0	1	0	0	31
33	true	constant	Boolean	0	1	0	1	32
34	false	constant	Boolean	0	1	0	0	33
35	writeln	procedure	Void	0	1	0	0	34
36	readln	procedure	Void	0	1	0	0	35
37	TC23	program	Void	1	1	0	0	36
38	hitung	procedure	Void	2	1	1	0	0
39	x	variable	Integer	0	1	2	0	0

BTAB (Block Table)

idx	last	lpar	psze	vsze
-----	------	------	------	------

0	37	0	0	0
1	38	0	0	0
2	39	39	1	0

ATAB (Array Table)

(kosong)

=== Semantic Analysis ===

Errors : 1

Warnings: 0

Status : FAILED

=== Decorated AST ===

ProgramNode(name: 'TC21') → tab_index:37, type:void, lev:0

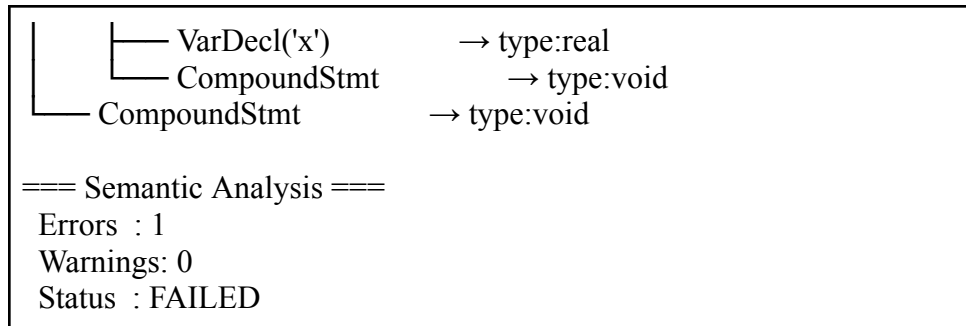
├── Declarations

├── ProcDecl(hitung)

→ tab_index:38, type:void, lev:1

└── VarDecl('x')

→ tab_index:39, type:integer, lev:2



- Analisis

Memverifikasi bahwa visitProcDecl() mendeteksi redeclaration di antara parameter prosedur. Parameter x pertama bertipe integer berhasil masuk tab[39], sedangkan x kedua bertipe real ditolak karena lookupCurrentScope() menemukan x sudah ada di scope prosedur. btab[2].lpar=39 dan psze=1 mengkonfirmasi hanya satu parameter yang terdaftar.

3.2.22. Kasus Uji 22

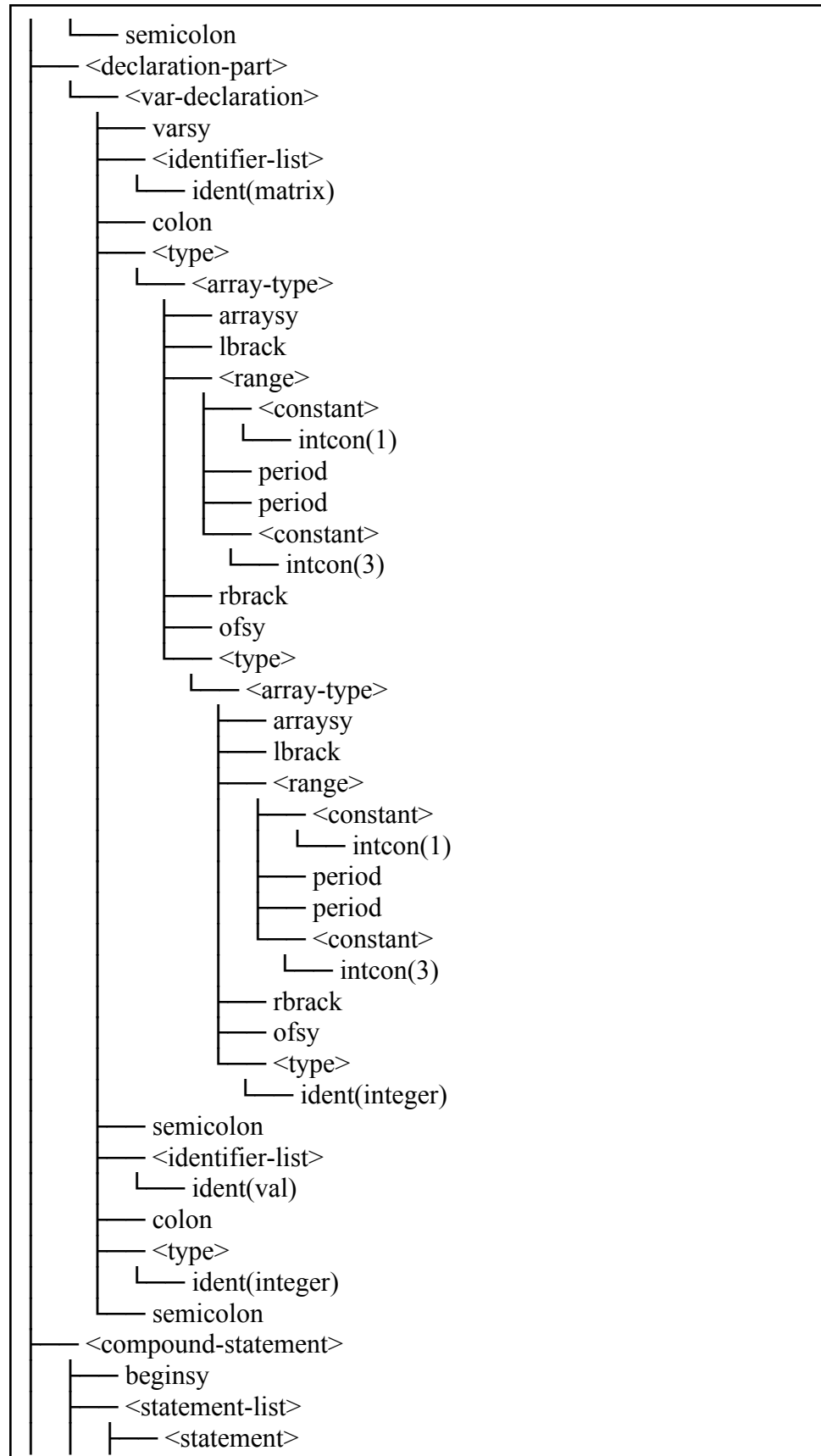
- Kasus: Array of Array
- Tujuan: Memverifikasi bahwa deklarasi dan akses array multidimensi (array of array) dapat dipetakan dengan benar ke ArrayAccessNode bertingkat dalam AST

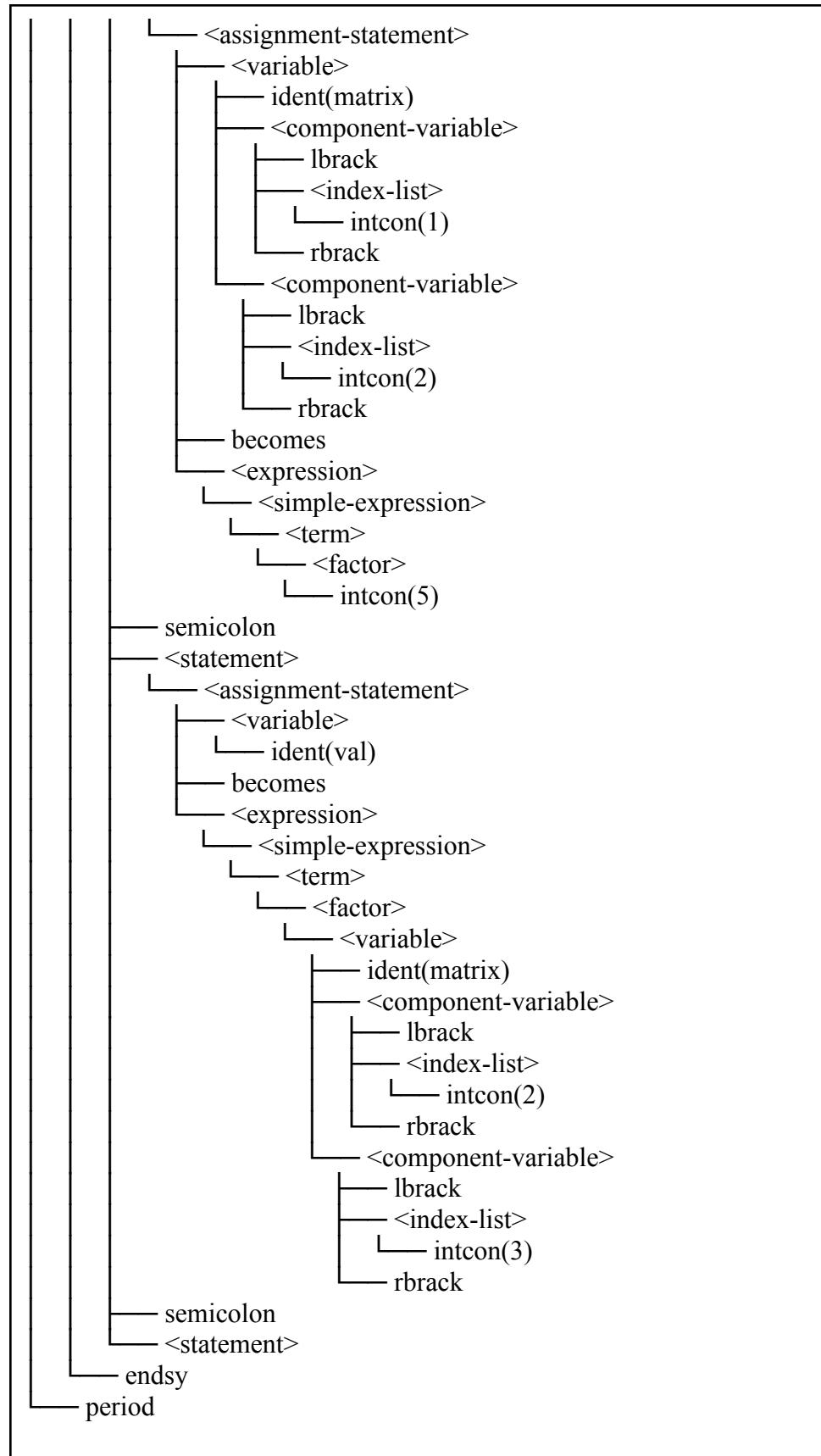
- Input:

```
program ArrayOfArrayTest;
var
  matrix: array[1..3] of array[1..3] of integer;
  val: integer;
begin
  matrix[1][2] := 5;
  val := matrix[2][3];
end.
```

- Output:

```
<program>
└─ <program-header>
   └─ programsy
      └─ ident(ArrayOfArrayTest)
```





=== Symbol Table ===

TAB (Identifier Table)

idx	name	obj	type	ref	nrm	lev	adr	link
1	AND	keyword	Unknown	0	1	0	0	0
2	ARRAY	keyword	Unknown	0	1	0	0	1
3	BEGIN	keyword	Unknown	0	1	0	0	2
4	CASE	keyword	Unknown	0	1	0	0	3
5	CONST	keyword	Unknown	0	1	0	0	4
6	DIV	keyword	Unknown	0	1	0	0	5
7	DOWNTO	keyword	Unknown	0	1	0	0	6
8	DO	keyword	Unknown	0	1	0	0	7
9	ELSE	keyword	Unknown	0	1	0	0	8
10	END	keyword	Unknown	0	1	0	0	9
11	FOR	keyword	Unknown	0	1	0	0	10
12	FUNCTION	keyword	Unknown	0	1	0	0	11
13	IF	keyword	Unknown	0	1	0	0	12
14	MOD	keyword	Unknown	0	1	0	0	13
15	NOT	keyword	Unknown	0	1	0	0	14
16	OF	keyword	Unknown	0	1	0	0	15
17	OR	keyword	Unknown	0	1	0	0	16
18	PROCEDURE	keyword	Unknown	0	1	0	0	17
19	PROGRAM	keyword	Unknown	0	1	0	0	18
20	RECORD	keyword	Unknown	0	1	0	0	19
21	REPEAT	keyword	Unknown	0	1	0	0	20
22	INTEGER	type	Integer	0	1	0	0	21
23	REAL	type	Real	0	1	0	0	22
24	BOOLEAN	type	Boolean	0	1	0	0	23
25	CHAR	type	Char	0	1	0	0	24
26	STRING	type	String	0	1	0	0	25
27	THEN	keyword	Unknown	0	1	0	0	26
28	TO	keyword	Unknown	0	1	0	0	27
29	TYPE	keyword	Unknown	0	1	0	0	28
30	UNTIL	keyword	Unknown	0	1	0	0	29
31	VAR	keyword	Unknown	0	1	0	0	30
32	WHILE	keyword	Unknown	0	1	0	0	31
33	true	constant	Boolean	0	1	0	1	32
34	false	constant	Boolean	0	1	0	0	33
35	writeln	procedure	Void	0	1	0	0	34
36	readln	procedure	Void	0	1	0	0	35
37	ArrayOfArrayTestprogram		Void	1	1	0	0	36
38	matrix	variable	Array	1	1	1	0	0
39	val	variable	Integer	0	1	1	9	38

BTAB (Block Table)

idx	last	lpar	psze	vsze
0	37	0	0	0
1	39	0	0	10

ATAB (Array Table)

idx	xtyp	etyp	eref	low	high	elsz	size
0	Integer	Integer	0	1	3	1	3
1	Integer	Array	0	1	3	3	9

=== Semantic Analysis ===

Errors : 0

Warnings: 0

Status : OK

=== Decorated AST ===

ProgramNode(name: 'ArrayOfArrayTest') → tab_index:37,

type:void, lev:0

Declarations

VarDecl('matrix') → tab_index:38, type:array, lev:1

VarDecl('val') → tab_index:39, type:integer, lev:1

CompoundStmt → type:void

Assign(matrix[1][2] := 5) → type:void

ArrayAccess → type:unknown

ArrayAccess → type:array

Var(matrix) → tab_index:38, type:array, lev:1

Num(1) → type:integer

Num(2) → type:integer

Num(5) → type:integer

Assign(val := matrix[2][3]) → type:void

Var(val) → tab_index:39, type:integer, lev:1

ArrayAccess → type:unknown

ArrayAccess → type:array

Var(matrix) → tab_index:38, type:array, lev:1

Num(2) → type:integer

Num(3) → type:integer

=== Semantic Analysis ===

Errors : 0

Warnings: 0

Status : OK

- Analisis:

Program berhasil dianalisis tanpa error. matrix terdaftar di tab[38] bertipe array. Akses matrix[1][2] dipetakan menjadi ArrayAccess bertingkat

dengan benar, mulai dari `ArrayAccess(ArrayAccess(Var(matrix), Num(1)), hingga Num(2))`. Namun tipe hasil akses kedua (`type:unknown`) menunjukkan semantic analyzer belum bisa resolve tipe elemen dari array of array. Ini bukan bug di `ParseTreeToAST` melainkan keterbatasan resolusi tipe di semantic analyzer.

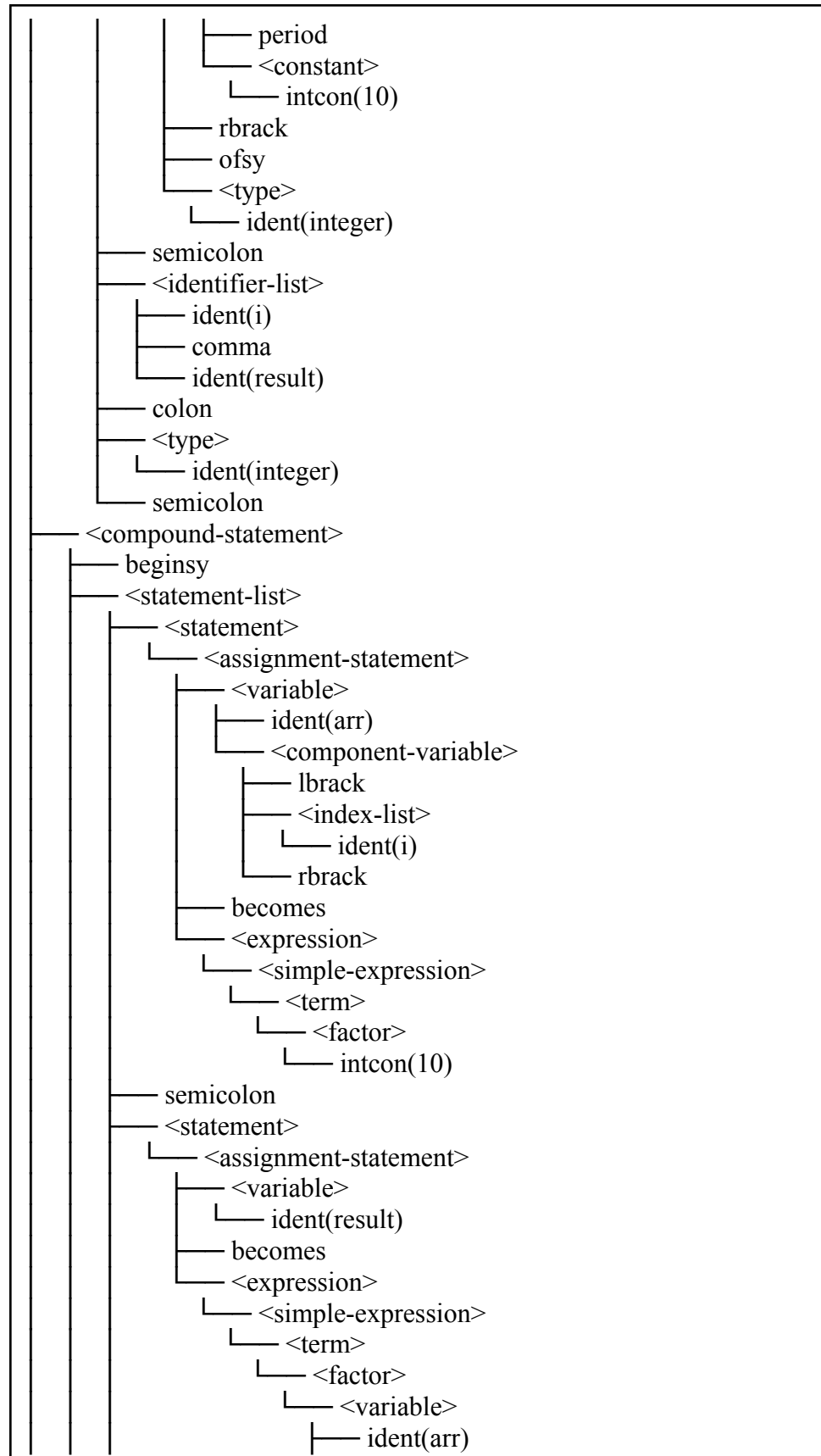
3.2.23. Kasus Uji 23

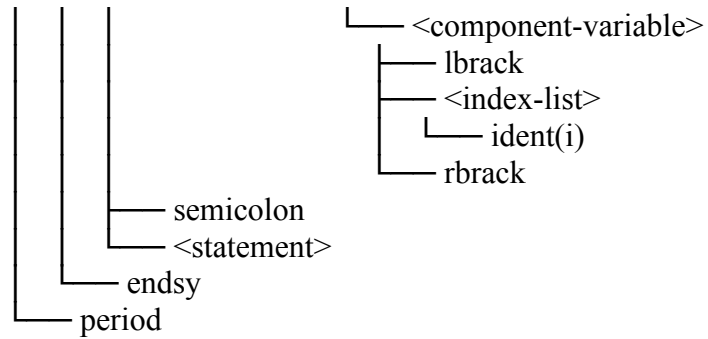
- Kasus: Array Index dengan Variable
- Tujuan: Memverifikasi bahwa index array berupa identifier/variable dapat dipetakan menjadi `VarNode` sebagai child dari `ArrayAccessNode`
- Input:

```
program ArrayExprIndexTest;
var
  arr: array[1..10] of integer;
  i, result: integer;
begin
  arr[i] := 10;
  result := arr[i];
end.
```

- Output:

```
<program>
├── <program-header>
│   ├── programsy
│   ├── ident(ArrayExprIndexTest)
│   └── semicolon
├── <declaration-part>
│   └── <var-declaration>
│       ├── varsy
│       ├── <identifier-list>
│       │   └── ident(arr)
│       ├── colon
│       ├── <type>
│       │   └── <array-type>
│       │       ├── arraysy
│       │       ├── lbrack
│       │       ├── <range>
│       │       │   ├── <constant>
│       │       │   │   └── intcon(1)
│       │       └── period
```





=== Symbol Table ===

TAB (Identifier Table)

idx	name	obj	type	ref	nrm	lev	adr	link

1	AND	keyword	Unknown	0	1	0	0	0
2	ARRAY	keyword	Unknown	0	1	0	0	1
3	BEGIN	keyword	Unknown	0	1	0	0	2
4	CASE	keyword	Unknown	0	1	0	0	3
5	CONST	keyword	Unknown	0	1	0	0	4
6	DIV	keyword	Unknown	0	1	0	0	5
7	DOWNTO	keyword	Unknown	0	1	0	0	6
8	DO	keyword	Unknown	0	1	0	0	7
9	ELSE	keyword	Unknown	0	1	0	0	8
10	END	keyword	Unknown	0	1	0	0	9
11	FOR	keyword	Unknown	0	1	0	0	10
12	FUNCTION	keyword	Unknown	0	1	0	0	11
13	IF	keyword	Unknown	0	1	0	0	12
14	MOD	keyword	Unknown	0	1	0	0	13
15	NOT	keyword	Unknown	0	1	0	0	14
16	OF	keyword	Unknown	0	1	0	0	15
17	OR	keyword	Unknown	0	1	0	0	16
18	PROCEDURE	keyword	Unknown	0	1	0	0	17
19	PROGRAM	keyword	Unknown	0	1	0	0	18
20	RECORD	keyword	Unknown	0	1	0	0	19
21	REPEAT	keyword	Unknown	0	1	0	0	20
22	INTEGER	type	Integer	0	1	0	0	21
23	REAL	type	Real	0	1	0	0	22
24	BOOLEAN	type	Boolean	0	1	0	0	23
25	CHAR	type	Char	0	1	0	0	24
26	STRING	type	String	0	1	0	0	25
27	THEN	keyword	Unknown	0	1	0	0	26
28	TO	keyword	Unknown	0	1	0	0	27
29	TYPE	keyword	Unknown	0	1	0	0	28
30	UNTIL	keyword	Unknown	0	1	0	0	29
31	VAR	keyword	Unknown	0	1	0	0	30


```

32 WHILE      keyword   Unknown   0  1  0  0  31
33 true       constant  Boolean   0  1  0  1  32
34 false      constant  Boolean   0  1  0  0  33
35 writeln    procedure Void      0  1  0  0  34
36 readln     procedure Void      0  1  0  0  35
37 ArrayExprIndexTestprogram Void    1  1  0  0  36
38 arr        variable  Array     0  1  1  0  0
39 i          variable  Integer   0  1  1  10 38
40 result     variable  Integer   0  1  1  11 39

```

BTAB (Block Table)

```

idx last lpar psze vsze
-----

```

```

0 37 0 0 0
1 40 0 0 12

```

ATAB (Array Table)

```

idx xtyp etyp eref low high elsz size
-----

```

```

0 Integer Integer 0 1 10 1 10

```

=== Semantic Analysis ===

Errors : 0

Warnings: 0

Status : OK

=== Decorated AST ===

ProgramNode(name: 'ArrayExprIndexTest') → tab_index:37,

type:void, lev:0

```

├── Declarations
│   ├── VarDecl('arr') → tab_index:38, type:array, lev:1
│   ├── VarDecl('i') → tab_index:39, type:integer, lev:1
│   └── VarDecl('result') → tab_index:40, type:integer, lev:1
├── CompoundStmt → type:void
│   ├── Assign(arr[i] := 10) → type:void
│   │   ├── ArrayAccess → type:integer
│   │   │   ├── Var(arr) → tab_index:38, type:array, lev:1
│   │   │   └── Var(i) → tab_index:39, type:integer, lev:1
│   │   └── Num(10) → type:integer
│   ├── Assign(result := arr[i]) → type:void
│   │   ├── Var(result) → tab_index:40, type:integer, lev:1
│   │   └── ArrayAccess → type:integer
│   │       ├── Var(arr) → tab_index:38, type:array, lev:1
│   │       └── Var(i) → tab_index:39, type:integer, lev:1

```

=== Semantic Analysis ===

Errors : 0
Warnings: 0
Status : OK

- Analisis:

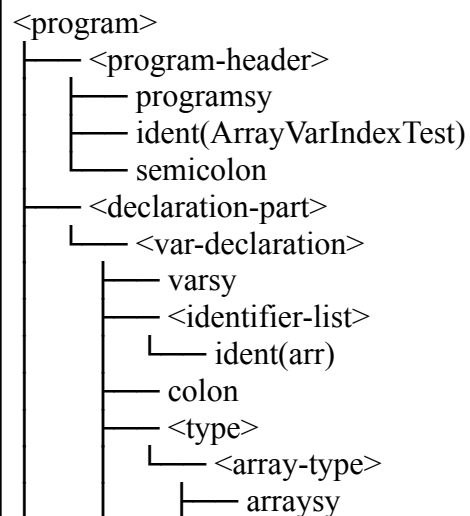
Program berhasil dianalisis tanpa error. Index array berupa variable i dipetakan menjadi VarNode(i) sebagai child ArrayAccess dengan benar. Tipe hasil akses array terisi type:integer sesuai tipe elemen array yang dideklarasikan.

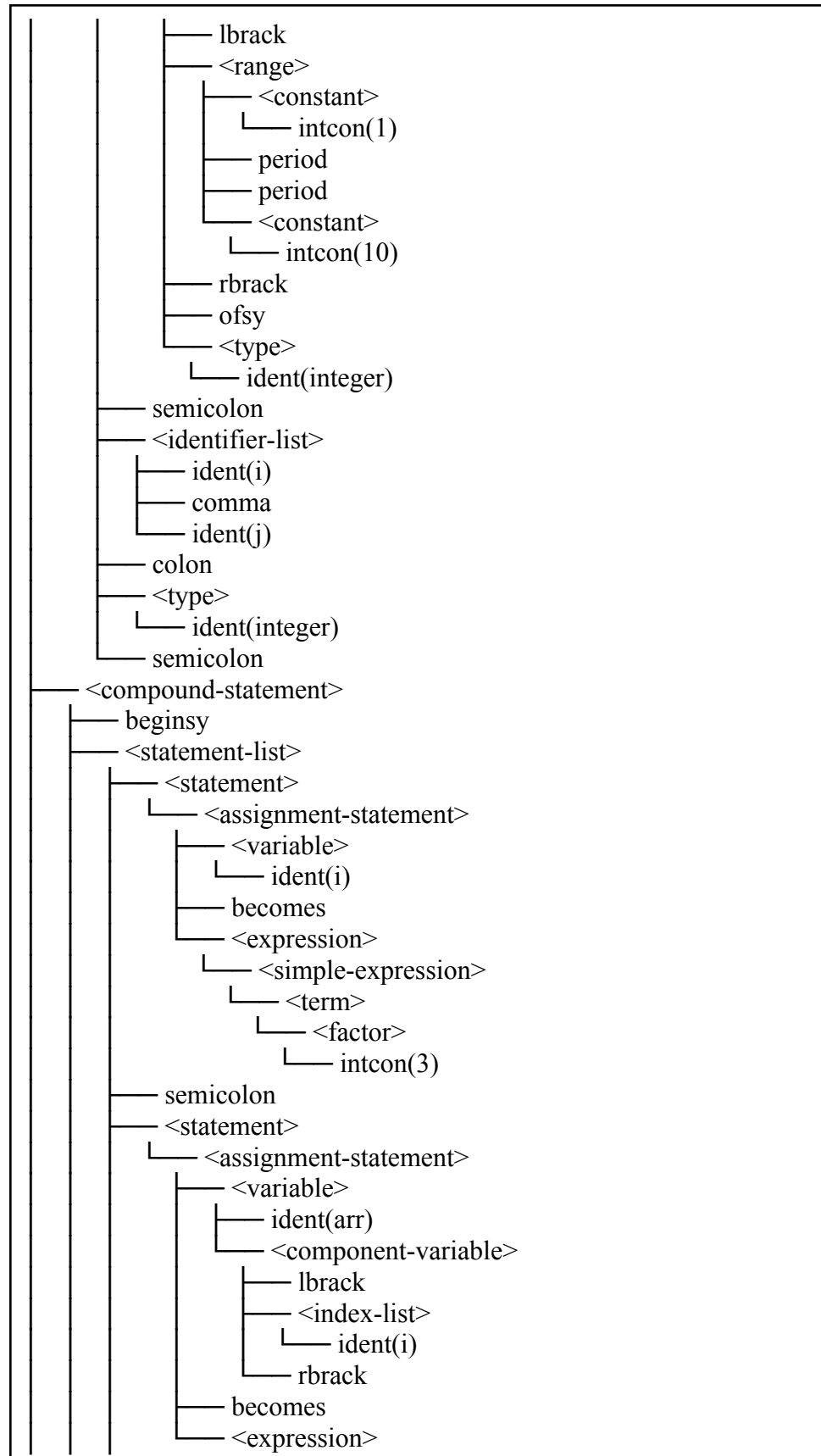
3.2.24. Kasus Uji 24

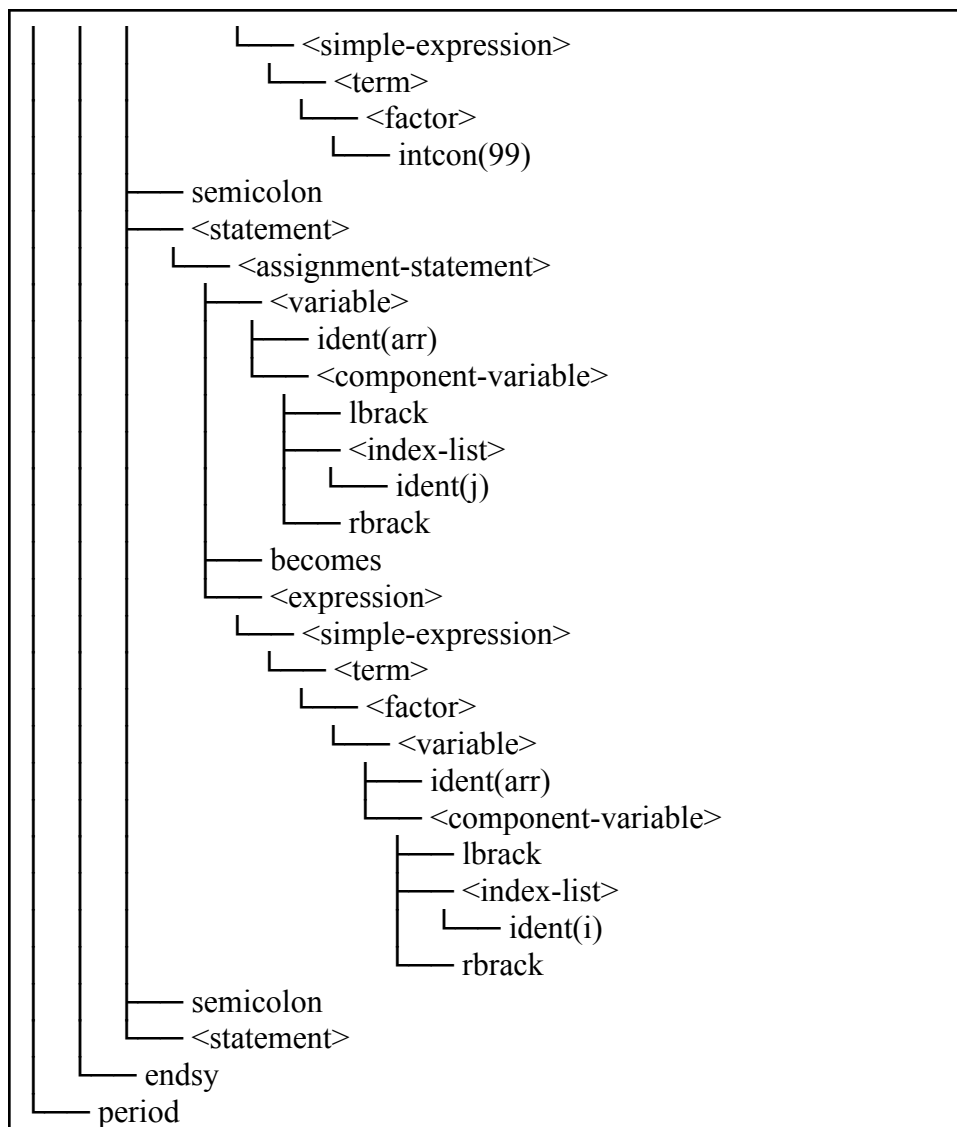
- Kasus: Array Index Variable pada Sisi Kiri dan Kanan Assignment
- Tujuan: Memverifikasi bahwa akses array dengan index variable dapat digunakan baik sebagai target assignment maupun sebagai nilai ekspresi
- Input:

```
program ArrayVarIndexTest;  
var  
  arr: array[1..10] of integer;  
  i, j: integer;  
begin  
  i := 3;  
  arr[i] := 99;  
  arr[j] := arr[i];  
end.
```

- Output:







=== Symbol Table ===

TAB (Identifier Table)

idx	name	obj	type	ref	nrm	lev	adr	link
1	AND	keyword	Unknown	0	1	0	0	0
2	ARRAY	keyword	Unknown	0	1	0	0	1
3	BEGIN	keyword	Unknown	0	1	0	0	2
4	CASE	keyword	Unknown	0	1	0	0	3
5	CONST	keyword	Unknown	0	1	0	0	4
6	DIV	keyword	Unknown	0	1	0	0	5
7	DOWNT0	keyword	Unknown	0	1	0	0	6
8	DO	keyword	Unknown	0	1	0	0	7
9	ELSE	keyword	Unknown	0	1	0	0	8
10	END	keyword	Unknown	0	1	0	0	9

11	FOR	keyword	Unknown	0	1	0	0	10
12	FUNCTION	keyword	Unknown	0	1	0	0	11
13	IF	keyword	Unknown	0	1	0	0	12
14	MOD	keyword	Unknown	0	1	0	0	13
15	NOT	keyword	Unknown	0	1	0	0	14
16	OF	keyword	Unknown	0	1	0	0	15
17	OR	keyword	Unknown	0	1	0	0	16
18	PROCEDURE	keyword	Unknown	0	1	0	0	17
19	PROGRAM	keyword	Unknown	0	1	0	0	18
20	RECORD	keyword	Unknown	0	1	0	0	19
21	REPEAT	keyword	Unknown	0	1	0	0	20
22	INTEGER	type	Integer	0	1	0	0	21
23	REAL	type	Real	0	1	0	0	22
24	BOOLEAN	type	Boolean	0	1	0	0	23
25	CHAR	type	Char	0	1	0	0	24
26	STRING	type	String	0	1	0	0	25
27	THEN	keyword	Unknown	0	1	0	0	26
28	TO	keyword	Unknown	0	1	0	0	27
29	TYPE	keyword	Unknown	0	1	0	0	28
30	UNTIL	keyword	Unknown	0	1	0	0	29
31	VAR	keyword	Unknown	0	1	0	0	30
32	WHILE	keyword	Unknown	0	1	0	0	31
33	true	constant	Boolean	0	1	0	1	32
34	false	constant	Boolean	0	1	0	0	33
35	writeln	procedure	Void	0	1	0	0	34
36	readln	procedure	Void	0	1	0	0	35
37	ArrayVarIndexTestprogram		Void	1	1	0	0	36
38	arr	variable	Array	0	1	1	0	0
39	i	variable	Integer	0	1	1	10	38
40	j	variable	Integer	0	1	1	11	39

BTAB (Block Table)

idx	last	lpar	psze	vsze
-----	------	------	------	------

0	37	0	0	0
1	40	0	0	12

ATAB (Array Table)

idx	xtyp	etyp	eref	low	high	elsz	size
-----	------	------	------	-----	------	------	------

0	Integer	Integer	0	1	10	1	10
---	---------	---------	---	---	----	---	----

=== Semantic Analysis ===

Errors : 0

Warnings: 0

Status : OK

```

=== Decorated AST ===
ProgramNode(name: 'ArrayVarIndexTest')           → tab_index:37,
type:void, lev:0
├── Declarations
│   ├── VarDecl('arr')      → tab_index:38, type:array, lev:1
│   ├── VarDecl('i')        → tab_index:39, type:integer, lev:1
│   └── VarDecl('j')        → tab_index:40, type:integer, lev:1
├── CompoundStmt
│   ├── Assign(i := 3)      → type:void
│   │   └── Var(i)          → tab_index:39, type:integer, lev:1
│   │       └── Num(3)      → type:integer
│   ├── Assign(arr[i] := 99) → type:void
│   │   ├── ArrayAccess    → type:integer
│   │   │   ├── Var(arr)   → tab_index:38, type:array, lev:1
│   │   │   └── Var(i)     → tab_index:39, type:integer, lev:1
│   │   └── Num(99)        → type:integer
│   ├── Assign(arr[j] := arr[i]) → type:void
│   │   ├── ArrayAccess    → type:integer
│   │   │   ├── Var(arr)   → tab_index:38, type:array, lev:1
│   │   │   └── Var(j)     → tab_index:40, type:integer, lev:1
│   │   └── ArrayAccess    → type:integer
│   │       ├── Var(arr)   → tab_index:38, type:array, lev:1
│   │       └── Var(i)     → tab_index:39, type:integer, lev:1
└── Semantic Analysis ===
    Errors : 0
    Warnings: 0
    Status : OK

```

- Analisis:

Program berhasil dianalisis tanpa error. Array dengan index variable dapat digunakan baik sebagai target assignment (`arr[i] := 99`) maupun sebagai nilai ekspresi (`arr[j] := arr[i]`). Kedua sisi dipetakan ke `ArrayAccessNode` dengan benar.

3.2.25. Kasus Uji 25

- Kasus: Operator Left-Associative
- Tujuan: Memverifikasi bahwa operator aritmatika `+`, `-`, `*` bersifat left-associative sehingga `a + b + c` dipetakan menjadi `BinOp(+, BinOp(+, a, b), c)` bukan `BinOp(+, a, BinOp(+, b, c))`

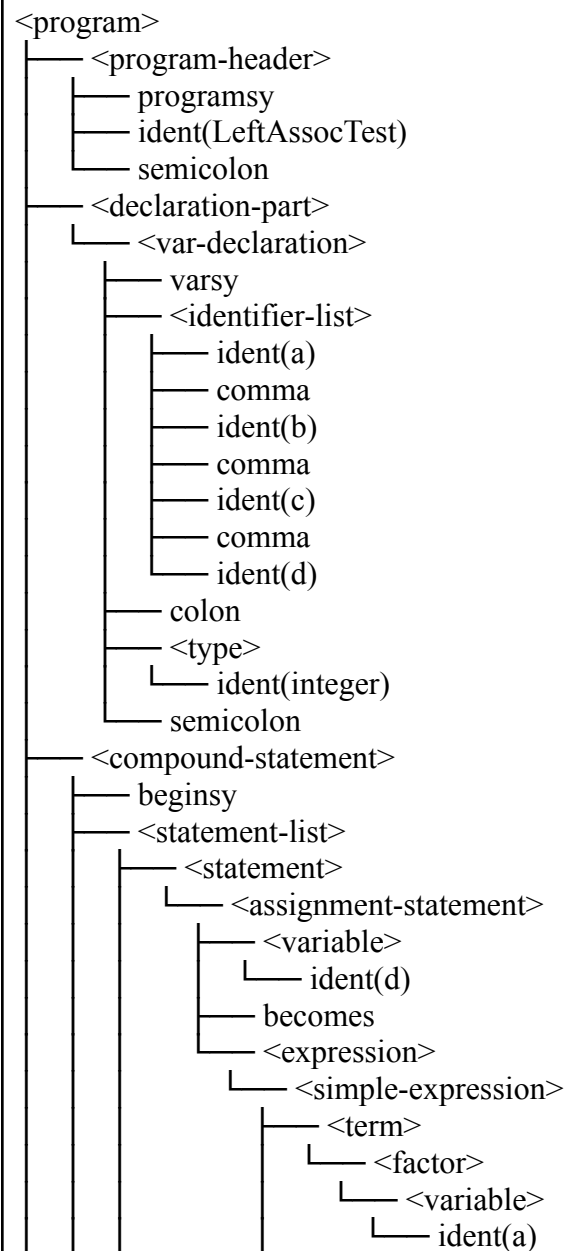
- Input:

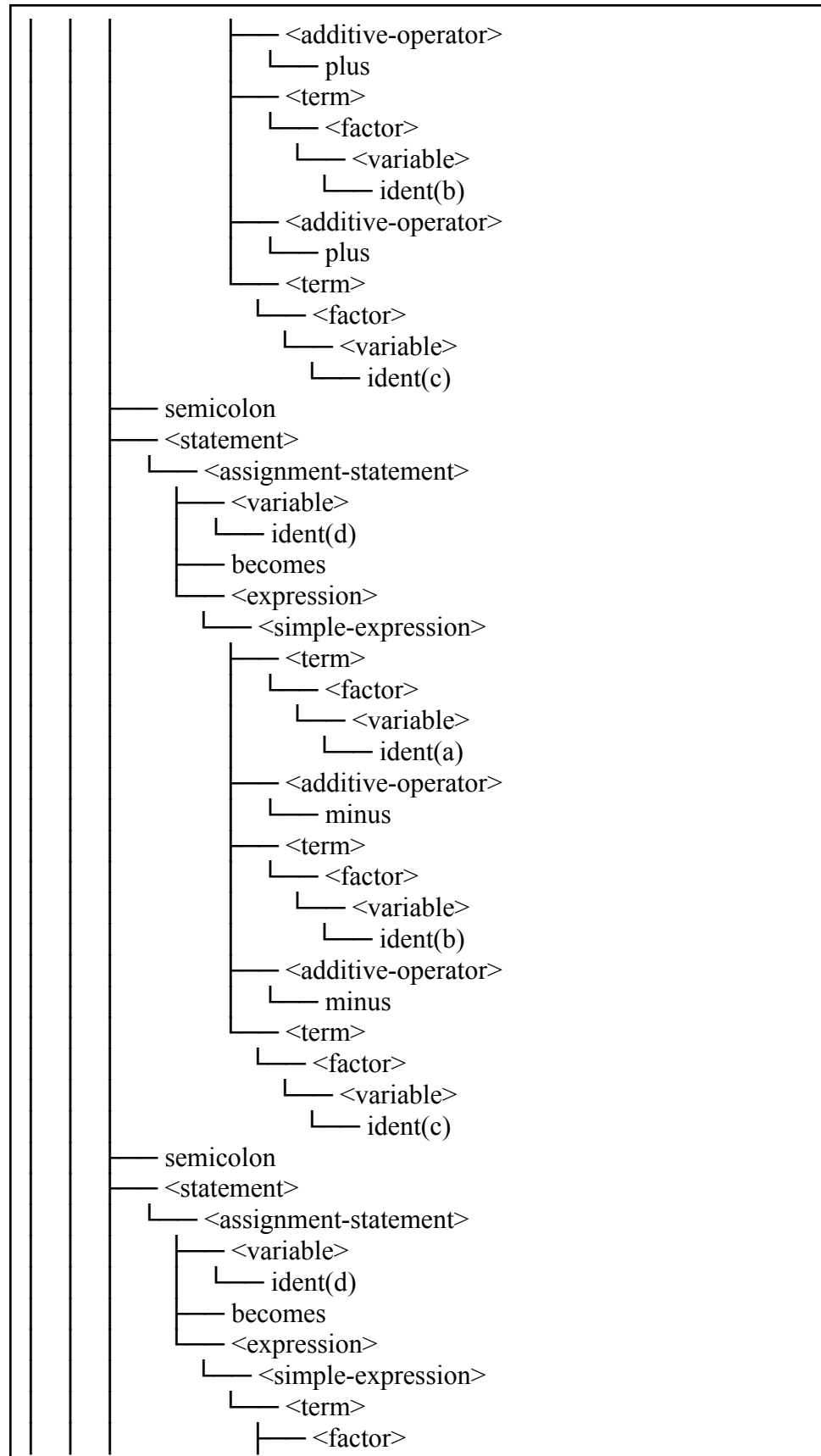
```

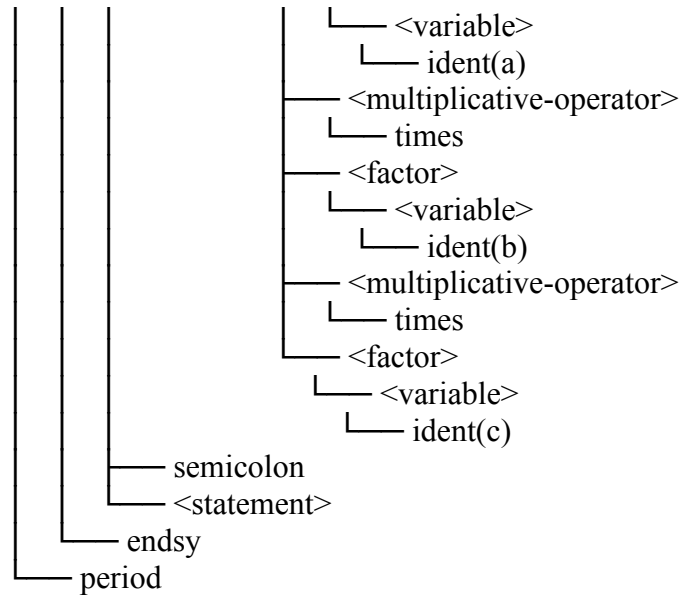
program LeftAssocTest;
var
  a, b, c, d: integer;
begin
  d := a + b + c;
  d := a - b - c;
  d := a * b * c;
end.

```

- Output:







=== Symbol Table ===

TAB (Identifier Table)

idx	name	obj	type	ref	nrm	lev	adr	link

1	AND	keyword	Unknown	0	1	0	0	0
2	ARRAY	keyword	Unknown	0	1	0	0	1
3	BEGIN	keyword	Unknown	0	1	0	0	2
4	CASE	keyword	Unknown	0	1	0	0	3
5	CONST	keyword	Unknown	0	1	0	0	4
6	DIV	keyword	Unknown	0	1	0	0	5
7	DOWNTO	keyword	Unknown	0	1	0	0	6
8	DO	keyword	Unknown	0	1	0	0	7
9	ELSE	keyword	Unknown	0	1	0	0	8
10	END	keyword	Unknown	0	1	0	0	9
11	FOR	keyword	Unknown	0	1	0	0	10
12	FUNCTION	keyword	Unknown	0	1	0	0	11
13	IF	keyword	Unknown	0	1	0	0	12
14	MOD	keyword	Unknown	0	1	0	0	13
15	NOT	keyword	Unknown	0	1	0	0	14
16	OF	keyword	Unknown	0	1	0	0	15
17	OR	keyword	Unknown	0	1	0	0	16
18	PROCEDURE	keyword	Unknown	0	1	0	0	17
19	PROGRAM	keyword	Unknown	0	1	0	0	18
20	RECORD	keyword	Unknown	0	1	0	0	19
21	REPEAT	keyword	Unknown	0	1	0	0	20
22	INTEGER	type	Integer	0	1	0	0	21
23	REAL	type	Real	0	1	0	0	22
24	BOOLEAN	type	Boolean	0	1	0	0	23

25	CHAR	type	Char	0	1	0	0	24
26	STRING	type	String	0	1	0	0	25
27	THEN	keyword	Unknown	0	1	0	0	26
28	TO	keyword	Unknown	0	1	0	0	27
29	TYPE	keyword	Unknown	0	1	0	0	28
30	UNTIL	keyword	Unknown	0	1	0	0	29
31	VAR	keyword	Unknown	0	1	0	0	30
32	WHILE	keyword	Unknown	0	1	0	0	31
33	true	constant	Boolean	0	1	0	1	32
34	false	constant	Boolean	0	1	0	0	33
35	writeln	procedure	Void	0	1	0	0	34
36	readln	procedure	Void	0	1	0	0	35
37	LeftAssocTest	program	Void	1	1	0	0	36
38	a	variable	Integer	0	1	1	0	0
39	b	variable	Integer	0	1	1	1	38
40	c	variable	Integer	0	1	1	2	39
41	d	variable	Integer	0	1	1	3	40

BTAB (Block Table)

idx	last	lpar	psze	vsze
-----	------	------	------	------

0	37	0	0	0
1	41	0	0	4

ATAB (Array Table)

(kosong)

=== Semantic Analysis ===

Errors : 0

Warnings: 0

Status : OK

=== Decorated AST ===

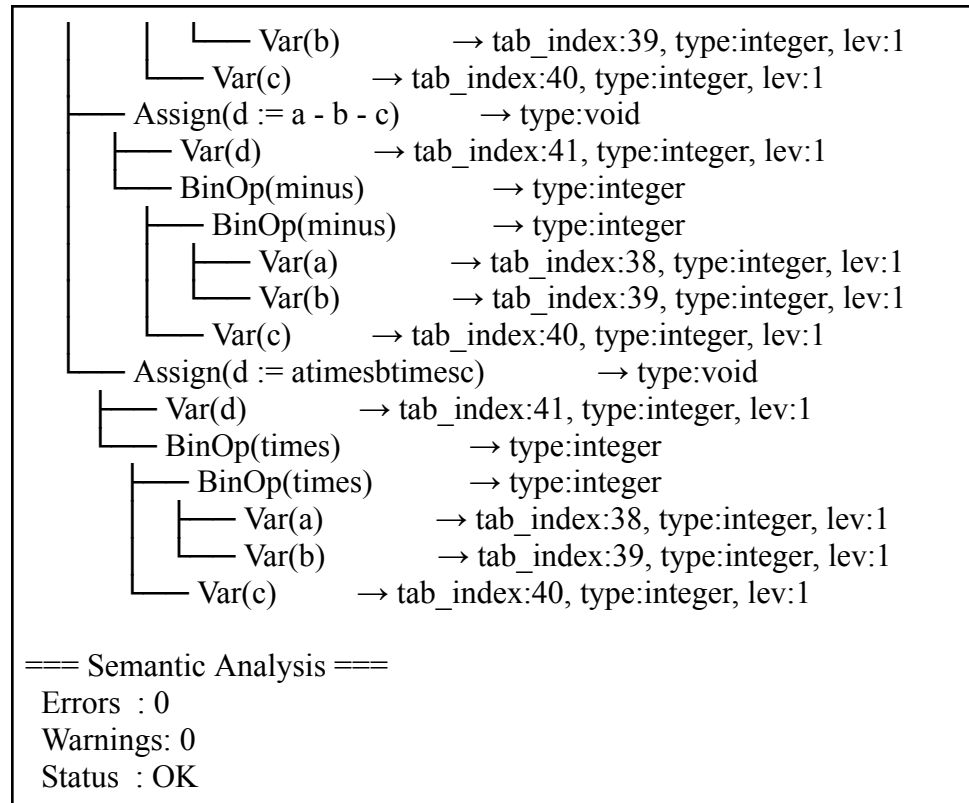
ProgramNode(name: 'LeftAssocTest') → tab_index:37,

type:void, lev:0

```

├── Declarations
│   ├── VarDecl('a') → tab_index:38, type:integer, lev:1
│   ├── VarDecl('b') → tab_index:39, type:integer, lev:1
│   ├── VarDecl('c') → tab_index:40, type:integer, lev:1
│   └── VarDecl('d') → tab_index:41, type:integer, lev:1
├── CompoundStmt → type:void
│   ├── Assign(d := a + b + c) → type:void
│   │   ├── Var(d) → tab_index:41, type:integer, lev:1
│   │   └── BinOp(plus) → type:integer
│   │       ├── BinOp(plus) → type:integer
│   │       │   ├── Var(a) → tab_index:38, type:integer, lev:1

```



- Analisis:

Program berhasil dianalisis tanpa error. Operator +, -, * terbukti bersifat left-associative karena $a + b + c$ dipetakan menjadi $\text{BinOp}(\text{plus}, \text{BinOp}(\text{plus}, a, b), c)$, bukan $\text{BinOp}(\text{plus}, a, \text{BinOp}(\text{plus}, b, c))$. Terdapat minor issue pada label Assign untuk perkalian yang muncul sebagai `atimesbtimesc` karena masalah formatting string label, tidak mempengaruhi struktur AST.

3.2.26. Kasus Uji 26

- Kasus: Parenthesized Expression Dihapus
- Tujuan: Memverifikasi bahwa tanda kurung dalam ekspresi dibuang saat konversi ke AST namun struktur ekspresi tetap benar sesuai prioritas operator
- Input:

```

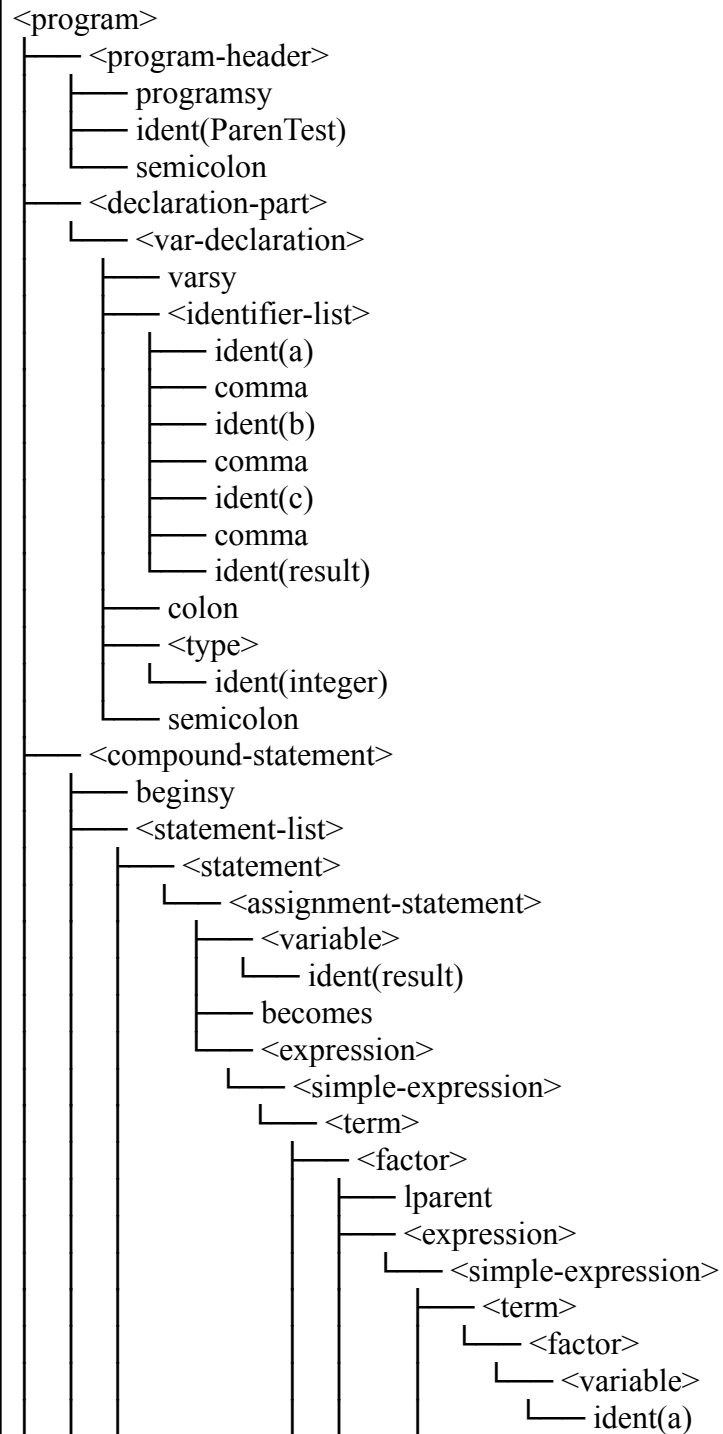
program ParenTest;
var
  a, b, c, result: integer;
  
```

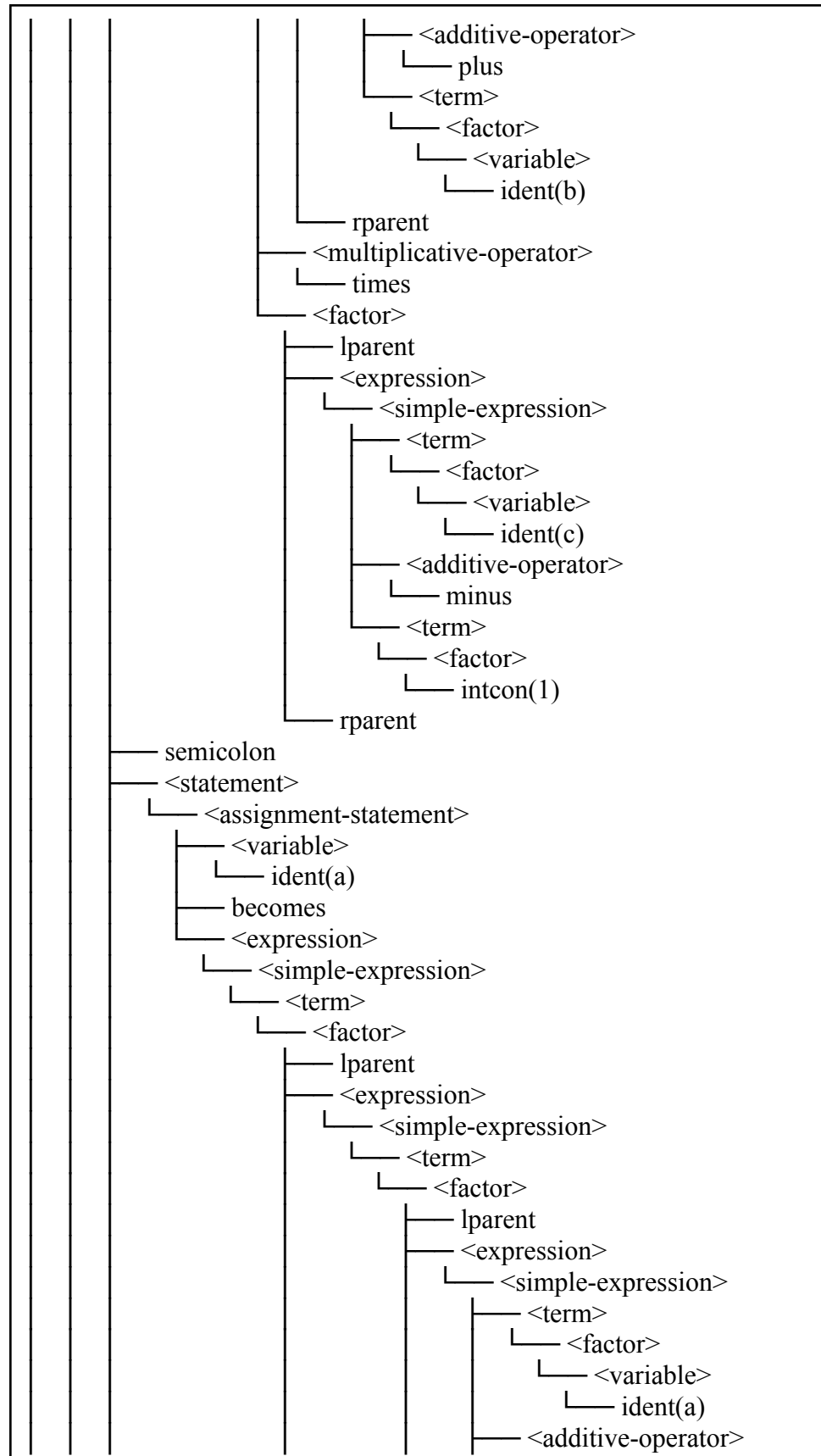
```

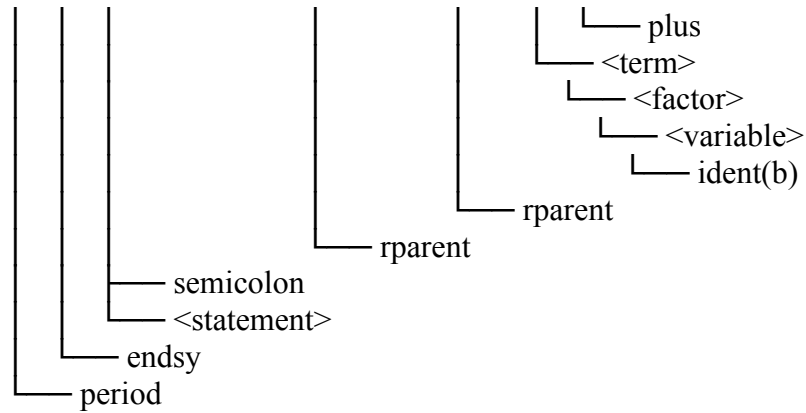
begin
  result := (a + b) * (c - 1);
  a := ((a + b));
end.

```

- Output:







=== Symbol Table ===

TAB (Identifier Table)

idx	name	obj	type	ref	nrm	lev	adr	link
1	AND	keyword	Unknown	0	1	0	0	0
2	ARRAY	keyword	Unknown	0	1	0	0	1
3	BEGIN	keyword	Unknown	0	1	0	0	2
4	CASE	keyword	Unknown	0	1	0	0	3
5	CONST	keyword	Unknown	0	1	0	0	4
6	DIV	keyword	Unknown	0	1	0	0	5
7	DOWNT0	keyword	Unknown	0	1	0	0	6
8	DO	keyword	Unknown	0	1	0	0	7
9	ELSE	keyword	Unknown	0	1	0	0	8
10	END	keyword	Unknown	0	1	0	0	9
11	FOR	keyword	Unknown	0	1	0	0	10
12	FUNCTION	keyword	Unknown	0	1	0	0	11
13	IF	keyword	Unknown	0	1	0	0	12
14	MOD	keyword	Unknown	0	1	0	0	13
15	NOT	keyword	Unknown	0	1	0	0	14
16	OF	keyword	Unknown	0	1	0	0	15
17	OR	keyword	Unknown	0	1	0	0	16
18	PROCEDURE	keyword	Unknown	0	1	0	0	17
19	PROGRAM	keyword	Unknown	0	1	0	0	18
20	RECORD	keyword	Unknown	0	1	0	0	19
21	REPEAT	keyword	Unknown	0	1	0	0	20
22	INTEGER	type	Integer	0	1	0	0	21
23	REAL	type	Real	0	1	0	0	22
24	BOOLEAN	type	Boolean	0	1	0	0	23
25	CHAR	type	Char	0	1	0	0	24
26	STRING	type	String	0	1	0	0	25
27	THEN	keyword	Unknown	0	1	0	0	26
28	TO	keyword	Unknown	0	1	0	0	27
29	TYPE	keyword	Unknown	0	1	0	0	28

30	UNTIL	keyword	Unknown	0	1	0	0	29
31	VAR	keyword	Unknown	0	1	0	0	30
32	WHILE	keyword	Unknown	0	1	0	0	31
33	true	constant	Boolean	0	1	0	1	32
34	false	constant	Boolean	0	1	0	0	33
35	writeln	procedure	Void	0	1	0	0	34
36	readln	procedure	Void	0	1	0	0	35
37	ParenTest	program	Void	1	1	0	0	36
38	a	variable	Integer	0	1	1	0	0
39	b	variable	Integer	0	1	1	1	38
40	c	variable	Integer	0	1	1	2	39
41	result	variable	Integer	0	1	1	3	40

BTAB (Block Table)

idx	last	lpar	psze	vsze
-----	------	------	------	------

0	37	0	0	0
1	41	0	0	4

ATAB (Array Table)

(kosong)

=== Semantic Analysis ===

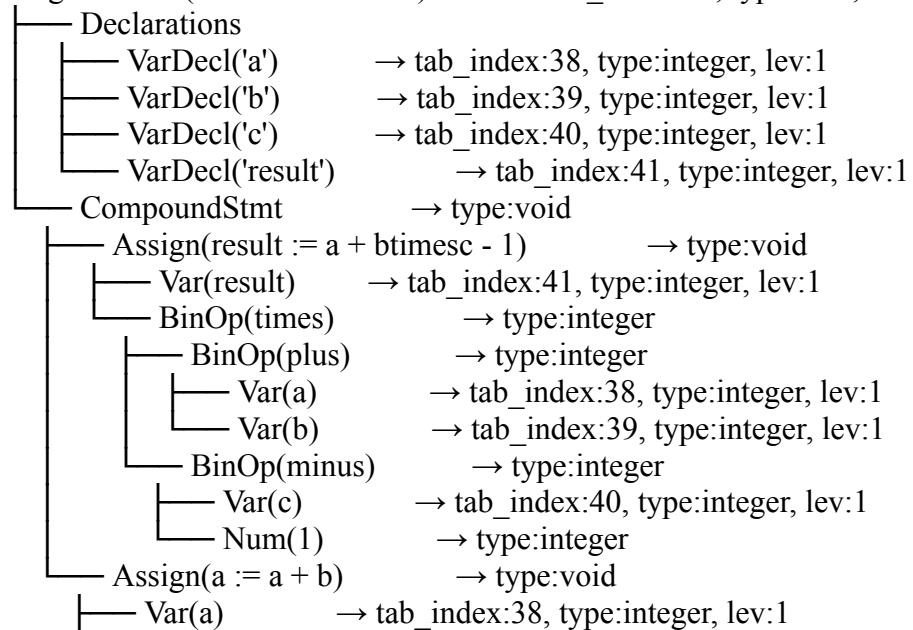
Errors : 0

Warnings: 0

Status : OK

=== Decorated AST ===

ProgramNode(name: 'ParenTest') → tab_index:37, type:void, lev:0



```

└─ BinOp(plus)    → type:integer
   └─ Var(a)      → tab_index:38, type:integer, lev:1
      └─ Var(b)    → tab_index:39, type:integer, lev:1

```

=== Semantic Analysis ===

Errors : 0
Warnings: 0
Status : OK

- Analisis:

Program berhasil dianalisis tanpa error. Tanda kurung berhasil dibuang dari AST — $(a + b) * (c - 1)$ dipetakan menjadi BinOp(times, BinOp(plus, a, b), BinOp(minus, c, 1)) tanpa node kurung. Kurung ganda $((a + b))$ juga dibuang dengan benar menghasilkan BinOp(plus, a, b).

3.2.27. Kasus Uji 27

- Kasus: For Downto
- Tujuan: Memverifikasi bahwa perulangan for downto dipetakan menjadi ForNode dengan isDownto = true
- Input:

```

program ForDowntoTest;
var
  i, sum: integer;
begin
  sum := 0;
  for i := 10 downto 1 do
  begin
    sum := sum + i;
  end;
end.

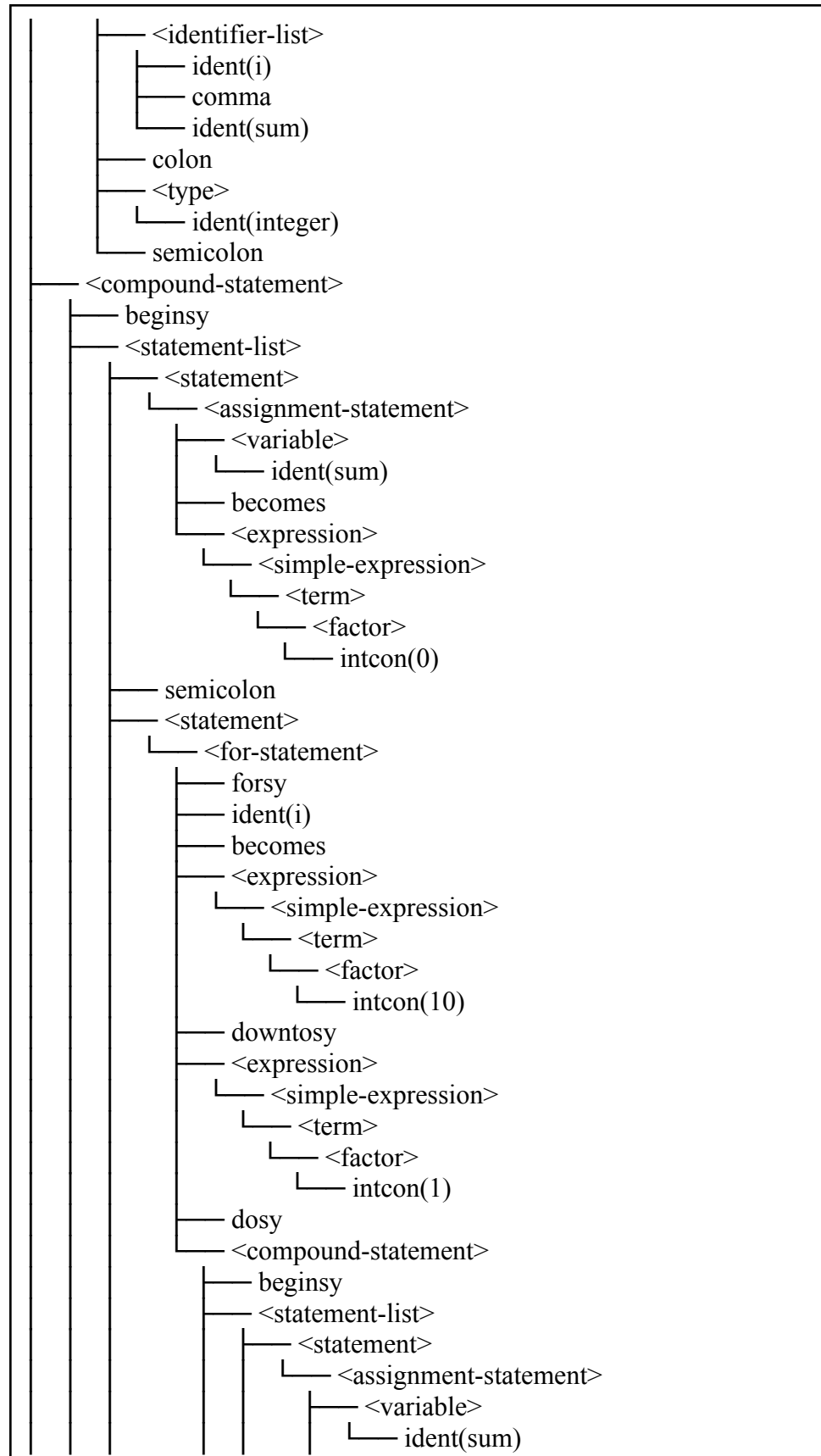
```

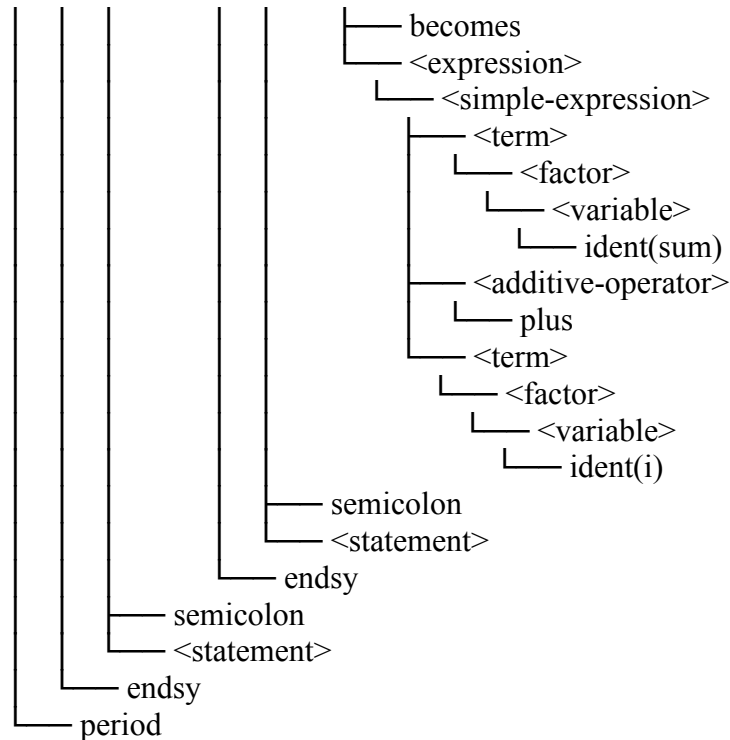
- Output:

```

<program>
└─ <program-header>
   └─ programsy
      └─ ident(ForDowntoTest)
         └─ semicolon
└─ <declaration-part>
   └─ <var-declaration>
      └─ varsy

```



=== Symbol Table ===

TAB (Identifier Table)

idx	name	obj	type	ref	nrm	lev	adr	link
1	AND	keyword	Unknown	0	1	0	0	0
2	ARRAY	keyword	Unknown	0	1	0	0	1
3	BEGIN	keyword	Unknown	0	1	0	0	2
4	CASE	keyword	Unknown	0	1	0	0	3
5	CONST	keyword	Unknown	0	1	0	0	4
6	DIV	keyword	Unknown	0	1	0	0	5
7	DOWNTO	keyword	Unknown	0	1	0	0	6
8	DO	keyword	Unknown	0	1	0	0	7
9	ELSE	keyword	Unknown	0	1	0	0	8
10	END	keyword	Unknown	0	1	0	0	9
11	FOR	keyword	Unknown	0	1	0	0	10
12	FUNCTION	keyword	Unknown	0	1	0	0	11
13	IF	keyword	Unknown	0	1	0	0	12
14	MOD	keyword	Unknown	0	1	0	0	13
15	NOT	keyword	Unknown	0	1	0	0	14
16	OF	keyword	Unknown	0	1	0	0	15
17	OR	keyword	Unknown	0	1	0	0	16
18	PROCEDURE	keyword	Unknown	0	1	0	0	17
19	PROGRAM	keyword	Unknown	0	1	0	0	18
20	RECORD	keyword	Unknown	0	1	0	0	19

21	REPEAT	keyword	Unknown	0	1	0	0	20
22	INTEGER	type	Integer	0	1	0	0	21
23	REAL	type	Real	0	1	0	0	22
24	BOOLEAN	type	Boolean	0	1	0	0	23
25	CHAR	type	Char	0	1	0	0	24
26	STRING	type	String	0	1	0	0	25
27	THEN	keyword	Unknown	0	1	0	0	26
28	TO	keyword	Unknown	0	1	0	0	27
29	TYPE	keyword	Unknown	0	1	0	0	28
30	UNTIL	keyword	Unknown	0	1	0	0	29
31	VAR	keyword	Unknown	0	1	0	0	30
32	WHILE	keyword	Unknown	0	1	0	0	31
33	true	constant	Boolean	0	1	0	1	32
34	false	constant	Boolean	0	1	0	0	33
35	writeln	procedure	Void	0	1	0	0	34
36	readln	procedure	Void	0	1	0	0	35
37	ForDowntoTest	program	Void	1	1	0	0	36
38	i	variable	Integer	0	1	1	0	0
39	sum	variable	Integer	0	1	1	1	38

BTAB (Block Table)

idx	last	lpar	psze	vsze

0	37	0	0	0
1	39	0	0	2

ATAB (Array Table)

(kosong)

=== Semantic Analysis ===

Errors : 0

Warnings: 0

Status : OK

=== Decorated AST ===

ProgramNode(name: 'ForDowntoTest') → tab_index:37,

type:void, lev:0

└─ Declarations

└─ VarDecl('i')

→ tab_index:38, type:integer, lev:1

└─ VarDecl('sum')

→ tab_index:39, type:integer, lev:1

└─ CompoundStmt

→ type:void

└─ Assign(sum := 0)

→ type:void

└─ Var(sum)

→ tab_index:39, type:integer, lev:1

└─ Num(0)

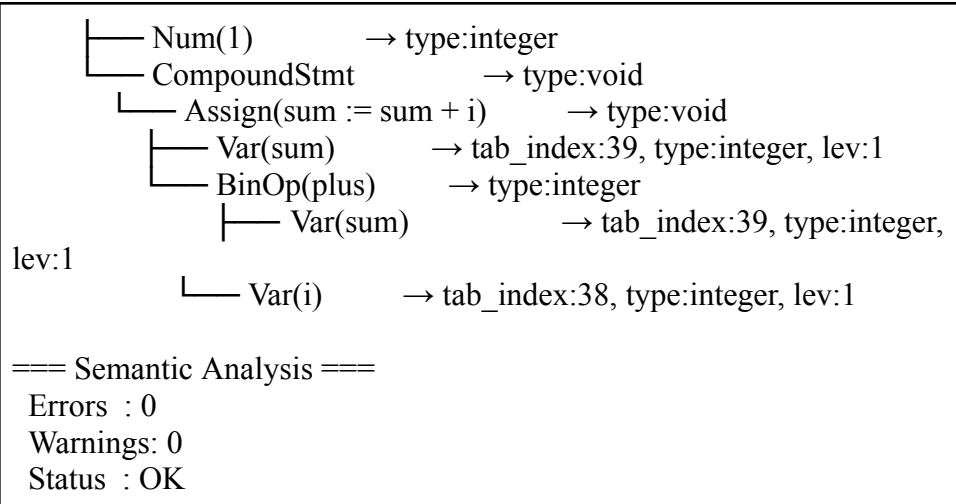
→ type:integer

└─ For(i, downto)

→ type:void

└─ Num(10)

→ type:integer



- Analisis:

Program berhasil dianalisis tanpa error. Perulangan for i := 10 downto 1 dipetakan menjadi For(i, downto) dengan isDownto = true. From expression Num(10) dan to expression Num(1) menjadi child pertama dan kedua ForNode. Body compound statement terpetakan dengan benar.

3.2.28. Kasus Uji 28

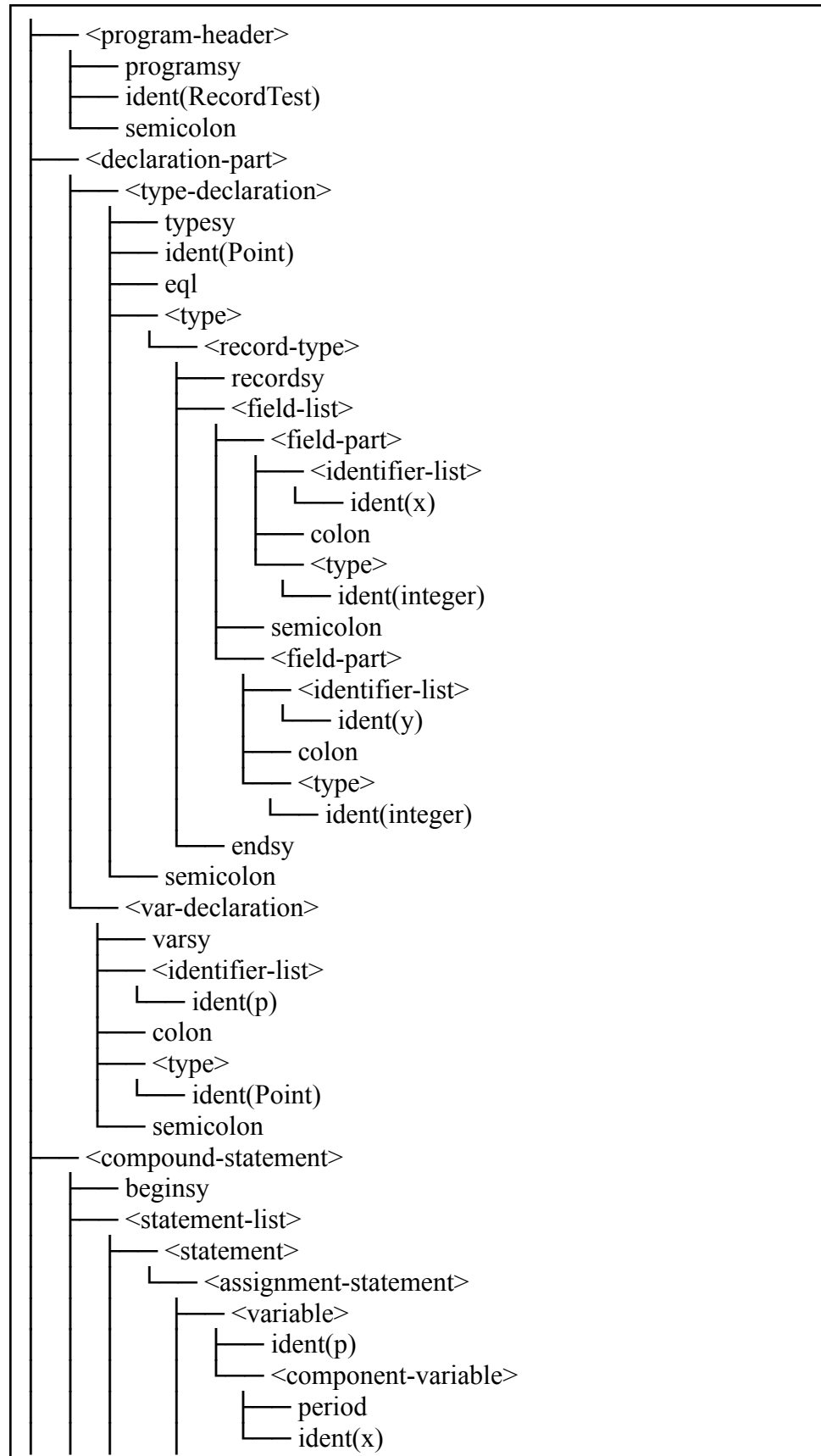
- Kasus: Record Field Access
- Tujuan: Memverifikasi bahwa deklarasi record dan akses field dengan operator titik dipetakan menjadi FieldAccessNode dengan record sebagai child-nya
- Input:

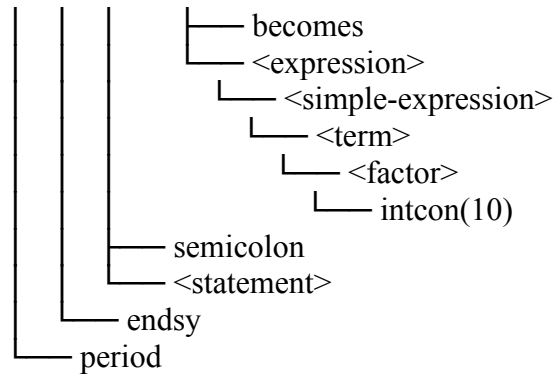
```

program RecordTest;
type
  Point == record
    x: integer;
    y: integer
  end;
var
  p: Point;
begin
  p.x := 10;
end.

```

- Output:





=== Symbol Table ===

TAB (Identifier Table)

idx	name	obj	type	ref	nrm	lev	adr	link

1	AND	keyword	Unknown	0	1	0	0	0
2	ARRAY	keyword	Unknown	0	1	0	0	1
3	BEGIN	keyword	Unknown	0	1	0	0	2
4	CASE	keyword	Unknown	0	1	0	0	3
5	CONST	keyword	Unknown	0	1	0	0	4
6	DIV	keyword	Unknown	0	1	0	0	5
7	DOWNT0	keyword	Unknown	0	1	0	0	6
8	DO	keyword	Unknown	0	1	0	0	7
9	ELSE	keyword	Unknown	0	1	0	0	8
10	END	keyword	Unknown	0	1	0	0	9
11	FOR	keyword	Unknown	0	1	0	0	10
12	FUNCTION	keyword	Unknown	0	1	0	0	11
13	IF	keyword	Unknown	0	1	0	0	12
14	MOD	keyword	Unknown	0	1	0	0	13
15	NOT	keyword	Unknown	0	1	0	0	14
16	OF	keyword	Unknown	0	1	0	0	15
17	OR	keyword	Unknown	0	1	0	0	16
18	PROCEDURE	keyword	Unknown	0	1	0	0	17
19	PROGRAM	keyword	Unknown	0	1	0	0	18
20	RECORD	keyword	Unknown	0	1	0	0	19
21	REPEAT	keyword	Unknown	0	1	0	0	20
22	INTEGER	type	Integer	0	1	0	0	21
23	REAL	type	Real	0	1	0	0	22
24	BOOLEAN	type	Boolean	0	1	0	0	23
25	CHAR	type	Char	0	1	0	0	24
26	STRING	type	String	0	1	0	0	25
27	THEN	keyword	Unknown	0	1	0	0	26
28	TO	keyword	Unknown	0	1	0	0	27
29	TYPE	keyword	Unknown	0	1	0	0	28
30	UNTIL	keyword	Unknown	0	1	0	0	29

```

31 VAR      keyword  Unknown  0  1  0  0  30
32 WHILE    keyword  Unknown  0  1  0  0  31
33 true     constant Boolean  0  1  0  1  32
34 false    constant Boolean  0  1  0  0  33
35 writeln  procedure Void    0  1  0  0  34
36 readln   procedure Void    0  1  0  0  35
37 RecordTest program Void    1  1  0  0  36
38 Point     type     Unknown  0  1  1  0  0
39 p         variable Unknown  0  1  1  0  38

```

BTAB (Block Table)

idx	last	lpar	psze	vsze
0	37	0	0	0
1	39	0	0	1

ATAB (Array Table)
(kosong)

=== Semantic Analysis ===

Errors : 0
Warnings: 0
Status : OK

=== Decorated AST ===

```

ProgramNode(name: 'RecordTest')      → tab_index:37, type:void,
lev:0
├── Declarations
│   ├── TypeDecl(Point)               → tab_index:38, type:unknown, lev:1
│   └── VarDecl('p')                  → tab_index:39, type:unknown, lev:1
├── CompoundStmt                      → type:void
│   ├── Assign(p.x := 10)              → type:void
│   └── FieldAccess(x)                 → type:unknown
│       ├── Var(p)                     → type:unknown
│       └── Num(10)                    → type:integer

```

=== Semantic Analysis ===

Errors : 0
Warnings: 0
Status : OK

- Analisis:

Program berhasil dianalisis tanpa error. Deklarasi record Point terdaftar di tab[38] sebagai TypeDecl. Akses field p.x dipetakan menjadi FieldAccess(x) dengan Var(p) sebagai child-nya. Tipe Point dan field x

muncul sebagai type:unknown karena resolusi tipe record belum sepenuhnya diimplementasikan di semantic analyzer, namun struktur AST sudah benar.

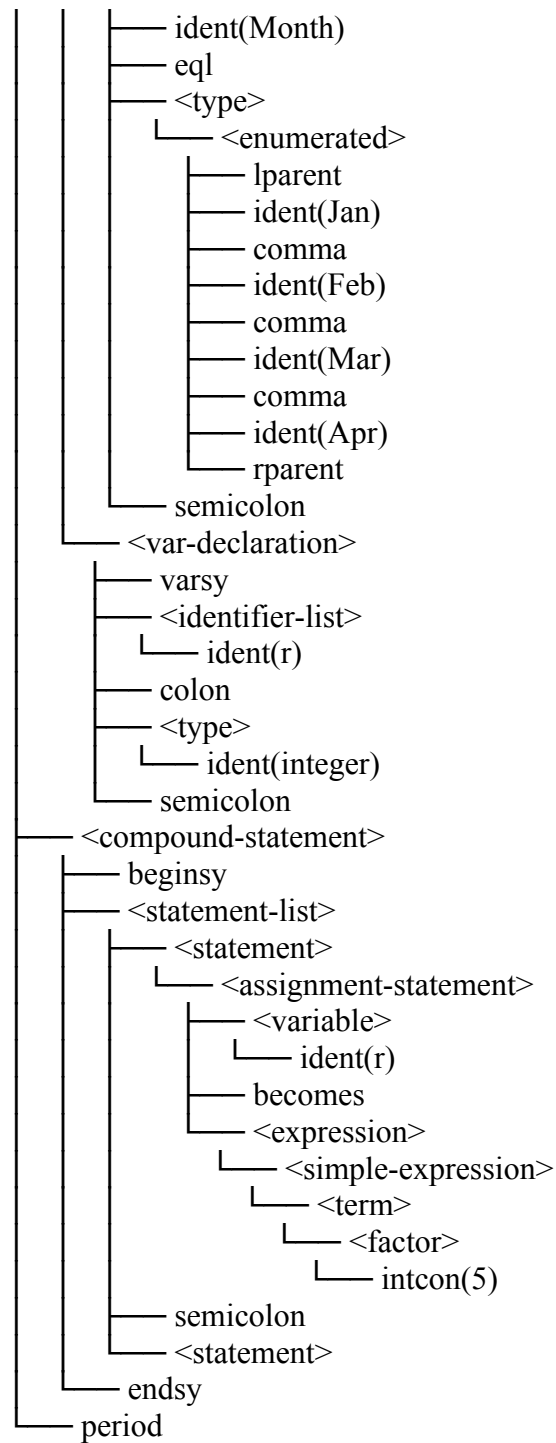
3.2.29. Kasus Uji 29

- Kasus: Type Declaration (Subrange dan Enumerated)
- Tujuan: Memverifikasi bahwa deklarasi tipe baru berupa subrange dan enumerated dipetakan menjadi TypeDeclNode dengan DataType yang sesuai
- Input:

```
program TypeDeclTest;  
type  
  Range == 1..10;  
  Month == (Jan, Feb, Mar, Apr);  
var  
  r: integer;  
begin  
  r := 5;  
end.
```

- Output:

```
<program>  
├── <program-header>  
│   ├── programsy  
│   ├── ident(TypeDeclTest)  
│   └── semicolon  
├── <declaration-part>  
│   ├── <type-declaration>  
│   │   ├── typesy  
│   │   ├── ident(Range)  
│   │   ├── eql  
│   │   ├── <type>  
│   │   │   └── <range>  
│   │   │       ├── <constant>  
│   │   │       │   └── intcon(1)  
│   │   │       ├── period  
│   │   │       ├── period  
│   │   │       ├── <constant>  
│   │   │       │   └── intcon(10)  
│   │   └── semicolon  
└── <declaration-part>
```

=== Symbol Table ===

TAB (Identifier Table)

idx	name	obj	type	ref	nrm	lev	adr	link
1	AND		keyword	Unknown	0	1	0	0

2	ARRAY	keyword	Unknown	0	1	0	0	1
3	BEGIN	keyword	Unknown	0	1	0	0	2
4	CASE	keyword	Unknown	0	1	0	0	3
5	CONST	keyword	Unknown	0	1	0	0	4
6	DIV	keyword	Unknown	0	1	0	0	5
7	DOWNT0	keyword	Unknown	0	1	0	0	6
8	DO	keyword	Unknown	0	1	0	0	7
9	ELSE	keyword	Unknown	0	1	0	0	8
10	END	keyword	Unknown	0	1	0	0	9
11	FOR	keyword	Unknown	0	1	0	0	10
12	FUNCTION	keyword	Unknown	0	1	0	0	11
13	IF	keyword	Unknown	0	1	0	0	12
14	MOD	keyword	Unknown	0	1	0	0	13
15	NOT	keyword	Unknown	0	1	0	0	14
16	OF	keyword	Unknown	0	1	0	0	15
17	OR	keyword	Unknown	0	1	0	0	16
18	PROCEDURE	keyword	Unknown	0	1	0	0	17
19	PROGRAM	keyword	Unknown	0	1	0	0	18
20	RECORD	keyword	Unknown	0	1	0	0	19
21	REPEAT	keyword	Unknown	0	1	0	0	20
22	INTEGER	type	Integer	0	1	0	0	21
23	REAL	type	Real	0	1	0	0	22
24	BOOLEAN	type	Boolean	0	1	0	0	23
25	CHAR	type	Char	0	1	0	0	24
26	STRING	type	String	0	1	0	0	25
27	THEN	keyword	Unknown	0	1	0	0	26
28	TO	keyword	Unknown	0	1	0	0	27
29	TYPE	keyword	Unknown	0	1	0	0	28
30	UNTIL	keyword	Unknown	0	1	0	0	29
31	VAR	keyword	Unknown	0	1	0	0	30
32	WHILE	keyword	Unknown	0	1	0	0	31
33	true	constant	Boolean	0	1	0	1	32
34	false	constant	Boolean	0	1	0	0	33
35	writeln	procedure	Void	0	1	0	0	34
36	readln	procedure	Void	0	1	0	0	35
37	TypeDeclTest	program	Void	1	1	0	0	36
38	Range	type	Unknown	0	1	1	0	0
39	Month	type	Unknown	0	1	1	0	38
40	r	variable	Integer	0	1	1	0	39

BTAB (Block Table)

idx last lpar psze vsze

```
-----
0 37 0 0 0
1 40 0 0 1
```

ATAB (Array Table)
(kosong)

=== Semantic Analysis ===

Errors : 0

Warnings: 0

Status : OK

=== Decorated AST ===

ProgramNode(name: 'TypeDeclTest') → tab_index:37,
type:void, lev:0

├── Declarations

│ ├── TypeDecl(Range) → tab_index:38, type:unknown, lev:1

│ ├── TypeDecl(Month) → tab_index:39, type:unknown, lev:1

│ └── VarDecl('r') → tab_index:40, type:integer, lev:1

└── CompoundStmt → type:void

├── Assign(r := 5) → type:void

│ ├── Var(r) → tab_index:40, type:integer, lev:1

│ └── Num(5) → type:integer

=== Semantic Analysis ===

Errors : 0

Warnings: 0

Status : OK

- Analisis:

Program berhasil dianalisis tanpa error. Deklarasi tipe Range dan Month masing-masing terdaftar di tab[38] dan tab[39] sebagai TypeDecl. Tipe keduanya muncul type:unknown karena resolusi subrange dan enumerated belum sepenuhnya diimplementasikan di semantic analyzer, namun struktur AST dan pendaftaran ke symbol table sudah benar.

3.2.30. Kasus Uji 30

- Kasus: Array dengan Field Access Terpisah
- Tujuan: Memverifikasi bahwa deklarasi array dan record dalam satu program dapat dipetakan secara bersamaan tanpa konflik pada AST
- Input:

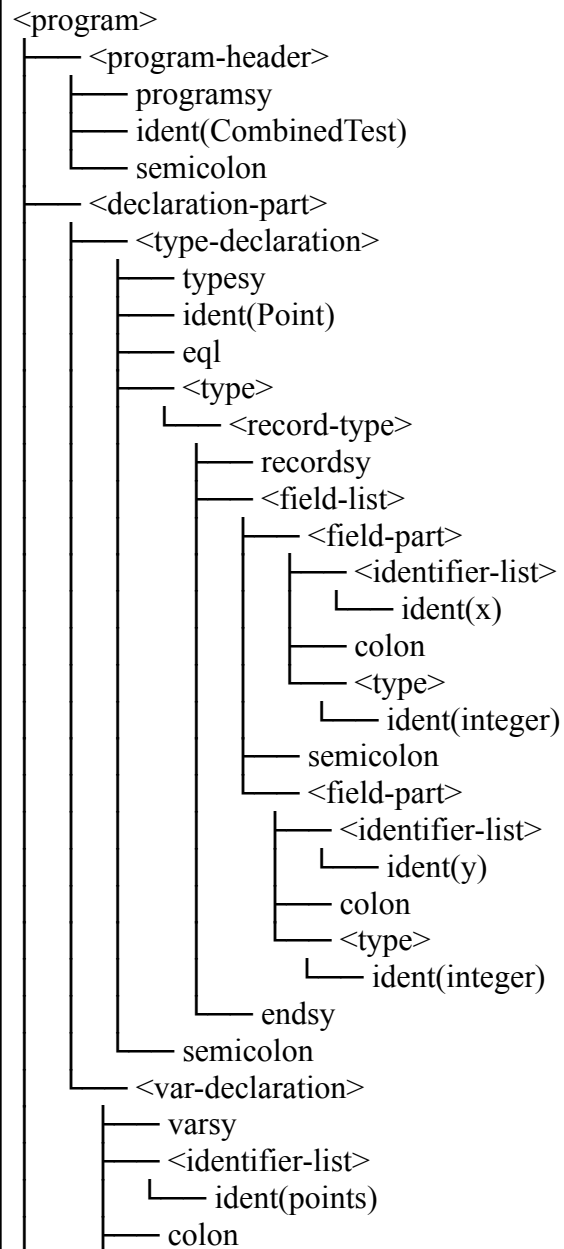
```
program CombinedTest;  
type  
  Point == record
```

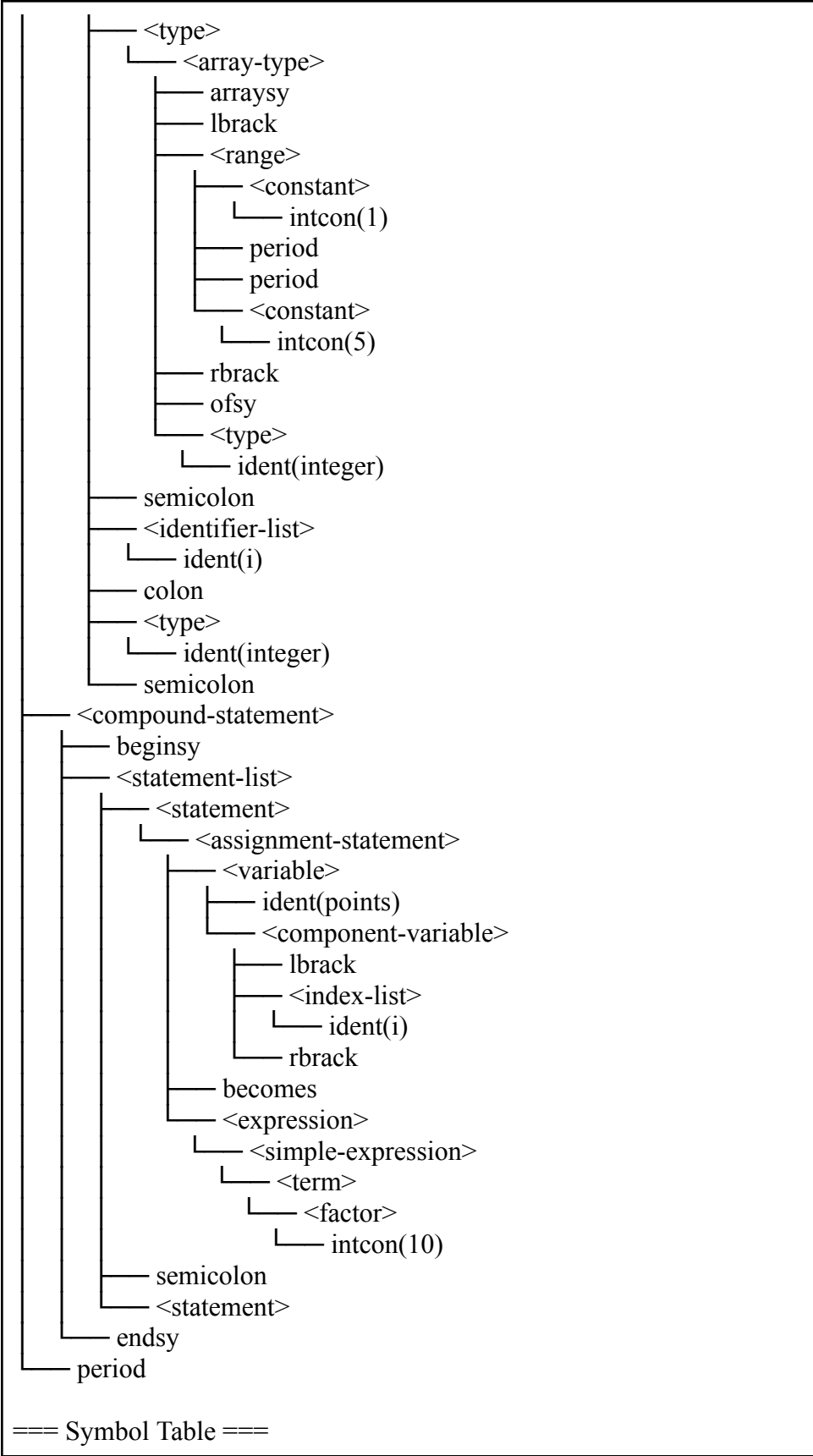
```

    x: integer;
    y: integer;
end;
var
    points: array[1..5] of integer;
    i: integer;
begin
    points[i] := 10;
end.

```

- Output:





TAB (Identifier Table)

idx	name	obj	type	ref	nrm	lev	adr	link
1	AND	keyword	Unknown	0	1	0	0	0
2	ARRAY	keyword	Unknown	0	1	0	0	1
3	BEGIN	keyword	Unknown	0	1	0	0	2
4	CASE	keyword	Unknown	0	1	0	0	3
5	CONST	keyword	Unknown	0	1	0	0	4
6	DIV	keyword	Unknown	0	1	0	0	5
7	DOWNT0	keyword	Unknown	0	1	0	0	6
8	DO	keyword	Unknown	0	1	0	0	7
9	ELSE	keyword	Unknown	0	1	0	0	8
10	END	keyword	Unknown	0	1	0	0	9
11	FOR	keyword	Unknown	0	1	0	0	10
12	FUNCTION	keyword	Unknown	0	1	0	0	11
13	IF	keyword	Unknown	0	1	0	0	12
14	MOD	keyword	Unknown	0	1	0	0	13
15	NOT	keyword	Unknown	0	1	0	0	14
16	OF	keyword	Unknown	0	1	0	0	15
17	OR	keyword	Unknown	0	1	0	0	16
18	PROCEDURE	keyword	Unknown	0	1	0	0	17
19	PROGRAM	keyword	Unknown	0	1	0	0	18
20	RECORD	keyword	Unknown	0	1	0	0	19
21	REPEAT	keyword	Unknown	0	1	0	0	20
22	INTEGER	type	Integer	0	1	0	0	21
23	REAL	type	Real	0	1	0	0	22
24	BOOLEAN	type	Boolean	0	1	0	0	23
25	CHAR	type	Char	0	1	0	0	24
26	STRING	type	String	0	1	0	0	25
27	THEN	keyword	Unknown	0	1	0	0	26
28	TO	keyword	Unknown	0	1	0	0	27
29	TYPE	keyword	Unknown	0	1	0	0	28
30	UNTIL	keyword	Unknown	0	1	0	0	29
31	VAR	keyword	Unknown	0	1	0	0	30
32	WHILE	keyword	Unknown	0	1	0	0	31
33	true	constant	Boolean	0	1	0	1	32
34	false	constant	Boolean	0	1	0	0	33
35	writeln	procedure	Void	0	1	0	0	34
36	readln	procedure	Void	0	1	0	0	35
37	CombinedTest	program	Void	1	1	0	0	36
38	Point	type	Unknown	0	1	1	0	0
39	points	variable	Array	0	1	1	0	38
40	i	variable	Integer	0	1	1	5	39

BTAB (Block Table)

idx	last	lpar	psze	vsze
0	37	0	0	0
1	40	0	0	6

0	37	0	0	0
1	40	0	0	6

ATAB (Array Table)

idx	xtyp	etyp	eref	low	high	elsz	size
0	Integer	Integer	0	1	5	1	5

=== Semantic Analysis ===

Errors : 0
Warnings: 0
Status : OK

=== Decorated AST ===

ProgramNode(name: 'CombinedTest') → tab_index:37,
type:void, lev:0

```

  └─ Declarations
    └─ TypeDecl(Point) → tab_index:38, type:unknown, lev:1
    └─ VarDecl('points') → tab_index:39, type:array, lev:1
    └─ VarDecl('i') → tab_index:40, type:integer, lev:1
  └─ CompoundStmt → type:void
    └─ Assign(points[i] := 10) → type:void
      └─ ArrayAccess → type:integer
        └─ Var(points) → tab_index:39, type:array, lev:1
        └─ Var(i) → tab_index:40, type:integer, lev:1
      └─ Num(10) → type:integer

```

=== Semantic Analysis ===

Errors : 0
Warnings: 0
Status : OK

- Analisis:

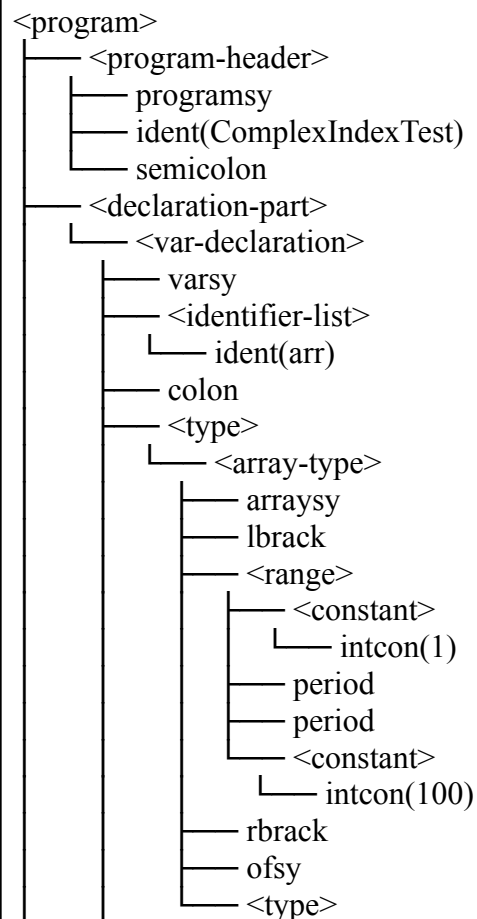
Program berhasil dianalisis tanpa error setelah revisi menghapus semicolon setelah field terakhir record. Point terdaftar di tab[38] sebagai TypeDecl, points di tab[39] bertipe array, dan i di tab[40] bertipe integer. ATAB mencatat array points dengan bound 1..5 dan ukuran 5. Akses points[i] dipetakan menjadi ArrayAccess(Var(points), Var(i)) dengan tipe hasil integer. Tipe Point muncul unknown karena resolusi tipe record belum sepenuhnya diimplementasikan di semantic analyzer.

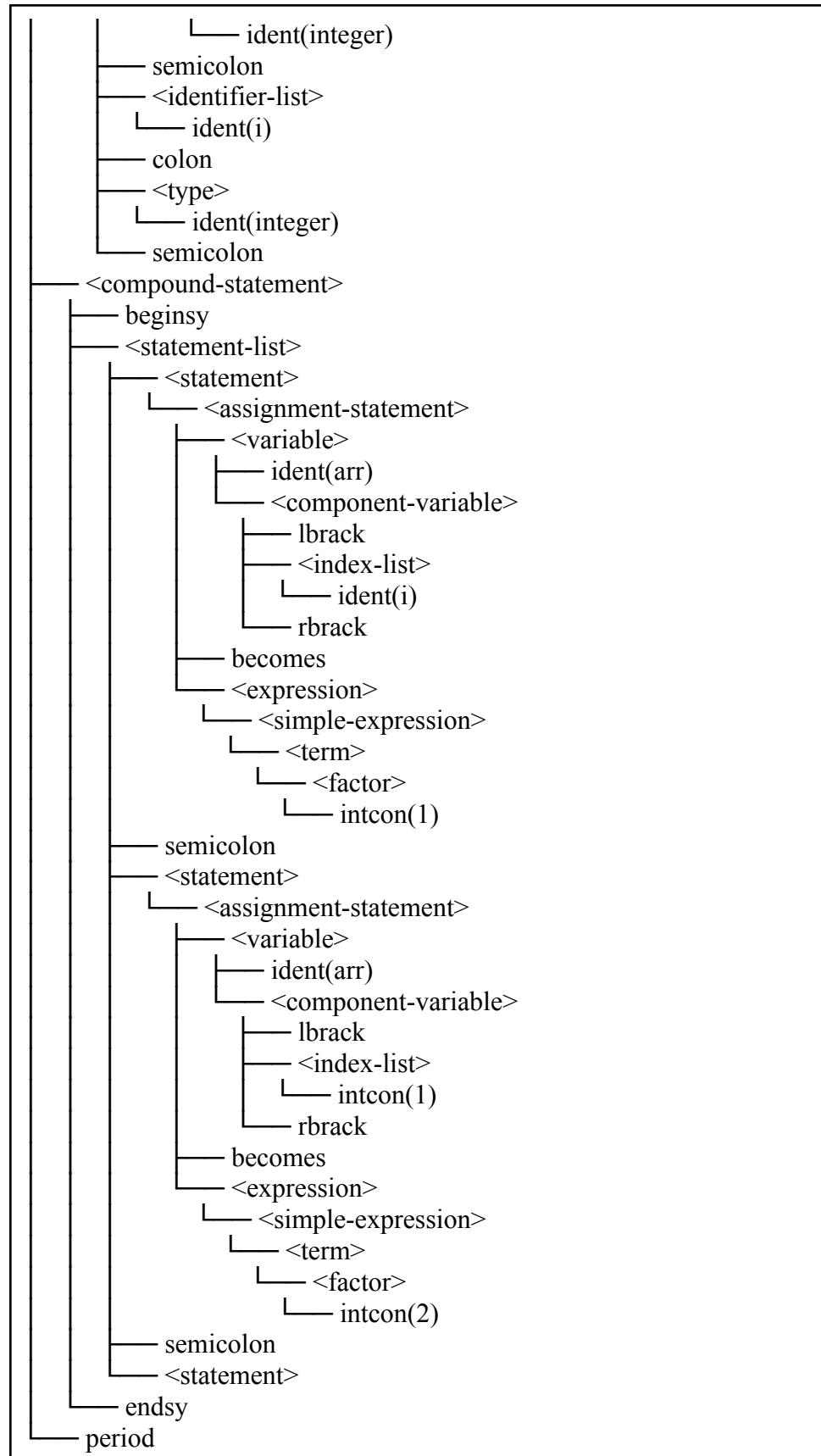
3.2.31. Kasus Uji 31

- Kasus: Array Index dengan Literal Integer dan Variable
- Tujuan: Memverifikasi bahwa array dapat diakses dengan index berupa literal integer (intcon) maupun variable (ident) dan keduanya dipetakan ke node yang benar sebagai child ArrayAccessNode
- Input:

```
program ComplexIndexTest;  
var  
  arr: array[1..100] of integer;  
  i: integer;  
begin  
  arr[i] := 1;  
  arr[1] := 2;  
end.
```

- Output:





=== Symbol Table ===

TAB (Identifier Table)

idx	name	obj	type	ref	nrm	lev	adr	link

1	AND	keyword	Unknown	0	1	0	0	0
2	ARRAY	keyword	Unknown	0	1	0	0	1
3	BEGIN	keyword	Unknown	0	1	0	0	2
4	CASE	keyword	Unknown	0	1	0	0	3
5	CONST	keyword	Unknown	0	1	0	0	4
6	DIV	keyword	Unknown	0	1	0	0	5
7	DOWNT0	keyword	Unknown	0	1	0	0	6
8	DO	keyword	Unknown	0	1	0	0	7
9	ELSE	keyword	Unknown	0	1	0	0	8
10	END	keyword	Unknown	0	1	0	0	9
11	FOR	keyword	Unknown	0	1	0	0	10
12	FUNCTION	keyword	Unknown	0	1	0	0	11
13	IF	keyword	Unknown	0	1	0	0	12
14	MOD	keyword	Unknown	0	1	0	0	13
15	NOT	keyword	Unknown	0	1	0	0	14
16	OF	keyword	Unknown	0	1	0	0	15
17	OR	keyword	Unknown	0	1	0	0	16
18	PROCEDURE	keyword	Unknown	0	1	0	0	17
19	PROGRAM	keyword	Unknown	0	1	0	0	18
20	RECORD	keyword	Unknown	0	1	0	0	19
21	REPEAT	keyword	Unknown	0	1	0	0	20
22	INTEGER	type	Integer	0	1	0	0	21
23	REAL	type	Real	0	1	0	0	22
24	BOOLEAN	type	Boolean	0	1	0	0	23
25	CHAR	type	Char	0	1	0	0	24
26	STRING	type	String	0	1	0	0	25
27	THEN	keyword	Unknown	0	1	0	0	26
28	TO	keyword	Unknown	0	1	0	0	27
29	TYPE	keyword	Unknown	0	1	0	0	28
30	UNTIL	keyword	Unknown	0	1	0	0	29
31	VAR	keyword	Unknown	0	1	0	0	30
32	WHILE	keyword	Unknown	0	1	0	0	31
33	true	constant	Boolean	0	1	0	1	32
34	false	constant	Boolean	0	1	0	0	33
35	writeln	procedure	Void	0	1	0	0	34
36	readln	procedure	Void	0	1	0	0	35
37	ComplexIndexTestprogram		Void	1	1	0	0	36
38	arr	variable	Array	0	1	1	0	0
39	i	variable	Integer	0	1	1	100	38

BTAB (Block Table)				
idx	last	lpar	psze	vsze
0	37	0	0	0
1	39	0	0	101

ATAB (Array Table)							
idx	xtyp	etyp	eref	low	high	elsz	size
0	Integer	Integer	0	1	100	1	100

=== Semantic Analysis ===

Errors : 0
Warnings: 0
Status : OK

=== Decorated AST ===

ProgramNode(name: 'ComplexIndexTest') → tab_index:37,
type:void, lev:0

- Declarations
 - VarDecl('arr') → tab_index:38, type:array, lev:1
 - VarDecl('i') → tab_index:39, type:integer, lev:1
- CompoundStmt → type:void
 - Assign(arr[i] := 1) → type:void
 - ArrayAccess → type:integer
 - Var(arr) → tab_index:38, type:array, lev:1
 - Var(i) → tab_index:39, type:integer, lev:1
 - Num(1) → type:integer
 - Assign(arr[1] := 2) → type:void
 - ArrayAccess → type:integer
 - Var(arr) → tab_index:38, type:array, lev:1
 - Num(1) → type:integer
 - Num(2) → type:integer

=== Semantic Analysis ===

Errors : 0
Warnings: 0
Status : OK

- Analisis:
Program berhasil dianalisis tanpa error. Array dengan index literal integer arr[1] dan index variable arr[i] keduanya dipetakan dengan benar. Index literal dipetakan ke NumberNode dan index variable dipetakan ke VarNode sebagai child ArrayAccessNode.

BAB IV

KESIMPULAN DAN SARAN

4.1.Kesimpulan

Berdasarkan perancangan, implementasi, dan pengujian yang telah dilakukan pada Milestone 3, dapat ditarik beberapa kesimpulan sebagai berikut:

- Program ... (semantic analyzer) untuk bahasa pemrograman Arion telah berhasil diimplementasikan menggunakan bahasa pemrograman C++. Implementasi ini dibangun sesuai dengan spesifikasi yang diminta.
- Program telah diimplementasikan dengan struktur Symbol Table dan Abstract Syntax Tree (Decorated). Proses pembuatan dan pendekorasi (via semantic analyzer) Abstract Syntax Tree (AST) dilakukan melalui transversal DFS pada Abstract Syntax Tree (AST) menggunakan fungsi visit untuk setiap jenis node. Sedangkan Symbol Table digunakan sebagai referensi dalam menentukan deklarasi dan identifier. Symbol Table berisi nama, jenis objek, tipe data, lexical level, dan alamat memori.
- Semantic Analyzer mampu menerima hasil Parser dari Milestone 2 dalam bentuk ParseTree. Program dapat melakukan transformasi dan membangun struktur Abstract Syntax Tree (AST) yang sesuai. Program kemudian menambahkan keterangan yang didapat selama proses transversal untuk membentuk Decorated Abstract Syntax Tree (AST).

4.2.Saran

Untuk pengembangan selanjutnya, terutama untuk milestone berikutnya, terdapat beberapa saran yang dapat diterapkan:

- Saat ini pesan error semantik yang dihasilkan belum menyertakan informasi posisi baris dan kolom secara konsisten karena SourceLocation selalu diisi dengan nilai default (0, 0). Ke depannya, informasi posisi dapat diteruskan dari node AST ke setiap pemanggilan fungsi error agar pesan yang dihasilkan lebih informatif dan mempermudah proses debugging bagi pengguna.
- Implementasi semantic analyzer saat ini menghentikan analisis pada beberapa titik kritis seperti saat root bukan ProgramNode. Untuk milestone selanjutnya,

dapat dipertimbangkan penambahan mekanisme error recovery yang lebih menyeluruh, misalnya dengan melanjutkan traversal AST meskipun suatu node menghasilkan `DataType::UNKNOWN`, sehingga semantic analyzer dapat melaporkan sebanyak mungkin error dalam satu kali proses analisis.

- Struktur program yang sudah terpisah berdasarkan tanggung jawabnya (`semanticAnalyzer.h/.cpp`, `TypeSystem.h/.cpp`, `ErrorHandler.h/.cpp`, `symbolTable.hpp/.cpp`) harus terus dijaga dan ditingkatkan pemisahannya. Modularitas ini sangat disarankan untuk mempermudah integrasi dengan tahapan kompilasi selanjutnya, terutama intermediate code generation yang akan membutuhkan akses langsung ke Decorated AST dan Symbol Table yang dihasilkan pada milestone ini.
- Pembagian tugas pengujian perlu didefinisikan lebih eksplisit sejak awal milestone. Pada milestone ini, test case baru dikerjakan bersama di akhir mendekati deadline sehingga cakupan pengujian menjadi kurang merata. Untuk milestone selanjutnya, setiap anggota sebaiknya bertanggung jawab atas minimal satu test case yang spesifik menguji bagian yang ia kerjakan, misalnya Orang 4 menguji seluruh skenario deklarasi dan Orang 5 menguji seluruh skenario statement dan ekspresi, sehingga integrasi dan validasi dapat dilakukan lebih awal dan lebih sistematis.

DAFTAR PUSTAKA

- Spivak, R. (2015, December 15). Let's Build A Simple Interpreter. Part 7: Abstract Syntax Trees. Ruslan's Blog. Retrieved from <https://ruslanspivak.com/lsbasi-part7>
- Crafting Interpreters. (n.d.). Parsing Expressions. Retrieved from <https://craftinginterpreters.com/parsing-expressions.html>
- Naufarrel. (2025, December 22). Pengenalan Context-Free Grammar (CFG) dan Derivasi. IF-Notes. Retrieved from [https://if-notes.naufarrel.dev/02.-Catatan/IF2224-Teori-Bahasa-Formal-dan-Otomata/UAS/Pengenalan-Context-Free-Grammar-\(CFG\)-dan-Derivasi](https://if-notes.naufarrel.dev/02.-Catatan/IF2224-Teori-Bahasa-Formal-dan-Otomata/UAS/Pengenalan-Context-Free-Grammar-(CFG)-dan-Derivasi)

LAMPIRAN

Link Release Milestone 3 Github

Seluruh kode sumber program dapat diakses melalui release dari repository berikut.

<https://github.com/andraalanna/GTW-Tubes-IF2224-2026>

Pembagian Tugas

NIM	Nama	Tugas	Kontribusi
13524047	Muhammad Jordan Ferimeison	• Integrasi • Buat fungsi visit(AssignNode), visit(BinOpNode), visit(VarNode), visit(IfNode), visit(WhileNode), visit(ForNode), visit(ProcCallNode) • Melakukan testing	20%
13524049	Arina Azka	• Mengerjakan AST Node • Mentranslasi parse tree ke AST • Melakukan testing • Mengerjakan type system dan error handler • Menyusun landasan teori Perbedaan Parse Tree dan Abstract Syntax Tree serta Syntax-Directed Translation & Semantic Action	20%
13524075	Hakam Avicena Mustain	• Menguji type system • Menguji error handler • Membuat landasan teori terkait type system,type compatibility, error handling, dan predefined identifier • Melakukan testing	20%

13524077	Yavie Azka Putra Araly	• Buat perancangan dan implementasi (scope management, alur kerja program) pada laporan • Mengerjakan fungsi visit(ProgramNode), visit(VarDeclNode), visit(ConstDeclNode), visit(TypeDeclNode), visit(ProcDeclNode), visit(FuncDeclNode) • Melakukan testing	20%
13524079	Angelina Andra Alana	• Mengerjakan SymbolTable • Mengerjakan Landasan Teori Sematic Analysis, AST, DecoratedAST, Atribute Grammar & SDT, SymbolTable, Scope Resolution, Visitor Pattern. • Melakukan testing • Memperbaiki Kesalahan yang terdapat pada Milestone sebelumnya	20%